

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



HCMUTE

ĐỒ ÁN TỐT NGHIỆP
THIẾT KẾ VÀ ĐÁNH GIÁ BỘ TẠO SỐ NGẪU
NHIÊN CHO ỨNG DỤNG MẬT MÃ TRÊN NỀN
TẢNG FPGA

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

SVTH: Ngô Đặng Hoàng Vũ MSSV: 22119258

Lê Trung Vĩnh MSSV: 22119257

GVHD: PGS.TS PHAN VĂN CA

TP. HỒ CHÍ MINH – 06/2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



HCMUTE

ĐỒ ÁN TỐT NGHIỆP
THIẾT KẾ VÀ ĐÁNH GIÁ BỘ TẠO SỐ NGẪU
NHIÊN CHO ỨNG DỤNG MẬT MÃ TRÊN NỀN
TẢNG FPGA

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

SVTH: Ngô Đặng Hoàng Vũ MSSV: 22119258

Lê Trung Vĩnh MSSV: 22119257

GVHD: PGS.TS PHAN VĂN CA

TP. HỒ CHÍ MINH – 06/2026

THÔNG TIN KHÓA LUẬN TỐT NGHIỆP

1. Thông tin sinh viên

Họ và tên sinh viên: Ngô Đặng Hoàng Vũ MSSV: 22119258

Email: ngovu8212@gmail.com

Họ và tên sinh viên: Lê Trung Vĩnh MSSV: 22119257

Email: vinhle.2064@gmail.com

2. Thông tin đề tài

- Tên đề tài: THIẾT KẾ VÀ ĐÁNH GIÁ BỘ TẠO SỐ NGẪU NHIÊN CHO ỨNG DỤNG MẬT MÃ TRÊN NỀN TẢNG FPGA

- Đơn vị quản lý: Bộ môn Kỹ Thuật Máy Tính – Viễn Thông, Khoa Điện Điện Tử, Trường Đại Học Công Nghệ Kỹ Thuật Tp. Hồ Chí Minh.

- Thời gian thực hiện: Từ ngày / / 20... đến ngày / / 20..

- Thời gian bảo vệ trước hội đồng: Ngày / / 20..

3. Lời cam đoan của sinh viên

Chúng tôi Ngô Đặng Hoàng Vũ và Lê Trung Vĩnh cam đoan KLTN là công trình nghiên cứu của nhóm tôi, dưới sự hướng dẫn của PGS.TS Phan Văn Ca. Kết quả công bố trong KLTN là trung thực và không sao chép từ bất kì công trình nào khác.

Tp. HCM, ngày ... tháng ... năm 20...

SV thực hiện đồ án

(Ký và ghi rõ họ tên)

Ngô Đặng Hoàng Vũ

Lê Trung Vĩnh

Giảng viên hướng dẫn xác nhận quyền báo cáo đã được chỉnh sửa theo đề nghị được ghi trong biên bản của Hội đồng đánh giá Khóa luận tốt nghiệp.

.....

Tp. HCM, ngày ... tháng ... năm 20..

Giảng viên hướng dẫn

(Ký, ghi rõ họ tên và học hàm – học vị)

Xác nhận của Bộ Môn

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP
(Dành cho giảng viên hướng dẫn)

Đề tài: Thiết kế và đánh giá bộ tạo số ngẫu nhiên cho ứng dụng
mật mã trên nền tảng FPGA

Sinh viên: + Ngô Đăng Hoàng Vũ

MSSV: 22119258

+ Lê Trung Vĩnh

MSSV: 22119257

Hướng dẫn: PGS.TS. Phạm Văn Ca

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề: ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể, đề tài có tính mới, cấp thiết, đề tài có khả năng ứng dụng, tính sáng tạo.

Đặt vấn đề, giải quyết vấn đề hợp lý.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

Phương pháp thực hiện, phân tích, thiết kế hợp lý.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá/ kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

Kết quả thực hiện phù hợp với mục tiêu.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

Kết luận phù hợp.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

Hình thức trình bày, bố cục báo cáo hợp lý.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

Kỹ năng thực hiện đề tài tốt.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

Tài liệu trích dẫn rõ ràng, nhiều hơn chỉ.

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

Đề tài thực hiện theo đề xuất của sinh viên.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

Không có.

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Thái độ học tập và nghiên cứu có bảo đảm, tiếp thu kiến thức.

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Đạt yêu cầu nộp phân bổ.

Tp. HCM, ngày ... tháng ... năm 20...

Người nhận xét
(Ký và ghi rõ họ tên)

Phạm Văn Ca

* Giáo viên hướng dẫn có thể ghi tiếp vào mặt sau

LỜI CẢM ƠN

Trước tiên, nhóm sinh viên xin gửi lời cảm ơn chân thành đến quý thầy, cô hiện đang là giảng viên của Khoa Điện – Điện Tử, Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh đã tận tình giảng dạy, tạo điều kiện và môi trường học tập thuận lợi, đồng thời cung cấp cho nhóm những kiến thức nền tảng cần thiết để hoàn thành đồ án tốt nghiệp này.

Đặc biệt, nhóm sinh viên xin bày tỏ lòng biết ơn sâu sắc đến PGS.TS Phan Văn Ca - người đã trực tiếp hướng dẫn nhóm trong suốt quá trình thực hiện đề tài. Thầy đã tận tình định hướng, góp ý và chỉ dẫn nhóm đi đúng hướng, giúp nhóm tháo gỡ những khó khăn về mặt kỹ thuật cũng như khắc phục những thiếu sót trong quá trình nghiên cứu và triển khai hệ thống MURO-TRNG trên nền tảng FPGA.

Nhóm sinh viên cũng xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn động viên, hỗ trợ và là nguồn động lực giúp nhóm vượt qua khó khăn để hoàn thành đồ án.

Do kiến thức và thời gian thực hiện còn hạn chế, đồ án khó tránh khỏi những thiếu sót. Nhóm sinh viên rất mong nhận được sự thông cảm cùng những ý kiến đóng góp quý báu từ quý thầy cô để đề tài được hoàn thiện hơn.

Nhóm sinh viên xin chân thành cảm ơn!

LỜI CAM ĐOAN

Nhóm sinh viên Ngô Đăng Hoàng Vũ và Lê Trung Vĩnh xin cam đoan rằng đề tài "Thiết kế và đánh giá bộ tạo số ngẫu nhiên cho ứng dụng mật mã trên nền tảng FPGA" là công trình nghiên cứu của nhóm, được thực hiện dưới sự hướng dẫn của PGS.TS Phan Văn Ca, và chưa từng được sử dụng để làm cơ sở cho bất kỳ luận văn, luận án hay công trình nghiên cứu nào khác.

Mọi trích dẫn, tham khảo trong luận văn đều được ghi rõ nguồn gốc và tôn trọng bản quyền tác giả. Các kết quả mô phỏng, triển khai phần cứng và kiểm thử được trình bày trong luận văn là trung thực và do chính nhóm thực hiện.

Nhóm sinh viên hoàn toàn chịu trách nhiệm trước các nội dung đã trình bày trong luận văn này.

Tp. Hồ Chí Minh, ngày 24 tháng 06 năm 2026

Sinh viên

(Ký và ghi rõ họ tên)

Ngô Đăng Hoàng Vũ Lê Trung Vĩnh

TÓM TẮT

Trong các hệ thống mật mã và an toàn thông tin hiện đại, bộ tạo số ngẫu nhiên thực (True Random Number Generator - TRNG) đóng vai trò nền tảng nhằm cung cấp nguồn entropy không thể dự đoán cho quá trình sinh khóa và xác thực. Đề tài tập trung nghiên cứu, thiết kế, hiện thực hóa và đánh giá toàn diện bộ tạo số ngẫu nhiên thực theo kiến trúc nhiều vòng dao động (Multiple Ring Oscillator TRNG - MURO-TRNG) trên nền tảng FPGA PYNQ-Z1 (Xilinx Zynq-7000 XC7Z020). Hệ thống khai thác nhiễu jitter pha sinh ra từ các vòng dao động dựa trên kiến trúc vòng khóa pha toàn số (ADPLL) làm nguồn entropy vật lý, hướng tới mục tiêu tối ưu hóa tài nguyên phần cứng, tiết kiệm năng lượng và đáp ứng tiêu chuẩn quốc tế NIST SP 800-22.

Toàn bộ hệ thống được mô tả bằng ngôn ngữ Verilog và triển khai trên Xilinx Vivado 2025.2. Lõi MURO-TRNG gồm mười vòng dao động ADPLL-based hoạt động ở các tần số tham chiếu khác nhau làm nguồn sinh entropy, một bộ ADPLL truyền thống tạo xung nhịp lấy mẫu đồng bộ, các bộ DFF lấy mẫu kết hợp mạng XOR để trộn entropy thành chuỗi bit ngẫu nhiên thô, và khối hậu xử lý XOR corrector nhằm khử thiên lệch bias. Để phục vụ thu thập và kiểm thử, nhóm sinh viên xây dựng một pipeline truyền dữ liệu hoàn chỉnh theo chuỗi MURO-TRNG Core (PL) → AXI-Stream FIFO → AXI DMA → DDR RAM → ARM Cortex-A9 (PS) → UDP/Gigabit Ethernet → PC. Trong quá trình triển khai, nhóm sinh viên đã phát hiện và khắc phục một lỗi kiến trúc trong thiết kế DCO của công trình gốc, nguyên nhân khiến vòng dao động bị khóa theo xung nhịp thay vì chạy tự do và không tích lũy được jitter thực sự, bằng cách tách vòng phản hồi NOR thành tín hiệu tổ hợp tự do ro_raw kết hợp bộ đồng bộ hai tầng (two-stage synchronizer) để xử lý vượt miền xung nhịp.

Kết quả thực nghiệm cho thấy hệ thống sau cải tiến đã vượt qua các phép thử khắt khe của bộ công cụ tiêu chuẩn quốc tế NIST SP 800-22 tại nhiều cấu hình tần số tham chiếu khác nhau, đồng thời đảm bảo tỷ lệ phân bố bit cân bằng lý tưởng. Đề tài cũng chứng minh lõi MURO-TRNG đạt được hiệu suất bằng

thông dữ liệu cao, tối ưu vượt trội về mặt diện tích sử dụng tài nguyên logic và duy trì mức tiêu thụ năng lượng thấp trên chip. Các kết quả trên khẳng định tính đúng đắn của kiến trúc cải tiến và chứng minh khả năng ứng dụng thực tiễn cao của hệ thống như một lõi IP độc lập, sẵn sàng tích hợp trong các cấu trúc vi mạch nhúng bảo mật (SoC).

MỤC LỤC

	Trang
LỜI CẢM ƠN.....	i
LỜI CAM ĐOAN.....	ii
TÓM TẮT.....	iii
MỤC LỤC.....	v
DANH MỤC CÁC TỪ VIẾT TẮT.....	ix
DANH MỤC HÌNH ẢNH.....	xii
DANH MỤC BẢNG BIỂU.....	xiv
CHƯƠNG 1 GIỚI THIỆU.....	1
1.1 GIỚI THIỆU.....	1
1.2 LÝ DO CHỌN ĐỀ TÀI.....	2
1.3 MỤC TIÊU ĐỀ TÀI.....	2
1.4 ĐỐI TƯỢNG NGHIÊN CỨU.....	3
1.5 PHẠM VI NGHIÊN CỨU.....	3
1.6 BỐ CỤC ĐỀ TÀI.....	4
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT.....	5
2.1 LÝ THUYẾT VỀ SINH SỐ NGẪU NHIÊN.....	5
2.1.1 Số ngẫu nhiên giả.....	5
2.1.2 Số ngẫu nhiên thực.....	7
2.2 CÁC KIẾN TRÚC TRNG TRÊN FPGA.....	8
2.2.1 Jitter TRNG.....	9
2.2.2 Metastability TRNG.....	12
2.3 VÒNG KHÓA PHA VÀ BỘ DAO ĐỘNG ĐIỀU KHIỂN SỐ.....	12

2.3.1 Hạn chế của PLL analog và ưu điểm của ADPLL	12
2.3.2 Kiến trúc ADPLL	13
2.3.3 Cấu trúc DCO dựa trên chuỗi cổng NOR	15
2.4 HẬU XỬ LÝ XOR CORRECTOR	16
2.5 CHUẨN KIỂM THỬ NIST SP800-22	17
2.5.1 Các phép thử trong NIST SP800-22	17
2.5.2 Tiêu chí đánh giá	18
2.6 TỔNG QUAN NỀN TẢNG PHẦN CỨNG VÀ GIAO TIẾP	19
2.6.1 Kiến trúc Zynq-7000 và bo mạch PYNQ-Z1	19
2.6.2 Giao tiếp PL và PS qua AXI DMA	20
2.6.3 Giao thức UDP và Gigabit Ethernet.....	22
CHƯƠNG 3 TÍNH TOÁN VÀ THIẾT KẾ HỆ THỐNG	23
3.1. YÊU CẦU HỆ THỐNG.....	23
3.2 THIẾT KẾ HỆ THỐNG.....	24
3.2.1 Sơ đồ khối hệ thống	24
3.2.2 Lưu đồ hoạt động của hệ thống.....	25
3.3 THIẾT KẾ TỪNG KHỐI.....	27
3.3.1 Kiến trúc TRNG.....	27
3.3.2 Khối tạo Entropy	28
3.3.3. Khối lấy mẫu và tạo bit ngẫu nhiên thô	32
3.3.4. Khối xử lý hậu kỳ	37
3.3.5. Khối thu thập dữ liệu	38
CHƯƠNG 4 KẾT QUẢ THỰC HIỆN	41
4.1 KẾT QUẢ THIẾT KẾ HỆ THỐNG	41

4.1.1 Tổng quan kiến trúc của hệ thống và pipeline thu thập dữ liệu	41
4.1.2 Ràng buộc vị trí và bảo toàn nguồn entropy	45
4.2 KẾT QUẢ MÔ PHỎNG TỪNG MODULE	47
4.2.1 Phase_detector	47
4.2.2 K_counter	47
4.2.3 DCO	48
4.2.4 ADPLL_RO	48
4.2.5 DCO_conventional	50
4.2.6 ADPLL_conventional	51
4.2.7 Xor_corrector	54
4.2.8 Clock_gen	55
4.2.9 MURO-TRNG	56
4.3 KẾT QUẢ TỔNG HỢP	59
4.3.1 Phát hiện lỗi kiến trúc trong DCO	59
4.3.2 Giải pháp cải tiến	60
4.3.3 Kết quả sau khi áp dụng cải tiến	61
4.4 ĐÁNH GIÁ HỆ THỐNG	65
4.4.1 Theo nhiệt độ và điện áp	66
4.4.2 Theo tần số tham chiếu	66
4.4.3 Đánh giá hệ thống đang thực hiện so với các hệ thống đã được công bố	68
4.4.4 Đánh giá tổng tài nguyên phần cứng	71
4.5 TÓM TẮT CHƯƠNG 4	73
CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	76

5.1 KẾT LUẬN.....	76
5.2 HƯỚNG PHÁT TRIỂN.....	77
TÀI LIỆU THAM KHẢO	78

DANH MỤC CÁC TỪ VIẾT TẮT

Viết tắt	Mô tả
ADC	Analog-to-Digital Converter
ADPLL	All Digital Phase Locked Loop
AMBA	Advanced Microcontroller Bus Architecture
AXI	Advanced eXtensible Interface
BRAM	Block Random Access Memory
CDC	Clock Domain Crossing
CLB	Configurable Logic Block
CPU	Central Processing Unit
DCO	Digital Controlled Oscillator
DDR	Double Data Rate
DFF	D Flip-Flop
DFT	Discrete Fourier Transform
DMA	Direct Memory Access
DSP	Digital Signal Processing
FF	Flip-Flop
FIFO	First In First Out
FPGA	Field Programmable Gate Array
IoT	Internet of Things

Viết tắt	Mô tả
IP	Intellectual Property (core)
LSB	Least Significant Bit
LUT	Look-Up Table
MSB	Most Significant Bit
MURO	Multiple Ring Oscillator
NIST	National Institute of Standards and Technology
PD	Phase Detector
PL	Programmable Logic
PLL	Phase Locked Loop
PRNG	Pseudo Random Number Generator
PS	Processing System
RO	Ring Oscillator
RTL	Register Transfer Level
S2MM	Stream to Memory-Mapped
SoC	System on Chip
SP	Special Publication
STS	Statistical Test Suite
T-FF	Toggle Flip-Flop
TRNG	True Random Number Generator

Viết tắt	Mô tả
UDP	User Datagram Protocol
XDC	Xilinx Design Constraints
XOR	Exclusive OR
PVT	Process–Voltage–Temperature

DANH MỤC HÌNH ẢNH

Hình 2.1: Cấu trúc Ring Oscillator cơ bản.....	9
Hình 2.2: Sự biến thiên thời gian Jitter	10
Hình 2.3: Sơ đồ của PLL Analog cơ bản	13
Hình 2.4: Sơ đồ khối kiến trúc ADPLL	14
Hình 2.5: Cấu trúc DCO dựa trên chuỗi cổng NOR	15
Hình 2.6: Cấu trúc XOR Corrector	17
Hình 2.7: Kiến trúc tổng quan Zynq-7000	20
Hình 2.8: Kiến trúc tổng quan của AXI4 protocol.....	21
Hình 3.1: Sơ đồ khối hệ thống	24
Hình 3.2: Lưu đồ hoạt động của hệ thống TRNG.....	26
Hình 3.3: Kiến trúc tổng thể của lõi MURO-TRNG	28
Hình 3.4: Khối tạo entropy sử dụng trong thiết kế	29
Hình 3.5: Bộ đếm k_counter 2 chế độ up và down.....	29
Hình 3.6: Cấu trúc DCO trong ADPLL	30
Hình 3.7: T-Flip Flop trong Bộ đếm chia N/2	30
Hình 3.8: Khối tạo tần số lấy mẫu theo kiến trúc ADPLL truyền thống ..	33
Hình 3.9: Cấu trúc K-counter 2 chế độ up và down	34
Hình 3.10: Cấu tạo DCO truyền thống	34
Hình 3.11: Cấu tạo của bộ chia N	35
Hình 3.12: Kiến trúc khối lấy mẫu và tạo Raw Random Bit	36
Hình 3.13: Khối tạo xử lý hậu kỳ sử dụng trong thiết kế	38
Hình 3.14: Kiến trúc khối thu thập dữ liệu	39
Hình 4.1: Pipeline thu thập dữ liệu	42

Hình 4.2: Vivado Block Design – toàn bộ hệ thống MURO-TRNG trên PYNQ-Z1	43
Hình 4.3: Schematic tổng quan của khối MURO-TRNG core	45
Hình 4.4: Vị trí trên Vivado – các RO được đặt tách biệt bằng PBLOCK	46
Hình 4.5: Behavioral simulation waveform – module phase_detector.....	47
Hình 4.6: Behavioral simulation waveform – module k_counter.....	47
Hình 4.7: Behavioral simulation waveform – module dco	48
Hình 4.8: Behavioral simulation waveform – module adpll_ring_osc	49
Hình 4.9: Behavioral simulation waveform – module dco_conventional	50
Hình 4.10: Behavioral simulation waveform – module conventional_adpll (fo = 6.25 MHz)	51
Hình 4.11: Behavioral simulation waveform – module conventional_adpll (fo = 12.5 MHz)	53
Hình 4.12: Behavioral simulation waveform – module conventional_adpll (fo = 25 MHz)	53
Hình 4.13: Behavioral simulation waveform – module xor_corrector.....	54
Hình 4.14: Behavioral simulation waveform – module clock_gen	55
Hình 4.15: Behavioral simulation waveform – module MURO-TRNG top-level (tb_muro_trng_top)	57
Hình 4.16: Kiến trúc DCO theo thiết kế gốc của paper [1] – DFF nằm trong feedback loop, clock $2Nf_0$	60
Hình 4.17: Kiến trúc DCO sau cải tiến	61

DANH MỤC BẢNG BIỂU

Bảng 3.1: Tổng hợp tần số tham chiếu 10 bộ dao động và thông số liên quan	31
Bảng 3.2: Tần số tham chiếu và thông số liên quan	35
Bảng 4.1: Tỷ lệ bit 0/1 trước và sau cải tiến kiến trúc DCO	62
Bảng 4.2: Kết quả NIST SP800-22 – MURO-TRNG tại $f_0 = 6,25$ MHz. 62	
Bảng 4.3: Kết quả NIST SP800-22 – MURO-TRNG tại $f_0 = 12,5$ MHz. 63	
Bảng 4.4: Kết quả NIST SP800-22 – MURO-TRNG tại $f_0 = 25$ MHz.... 64	
Bảng 4.5: Công suất hệ thống theo nhiệt độ và điện áp VDD	66
Bảng 4.6: Kết quả timing, throughput, công suất theo tần số tham chiếu 67	
Bảng 4.7: So sánh hệ thống MURO-TRNG Chúng tôi với kiến trúc gốc và các công trình liên quan	68
Bảng 4.8: Tài nguyên phần cứng từng thành phần – MURO-TRNG trên PYNQ-Z1 (XC7Z020)	71
Bảng 4.9: Bảng tóm tắt các test case.....	73

CHƯƠNG 1 GIỚI THIỆU

1.1 GIỚI THIỆU

Trong kỷ nguyên chuyển đổi số, bảo mật thông tin đã trở thành một trong những thách thức cốt lõi của các hệ thống kỹ thuật số hiện đại. Từ các giao dịch tài chính, hệ thống IoT đến các thiết bị nhúng – tất cả đều dựa vào các cơ chế mã hoá để đảm bảo tính bí mật và toàn vẹn của dữ liệu. Nền tảng của mọi hệ thống mật mã học chính là khả năng sinh ra các chuỗi số ngẫu nhiên chất lượng cao, không thể dự đoán và không thể tái tạo.

Số ngẫu nhiên được phân thành hai loại: số ngẫu nhiên giả (Pseudo-Random Number Generator – PRNG) và số ngẫu nhiên thực sự (True Random Number Generator – TRNG). PRNG sử dụng thuật toán toán học để tạo ra chuỗi số có tính ngẫu nhiên về mặt thống kê, tuy nhiên bản chất xác định của chúng khiến chuỗi này có thể bị tái tạo nếu biết trạng thái khởi đầu. Trái lại, TRNG khai thác các hiện tượng vật lý không xác định như nhiễu nhiệt hay jitter trong mạch điện tử để tạo ra nguồn entropy thực sự, đảm bảo tính ngẫu nhiên hoàn toàn không thể đoán trước.

Hiện nay, việc hiện thực hóa các bộ TRNG bằng kỹ thuật số thuần túy trên nền tảng FPGA (Field Programmable Gate Array) đang trở thành một xu hướng nghiên cứu quan trọng. FPGA mang lại khả năng lập trình linh hoạt, tốc độ xử lý dữ liệu cao và cho phép tích hợp trực tiếp vào các hệ thống nhúng với cấu trúc ít phức tạp hơn đáng kể so với các giải pháp sử dụng mạch tương tự. Tuy nhiên, thách thức lớn nhất đặt ra cho các nhà thiết kế là làm thế nào để xây dựng được một kiến trúc TRNG không chỉ đảm bảo tính ngẫu nhiên tuyệt đối mà còn phải tối ưu hóa về mặt tài nguyên phần cứng và hiệu suất năng lượng.

Xuất phát từ nhu cầu đó, đề tài tập trung vào việc nghiên cứu và triển khai một kiến trúc TRNG hiệu quả trên FPGA, tận dụng các ưu điểm của kỹ thuật số hiện đại để tạo ra một nguồn entropy đáng tin cậy, góp phần nâng cao năng lực bảo mật cho các hệ thống kỹ thuật số hiện nay.

1.2 LÝ DO CHỌN ĐỀ TÀI

Trong các hệ thống an ninh hiện đại, việc bảo vệ thông tin phụ thuộc hoàn toàn vào độ an toàn của các khóa mã hóa. Để các khóa này không thể bị bẻ gãy, hệ thống cần một nguồn tạo ra các chuỗi số ngẫu nhiên thực sự, đảm bảo tính bất định và không bao giờ lặp lại.

Tuy nhiên, việc tạo ra số ngẫu nhiên thực ngay trên các chip xử lý mà vẫn đảm bảo tốc độ nhanh và không làm tiêu tốn quá nhiều diện tích mạch điện là một thách thức lớn trong thiết kế vi mạch. Các giải pháp truyền thống thường gặp khó khăn trong việc cân bằng giữa chất lượng độ ngẫu nhiên và hiệu suất hoạt động của hệ thống.

Vì vậy, đề tài này chọn hướng nghiên cứu triển khai kiến trúc MURO-TRNG (Multiple Ring Oscillator TRNG) – bộ sinh số ngẫu nhiên dựa trên các vòng dao động kỹ thuật số thuần túy – trên nền tảng FPGA PYNQ-Z1. Giải pháp này giúp tận dụng tối đa các đặc tính vật lý sẵn có bên trong chip để tạo ra nguồn dữ liệu bảo mật cao, đồng thời giúp hệ thống vận hành ổn định và dễ dàng tích hợp vào các thiết bị thực tế như hệ thống IoT hay các thiết bị truyền thông. Việc thực hiện đề tài sẽ giúp kiểm chứng khả năng ứng dụng thực tế của kiến trúc này, hướng tới xây dựng các hệ thống bảo mật tự chủ và hiệu quả.

1.3 MỤC TIÊU ĐỀ TÀI

Mục tiêu nghiên cứu của đề tài “Thiết kế và đánh giá bộ sinh số ngẫu nhiên thực dựa trên kiến trúc nhiều vòng dao động (MURO-TRNG) sử dụng ADPLL trên FPGA” là thiết kế, hiện thực hóa và đánh giá toàn diện một lõi IP có khả năng sinh chuỗi bit ngẫu nhiên chất lượng cao, tối ưu về diện tích phần cứng và hiệu suất năng lượng. Hệ thống này khai thác nhiễu jitter pha từ các vòng dao động kỹ thuật số được điều khiển bởi ADPLL để tạo ra nguồn entropy thực sự, sau đó trải qua quá trình xử lý hậu kỳ bằng mạch XOR corrector nhằm cung cấp các chuỗi bit bất định, không thể dự đoán, phục vụ cho nhu cầu bảo mật và mã hóa trong các ứng dụng IoT, truyền thông không dây và các hệ thống nhúng hiện đại.

1.4 ĐỐI TƯỢNG NGHIÊN CỨU

Đề tài tập trung nghiên cứu các thành phần kỹ thuật cấu thành nên kiến trúc MURO-TRNG và các tiêu chuẩn đánh giá liên quan, bao gồm:

Kiến trúc ADPLL (All Digital Phase Locked Loop) và bộ DCO (Digital Controlled Oscillator) xây dựng từ chuỗi cổng NOR – thành phần cốt lõi tạo ra nguồn entropy jitter pha.

Mười vòng dao động ADPLL-based ring oscillator hoạt động ở các tần số tham chiếu khác nhau, đóng vai trò nguồn entropy chính của hệ thống.

Conventional ADPLL với chức năng sinh sampling clock phục vụ quá trình lấy mẫu đồng bộ tín hiệu ngẫu nhiên từ các ring oscillator.

Khối xử lý XOR corrector nhằm giảm thiểu bias trong bitstream ngẫu nhiên đầu ra.

Chuẩn kiểm thử NIST SP800-22 – tiêu chuẩn quốc tế đánh giá chất lượng ngẫu nhiên của bitstream.

1.5 PHẠM VI NGHIÊN CỨU

Đề tài tập trung vào thiết kế, mô phỏng, triển khai và đánh giá hệ thống MURO-TRNG trên PYNQ-Z1. Phạm vi nghiên cứu bao gồm:

Thiết kế toàn bộ RTL bằng ngôn ngữ Verilog và thực hiện synthesis, implementation trên Xilinx Vivado 2025.2.

Mô phỏng behavioral từng module nhằm xác nhận hoạt động đúng về mặt logic trước khi triển khai phần cứng.

Triển khai và kiểm thử hệ thống trên PYNQ-Z1 (XC7Z020), áp dụng constraints để bảo toàn đặc tính vật lý của ring oscillator.

Xây dựng và vận hành pipeline thu thập dữ liệu theo chuỗi MURO-TRNG Core (PL) → AXI-Stream FIFO → AXI DMA → DDR RAM → ARM Cortex-A9 (PS) → UDP/Gigabit Ethernet → PC.

Đánh giá chất lượng ngẫu nhiên theo chuẩn NIST SP800-22 và các thông số kỹ thuật: timing, throughput, công suất, energy/bit.

Khảo sát sự ổn định công suất theo nhiệt độ môi trường và điện áp trong dải hoạt động của chip XC7Z020 Commercial Grade.

Nghiên cứu không bao gồm việc chuyển đổi sang quy trình thiết kế ASIC, chế tạo chip thực tế, hoặc xây dựng một giao thức mật mã học hoàn chỉnh dựa trên hệ thống TRNG được phát triển.

1.6 BỐ CỤC ĐỀ TÀI

Nội dung của đề tài được trình bày trong 5 chương như sau:

Chương 1 GIỚI THIỆU: Trình bày bối cảnh nghiên cứu, lý do chọn đề tài, mục tiêu, đối tượng và phạm vi nghiên cứu, cùng bố cục tổng thể của quyển báo cáo.

Chương 2 CƠ SỞ LÝ THUYẾT: Giới thiệu nền tảng lý thuyết về TRNG, phân loại các kiến trúc TRNG trên FPGA, nguyên lý hoạt động của ADPLL, DCO và ring oscillator, cùng tổng quan về chuẩn kiểm thử NIST SP800-22.

Chương 3 TÍNH TOÁN VÀ THIẾT KẾ HỆ THỐNG: Trình bày kiến trúc tổng thể của hệ thống MURO-TRNG, thiết kế chi tiết từng module và pipeline thu thập dữ liệu theo chuỗi MURO-TRNG Core (PL) → AXI-Stream FIFO → AXI DMA → DDR RAM → ARM Cortex-A9 (PS) → UDP/Gigabit Ethernet → PC.

Chương 4 KẾT QUẢ THỰC HIỆN: Trình bày kết quả mô phỏng từng module, kết quả triển khai phần cứng và vận hành pipeline trên PYNQ-Z1, đánh giá timing, throughput, công suất và kết quả kiểm thử NIST SP800-22, đồng thời phân tích so sánh với các công trình liên quan.

Chương 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN: Tổng kết các kết quả đạt được, rút ra kết luận về tính đúng đắn và hiệu năng của hệ thống, đồng thời đề xuất các hướng nghiên cứu và ứng dụng tiếp theo.

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

Chương này trình bày nền tảng lý thuyết phục vụ cho đề tài "Thiết kế và đánh giá bộ tạo số ngẫu nhiên cho ứng dụng mật mã học trên nền tảng FPGA". Nội dung bao gồm lý thuyết sinh số ngẫu nhiên, các kiến trúc TRNG trên FPGA, nguyên lý ADPLL và DCO, kỹ thuật hậu xử lý XOR Corrector, chuẩn kiểm thử NIST SP800-22, và tổng quan nền tảng phần cứng Zynq-7000.

2.1 LÝ THUYẾT VỀ SINH SỐ NGẪU NHIÊN

Số ngẫu nhiên là nền tảng của mọi hệ thống mật mã học hiện đại. Chất lượng của chuỗi số ngẫu nhiên ảnh hưởng trực tiếp đến độ an toàn của các thuật toán mã hoá, giao thức xác thực và cơ chế bảo vệ dữ liệu. Theo lý thuyết thông tin, một chuỗi số được xem là ngẫu nhiên lý tưởng khi các bit xuất hiện độc lập với nhau và có phân bố đều, tức là Shannon Entropy đạt giá trị tối đa. Shannon Entropy của một nguồn ngẫu nhiên được định nghĩa theo công thức:

$$H = - \sum_{i=1}^n P(x_i) \times \log_2 P(x_i) \quad (2.1)$$

Trong đó $P(x_i)$ là xác suất xuất hiện của giá trị x_i trong chuỗi bit. Đối với một bộ TRNG lý tưởng, xác suất xuất hiện của bit '0' và '1' phải bằng nhau ($P=0.5$), khi đó giá trị Entropy sẽ đạt cực đại là 1 bit trên mỗi bit dữ liệu. Ngoài ra, Metric Entropy – định nghĩa là Shannon Entropy chia cho chiều dài chuỗi – được dùng để đánh giá chất lượng ngẫu nhiên trên toàn bộ bitstream:

$$H_m = H/L \quad (2.2)$$

Trong đó L là chiều dài chuỗi bit. Metric Entropy tiến gần đến giá trị 1 khi chuỗi có chất lượng ngẫu nhiên tốt.

2.1.1 Số ngẫu nhiên giả

Số ngẫu nhiên giả (PRNG) là hệ thống sử dụng các thuật toán toán học xác định để sản sinh ra chuỗi số có các đặc tính thống kê tương tự như sự ngẫu nhiên. Đặc điểm cốt yếu của PRNG là hoạt động hoàn toàn dựa trên các phép

tính logic mà không cần các yếu tố đầu vào vật lý từ môi trường bên ngoài. Quá trình này bắt đầu từ một giá trị khởi tạo được gọi là "seed"; từ giá trị này, thuật toán sẽ mở rộng thành một chuỗi bit dài hơn. Tuy nhiên, do bản chất là một hệ thống xác định, chuỗi PRNG hoàn toàn có thể bị tái lập nếu biết được thuật toán và giá trị seed ban đầu, khiến chúng trở nên yếu thế về mặt bảo mật khi so sánh với tính không thể dự đoán và không thể lặp lại của TRNG. Các kiến trúc PRNG phổ biến hiện nay bao gồm:

2.1.1.1 Linear Congruential Generator

LCG là một trong những thuật toán PRNG đơn giản và lâu đời nhất, sinh số theo công thức hồi quy tuyến tính:

$$X_{n+1} = (a \times X_n + c) \bmod m \quad (2.3)$$

Trong đó: X_n là số ngẫu nhiên hiện tại, a là hệ số nhân, c là số gia, m là modulus và X_0 là giá trị hạt giống khởi tạo ban đầu. Mặc dù có ưu điểm về tốc độ thực thi cực nhanh và yêu cầu tài nguyên thấp, LCG lại bộc lộ nhược điểm về chu kỳ ngắn và tính tương quan cao giữa các số liên tiếp, do đó không được khuyến cáo sử dụng cho các ứng dụng mật mã bảo mật [12].

2.1.1.2 Linear Feedback Shift Register

LFSR là cấu trúc phần cứng bao gồm các tầng thanh ghi dịch, trong đó giá trị đầu vào được tính toán bằng phép XOR của các bit ở các vị trí xác định. Hàm phản hồi của LFSR được biểu diễn dưới dạng đa thức trên trường hữu hạn. Một LFSR n bit đạt chu kỳ tối đa $2^n - 1$ khi đa thức sinh là đa thức nguyên thủy. Nhờ hiệu suất phần cứng cao, LFSR được ứng dụng rộng rãi trong mã hoá dòng và kiểm tra lỗi CRC [3].

2.1.1.3 Mersenne Twister

Mersenne Twister (MT19937) là thuật toán PRNG được sử dụng phổ biến nhất trong các ứng dụng mô phỏng và khoa học, với chu kỳ cực lớn $2^{19937} - 1$ và phân bố đồng đều trong không gian 623 chiều. Tuy nhiên, MT19937 không được coi là bộ sinh số an toàn về mặt mật mã vì trạng thái nội bộ của nó có thể bị khôi

phục hoàn toàn chỉ bằng cách quan sát 624 giá trị đầu ra liên tiếp, cho phép đối thủ dự đoán chính xác các số sẽ xuất hiện trong tương lai [9].

2.1.1.4 Cryptographically Secure PRNG (CSPRNG)

CSPRNG là phân lớp PRNG được thiết kế khắt khe để đáp ứng tiêu chuẩn "không thể dự đoán bit tiếp theo" (next-bit unpredictability). Điều này có nghĩa là ngay cả khi biết toàn bộ các bit đã được tạo ra trước đó, một thực thể có năng lực tính toán mạnh vẫn không thể dự đoán bit kế tiếp với xác suất lớn hơn 50%. Các kiến trúc CSPRNG hiện đại như AES-CTR DRBG hay Hash-DRBG thường được chuẩn hóa theo tiêu chuẩn NIST SP 800-90A. Dù có tính bảo mật cao, điểm yếu của CSPRNG vẫn nằm ở sự phụ thuộc vào chất lượng của seed đầu vào; nếu seed có entropy thấp hoặc bị lộ, toàn bộ tính an toàn của hệ thống sẽ bị phá vỡ [14].

2.1.2 Số ngẫu nhiên thực

Bộ sinh số ngẫu nhiên thực sự (TRNG) hoạt động dựa trên việc khai thác các thực thể vật lý như sự phân rã phóng xạ, entropy hệ thống hoặc nhiễu điện tử để tạo ra tính ngẫu nhiên. Khác với các thuật toán toán học của PRNG, TRNG tận dụng những biến động không xác định trong các quá trình vật lý để sản sinh ra những chuỗi số vô hạn, đảm bảo tính bảo mật cao, không thể dự đoán và không thể lặp lại ngay cả khi cấu trúc phần cứng bị lộ diện,. Dựa trên bản chất của nguồn entropy, TRNG được phân loại thành sáu nhóm kiến trúc chính :

2.1.2.1 Nhóm dựa trên nhiễu Johnson

Khai thác nhiễu nhiệt phát sinh từ chuyển động ngẫu nhiên của các hạt mang điện bên trong điện trở. Mặc dù là một nguồn entropy dồi dào, nhưng việc trích xuất nhiễu này thường đòi hỏi các mạch khuếch đại tương tự phức tạp, gây khó khăn khi muốn tích hợp trực tiếp vào các hệ thống thuần số như FPGA [10].

2.1.2.2 Nhóm dựa trên hiện tượng bất định thời gian (Jitter)

Khai thác sự biến thiên ngẫu nhiên về pha (jitter) của các cạnh xung nhịp trong các mạch dao động. Jitter pha tích lũy theo thời gian do nhiễu nhiệt, biến

động điện áp và nhiễu nền, tạo ra nguồn entropy thực sự. Đây là kiến trúc phổ biến nhất trên FPGA và là nền tảng của kiến trúc MURO-TRNG được nghiên cứu trong đề tài này [1].

2.1.2.3 Nhóm dựa trên trạng thái không ổn định (Metastability)

Khai thác trạng thái không xác định của Flip-Flop khi các tín hiệu đầu vào vi phạm điều kiện thời gian thiết lập hoặc thời gian giữ. Khi đó, đầu ra sẽ rơi vào vùng bất định trước khi ổn định tại mức logic 0 hoặc 1 một cách ngẫu nhiên [2].

2.1.2.4 Nhóm dựa trên lý thuyết hỗn loạn (Chaos)

Khai thác đặc tính nhạy cảm cực hạn với điều kiện ban đầu của các hệ thống phi tuyến. Một thay đổi nhỏ ở đầu vào sẽ dẫn đến kết quả đầu ra hoàn toàn khác biệt sau một khoảng thời gian, tạo ra tính ngẫu nhiên cao [13].

2.1.2.5 Nhóm hệ thống lai

Kết hợp nhiều nguồn entropy khác nhau để tăng chất lượng ngẫu nhiên và khả năng chống tấn công. Ví dụ điển hình là kết hợp jitter từ nhiều ring oscillator với metastability của flip-flop, sau đó tổng hợp qua cổng XOR để thu được bitstream có entropy cao hơn từng nguồn đơn lẻ [1] [2].

2.1.2.6 Nhóm dựa trên vật lý lượng tử (Quantum)

Khai thác các hiện tượng lượng tử cơ bản như phân rã phóng xạ, phân cực photon hay hiệu ứng đường hầm lượng tử để tạo entropy tuyệt đối không thể dự đoán về mặt vật lý. Quantum TRNG cho chất lượng entropy cao nhất nhưng đòi hỏi phần cứng chuyên dụng, không thể tích hợp vào FPGA thông thường [11].

2.2 CÁC KIẾN TRÚC TRNG TRÊN FPGA

FPGA (Field Programmable Gate Array) được xem là môi trường lý tưởng để hiện thực hóa các bộ sinh số ngẫu nhiên thực sự nhờ vào tính linh hoạt trong lập trình và khả năng khai thác trực tiếp các đặc tính vật lý của mạch logic số. Việc áp dụng các kỹ thuật số thuần túy giúp bộ TRNG dễ dàng tích hợp vào các hệ thống tái cấu trúc mà không cần đến các linh kiện tương tự bên ngoài. Trong

số sáu nhóm TRNG, chỉ có Jitter-based, Metastability-based và Mixed-based phù hợp để triển khai thuần số trên FPGA mà không cần bất kỳ linh kiện nào bên ngoài.

2.2.1 Jitter TRNG

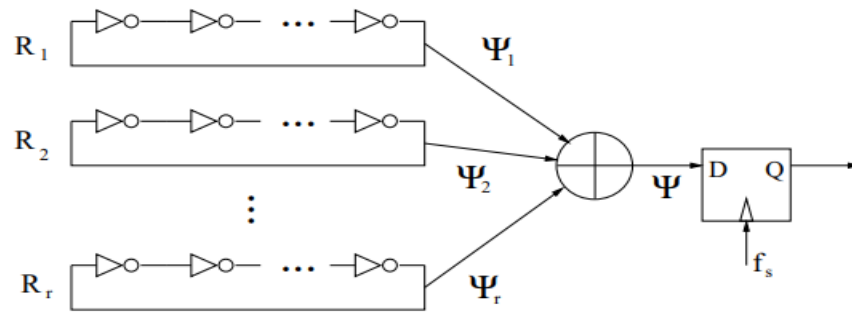
Kiến trúc này khai thác sự biến động không xác định về thời gian (jitter) phát sinh trong các mạch dao động làm nguồn entropy chính. Đây là phương pháp phổ biến nhất trên FPGA do tính đơn giản trong thiết kế, khả năng tiết kiệm tài nguyên phần cứng và hiệu suất trích xuất bit ngẫu nhiên cao.

2.2.1.1 Nguyên lý của bộ dao động vòng (Ring Oscillator)

Bộ dao động vòng đóng vai trò là nguồn entropy cốt lõi cho hệ thống,. Một bộ RO cơ bản thường được cấu thành từ một số lẻ các cổng logic đảo (ví dụ như cổng NOT/NOR) mắc nối tiếp thành một vòng phản hồi kín,. Do không có phần tử nhớ và có số tầng đảo ngược pha, tín hiệu sẽ liên tục dao động quanh vòng lặp. Tần số dao động lý thuyết của bộ RO được xác định bởi tổng trễ lan truyền của các phần tử trong vòng:

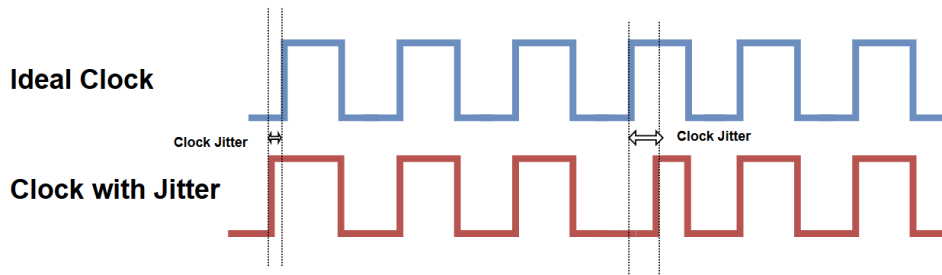
$$f_{RO} = \frac{1}{2 \times n \times t_{pd}} \quad (2.4)$$

Trong đó: n là số phần tử đảo trong chuỗi và t_{pd} là trễ lan truyền trung bình của mỗi phần tử. Hệ số 2 xuất hiện do tín hiệu cần hai lần đi qua chuỗi (một lần cho sườn lên và một lần cho sườn xuống) để hoàn thành một chu kỳ dao động hoàn chỉnh.



Hình 2.1: Cấu trúc Ring Oscillator cơ bản [4]

2.2.1.2 Nguồn gốc Jitter pha



Hình 2.2: Sự biến thiên về thời gian jitter

Trong thực tế, thời gian trễ t_{pd} của các cổng logic không cố định mà liên tục biến thiên ngẫu nhiên theo từng chu kỳ dao động. Sự sai lệch giữa thời điểm xuất hiện thực tế của cạnh tín hiệu so với vị trí lý tưởng của nó được gọi là jitter pha. Jitter pha không phải là hiện tượng nhất thời mà có tính tích lũy: sai lệch pha của chu kỳ sau chịu ảnh hưởng từ chu kỳ trước, khiến độ bất định về pha tăng dần theo thời gian hoạt động theo cơ chế bước đi ngẫu nhiên. Đây chính là đặc tính then chốt biến ring oscillator thành nguồn entropy cho hệ thống TRNG — bộ dao động vòng hoạt động càng lâu, vùng bất định về pha càng mở rộng và chất lượng ngẫu nhiên của bit lấy mẫu càng cao.

Trên nền tảng FPGA, jitter trong fabric logic phát sinh từ bốn nguyên nhân vật lý chính. Nhiều nhất là nguyên nhân căn bản nhất, xuất phát từ chuyển động nhiệt ngẫu nhiên của các hạt mang điện trong kênh CMOS, làm biến động điện áp ngưỡng chuyển mạch của transistor và kéo theo biến thiên ngẫu nhiên của trễ lan truyền tại mỗi cổng logic. Nhiều nguồn cấp hình thành do sụt áp trên đường dây VDD nội bộ chip và nhiễu chuyển mạch từ các khối logic lân cận đang hoạt động đồng thời, trực tiếp làm thay đổi tốc độ nạp và xả tín hiệu trong chuỗi NOR của ring oscillator. Crosstalk điện từ xuất hiện do ghép điện dung ký sinh giữa các đường định tuyến kề nhau trên fabric, tạo xung nhiễu cảm ứng lên tín hiệu ring oscillator mỗi khi các đường lân cận chuyển trạng thái. Cuối cùng, biến thiên PVT phản ánh dao động ngắn hạn của điều kiện vận hành thực tế nhiệt độ môi trường, điện áp cấp, và sự phân tán công nghệ trong sản xuất tạo ra thành

phần biến thiên trễ lan truyền không thể loại bỏ hoàn toàn bằng bất kỳ mạch bù ngoài nào.

2.2.1.3 Lấy mẫu và thu entropy

Để thu thập entropy, tín hiệu từ bộ dao động vòng với pha bất định được lấy mẫu bởi một xung nhịp độc lập thông qua phần tử nhớ D Flip-Flop. Quá trình này bắt lại trạng thái logic (0 hoặc 1) của bộ dao động tại các thời điểm được ấn định bởi xung nhịp lấy mẫu. Do jitter pha đã tích lũy qua nhiều chu kỳ trước đó, cạnh tín hiệu của ring oscillator có thể rơi vào vùng trước hoặc sau điểm lấy mẫu một cách ngẫu nhiên, khiến giá trị bit đầu ra của DFF không thể dự đoán — đây chính là bit ngẫu nhiên thô.

Chất lượng entropy của mỗi bit phụ thuộc trực tiếp vào lượng jitter đã tích lũy tại thời điểm lấy mẫu. Khi jitter tích lũy đủ lớn so với cửa sổ ổn định của DFF, xác suất để cạnh tín hiệu rơi vào hai phía của điểm lấy mẫu tiến gần đến 0,5 cho mỗi phía, tạo ra phân phối bit ngẫu nhiên cân bằng với entropy gần đạt mức tối đa 1 bit/mẫu. Ngược lại, nếu jitter tích lũy không đủ, cạnh tín hiệu luôn rơi rõ ràng về một phía, bit đầu ra bị thiên kiến và entropy giảm đáng kể. Vì vậy, thiết kế cần duy trì đủ thời gian tích lũy jitter giữa hai lần lấy mẫu liên tiếp. Trong kiến trúc MURO-TRNG, điều kiện này được đảm bảo thông qua cơ chế ADPLL kiểm soát tần số ring oscillator về trạng thái khoá pha với tần số tham chiếu ổn định, duy trì điểm lấy mẫu nằm ổn định trong vùng bất định của jitter qua suốt quá trình hoạt động.

2.2.1.4 Mô hình Multiple Ring Oscillator

Để gia tăng throughput và cải thiện chất lượng entropy, kỹ thuật Multiple Ring Oscillator được áp dụng bằng cách sử dụng đồng thời nhiều bộ dao động vòng hoạt động ở các dải tần số khác nhau. Tín hiệu đầu ra từ các bộ dao động này sau khi được lấy mẫu sẽ được tổng hợp thông qua một cổng cộng logic XOR:

$$b_{raw} = b_1 \oplus b_2 \oplus \dots \oplus b_n \quad (2.5)$$

Việc kết hợp nhiều nguồn entropy độc lập giúp tăng cường độ hỗn loạn của hệ thống, giảm thiểu các thiên kiến thống kê (bias) trong chuỗi bit thô và giúp hệ thống có khả năng chống lại các cuộc tấn công phân tích tương quan tốt hơn.

2.2.2 Metastability TRNG

Kiến trúc này khai thác trạng thái bất ổn định (metastability) của các phần tử nhớ như Flip-Flop khi các điều kiện về thời gian thiết lập (setup time) hoặc thời gian giữ (hold time) bị vi phạm. Trong điều kiện này, Flip-Flop không thể phân giải ngay lập tức về mức logic 0 hoặc 1 mà rơi vào trạng thái điện áp trung gian trong một khoảng thời gian trước khi ổn định theo một hướng không thể dự đoán được.

Khi các điều kiện vật lý của tín hiệu đầu vào và xung nhịp đạt đến điểm cân bằng lý tưởng, xác suất phân giải về mức logic 1 hoặc 0 sẽ tiến gần đến giá trị 0.5, tạo ra một nguồn entropy chất lượng cao. Tuy nhiên, nhược điểm lớn nhất của kiến trúc này là khó kiểm soát và duy trì tần suất xảy ra trạng thái metastability một cách ổn định trên các nền tảng FPGA. Bên cạnh đó, các thiết kế dựa trên metastability thường tiêu tốn nhiều tài nguyên phần cứng hơn và có cấu trúc mạch điều phối phức tạp hơn so với các phương pháp dựa trên jitter.

2.3 VÒNG KHÓA PHA VÀ BỘ DAO ĐỘNG ĐIỀU KHIỂN SỐ

2.3.1 Hạn chế của PLL analog và ưu điểm của ADPLL

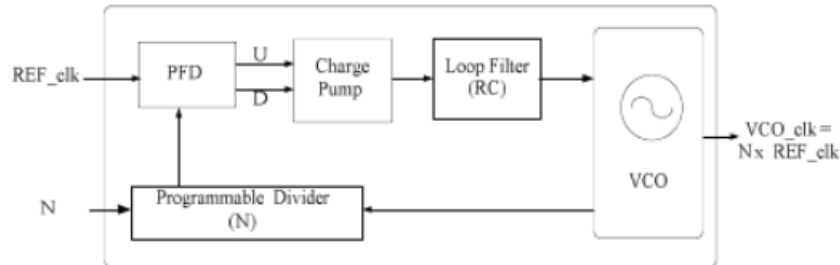
Vòng khóa pha (Phase Locked Loop) truyền thống là hệ thống phản hồi tương tự gồm bộ tách pha tương tự, bộ lọc vòng RC và bộ dao động điều khiển điện áp (VCO). Khi được tích hợp vào FPGA, PLL analog gặp các hạn chế sau:

Diện tích lớn: Các thành phần RC và mạch tương tự đòi hỏi diện tích silicon đáng kể, trong khi FPGA được tối ưu hóa cho logic số.

Kém linh hoạt: Tham số PLL tương tự được cố định bởi giá trị linh kiện vật lý, khó điều chỉnh sau khi tổng hợp.

Nhạy nhiễu: Mạch VCO tương tự dễ bị ảnh hưởng bởi nhiễu điện từ và biến động nguồn cấp trên FPGA.

Vị trí cố định: PLL tương tự được tích hợp tại các vị trí đặc biệt trên chip, không thể đặt tự do như logic số.



Hình 2.3: Sơ đồ của PLL Analog cơ bản [5]

ADPLL (All Digital Phase Locked Loop) thay thế toàn bộ các thành phần tương tự bằng logic số thuần túy, giải quyết triệt để các hạn chế trên. ADPLL có thể được tổng hợp và triển khai trực tiếp lên fabric logic của FPGA, dễ dàng nhân bản thành nhiều thực thể với tham số khác nhau và hoạt động ổn định hơn trong môi trường nhiễu kỹ thuật số. Đây là lý do ADPLL được chọn làm nền tảng để xây dựng các ring oscillator trong kiến trúc MURO-TRNG.

2.3.2 Kiến trúc ADPLL

Kiến trúc ADPLL trong đề tài gồm bốn thành phần chính kết nối thành vòng kín phản hồi:

2.3.2.1 Bộ tách pha (Phase Detector)

Phase Detector thực hiện so sánh pha giữa tín hiệu tham chiếu f_{ref} và tín hiệu phản hồi từ bộ chia $N/2$ Counter. Trong kiến trúc này, Phase Detector được xây dựng bằng cổng XOR: khi hai tín hiệu cùng pha, ngõ ra XOR ở mức thấp; khi lệch pha, ngõ ra lên mức cao và kích hoạt K Counter đếm lên.

2.3.2.2 Bộ lọc vòng (K Counter)

K Counter là bộ đếm, đóng vai trò bộ lọc vòng số. Khi bộ tách pha phát hiện sai lệch pha, K Counter đếm lên; khi pha đồng bộ, K Counter dừng đếm. Bit

có trọng số lớn nhất (MSB) của K Counter – gọi là tín hiệu Carry – được dùng làm tín hiệu điều khiển DCO. Tần số xung nhịp của K Counter được xác định theo công thức:

$$f_k = M \times f_{ref} \quad (2.6)$$

Trong đó: f_k là tần số xung nhịp của K_counter (Hz), M là số bit đếm của K_counter và f_{ref} là tần số tham chiếu đầu vào ADPLL (Hz).

2.3.2.3 Bộ dao động điều khiển kỹ thuật số (DCO)

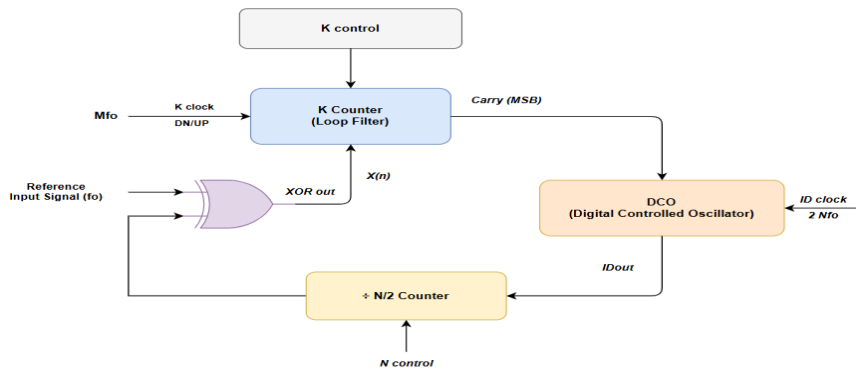
DCO nhận tín hiệu Carry từ K_counter để điều chỉnh tần số dao động. Khi ADPLL đạt trạng thái khoá pha, ngõ ra DCO ký hiệu IDout có tần số dao động xác định theo công thức:

$$f_{dco} = 2 \times N \times f_{ref} \quad (2.7)$$

Trong đó N là tham số bộ chia và f_{ref} là tần số tham chiếu (Hz).

2.3.2.4 Bộ chia tần N/2 (Divide-by-N/2 Counter)

Bộ chia N/2 sử dụng T Flip-Flop để chia đôi tần số IDout, tạo tín hiệu phản hồi đưa trở lại so sánh với f_{ref} , khép kín vòng khoá pha. Khi ADPLL đạt trạng thái khoá, toàn bộ quan hệ tần số giữa các thành phần được duy trì ổn định, trong khi jitter pha vẫn tiếp tục tích lũy ngẫu nhiên theo thời gian – đây chính là cơ chế tạo ra entropy của hệ thống.



Hình 2.4: Sơ đồ khối kiến trúc ADPLL [1]

2.3.3 Cấu trúc DCO dựa trên chuỗi cổng NOR

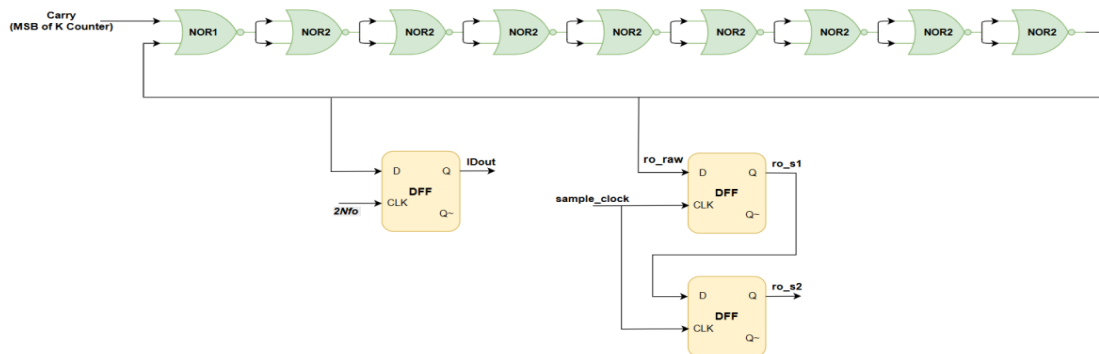
DCO trong kiến trúc MURO-TRNG được xây dựng từ chín cổng NOR hai đầu vào mắc nối tiếp thành chuỗi kết hợp với một phần tử nhớ DFF. Chuỗi NOR tạo thành vòng lặp tổ hợp tự do với tần số dao động phụ thuộc trực tiếp vào trễ lan truyền vật lý của cổng NOR trên FPGA:

$$f_{RO} = \frac{1}{2 \times 9 \times t_{nor}} \quad (2.8)$$

Trong đó : t_{nor} là trễ lan truyền trung bình của mỗi cổng NOR. Tín hiệu dao động tự do của chuỗi NOR được ký hiệu là ro_raw và là nguồn jitter entropy chính của hệ thống.

Cổng NOR đầu tiên trong chuỗi nhận hai đầu vào: một là tín hiệu Carry từ K Counter dùng để điều chỉnh tần số, một là ngõ ra của cổng NOR cuối tạo vòng phản hồi. Khi Carry = 0, tất cả cổng NOR hoạt động như bộ đảo (inverter) và vòng dao động tự do tích lũy jitter; khi Carry = 1, ngõ ra cổng NOR đầu tiên bị kéo cưỡng bức xuống mức thấp, làm chậm tần số dao động để ADPLL duy trì trạng thái khoá pha.

DFF đóng vai trò lấy mẫu đồng bộ tín hiệu ro_raw tại mỗi sườn lên của xung nhịp 2Nfo để tạo ra IDout ổn định cấp cho bộ chia N/2. Điều kiện quan trọng nhất trong thiết kế: DFF phải được đặt ngoài vòng phản hồi tổ hợp của chuỗi NOR.



Hình 2.5: Cấu trúc DCO dựa trên chuỗi cổng NOR

2.4 HẬU XỬ LÝ XOR CORRECTOR

Trong thực tế triển khai trên phần cứng, các chuỗi bit ngẫu nhiên thô được trích xuất từ cấu trúc MURO-TRNG thường tồn tại một mức độ sai lệch thống kê bias nhất định,. Hiện tượng này phát sinh do sự không cân bằng tuyệt đối về đặc tính vật lý giữa các linh kiện trong mạch logic hoặc do các tác động không đồng nhất từ môi trường vận hành. Để khắc phục vấn đề này và đảm bảo chất lượng ngẫu nhiên đạt tiêu chuẩn an toàn mật mã, một mạch hậu xử lý dựa trên kỹ thuật XOR Corrector được tích hợp vào hệ thống. Kỹ thuật XOR Corrector hoạt động dựa trên nguyên lý thực hiện phép toán logic XOR giữa hai bit ngẫu nhiên thô liên tiếp trong chuỗi dữ liệu đầu vào. Cụ thể, giá trị bit đầu ra b_{out} tại thời điểm k được xác định theo công thức:

$$b_{out}[k] = b_{raw}[2k] \oplus b_{raw}[2k + 1] \quad (2.9)$$

Trong đó: $b_{out}[k]$ là bit đầu ra sau hậu xử lý tại vị trí k , $b_{raw}[2k]$, $b_{raw}[2k + 1]$ là cặp bit thô liên tiếp tại vị trí $2k$ và $2k+1$ trong chuỗi đầu vào.

Hiệu quả của phương pháp này được minh chứng thông qua xác suất thống kê. Giả sử xác suất xuất hiện bit '1' trong chuỗi thô bị sai lệch một lượng ε so với giá trị lý tưởng 0.5, tức là $P(b=1)=0.5+\varepsilon$. Khi đó, xác suất để bit đầu ra sau phép XOR bằng '1' là tổng xác suất của hai trường hợp có đầu vào khác nhau:

Trường hợp 1: $b_1=0$ và $b_2=1$

Trường hợp 2: $b_1=1$ và $b_2=0$

Phương trình xác suất đầu ra được biểu diễn như sau:

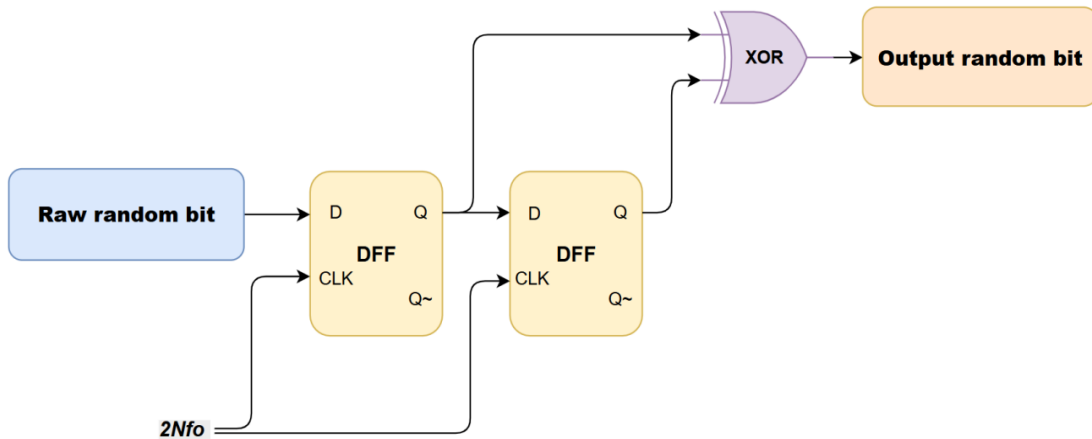
$$P(b_{out} = 1) = P(b_1 = 0) \times P(b_2 = 1) + P(b_1 = 1) \times P(b_2 = 0)$$

$$P(b_{out} = 1) = (0.5 - \varepsilon) \times (0.5 + \varepsilon) + (0.5 + \varepsilon) \times (0.5 - \varepsilon)$$

$$P(b_{out} = 1) = 2 \times (0.25 - \varepsilon^2) = 0.5 - 2\varepsilon^2$$

Vì sai lệch ε thường rất nhỏ, nên giá trị ε^2 sẽ tiến dần về mức không đáng kể. Kết quả là xác suất xuất hiện bit '1' sau hậu xử lý sẽ tiệm cận giá trị lý tưởng

0.5, giúp triệt tiêu gần như hoàn toàn các sai lệch thống kê. Mặc dù cải thiện đáng kể độ ngẫu nhiên, việc sử dụng XOR Corrector dẫn đến hệ quả tất yếu là tốc độ trích xuất bit (throughput) của hệ thống bị giảm đi một nửa. Điều này là do hệ thống cần tiêu thụ hai bit thô đầu vào để sản sinh ra một bit ngẫu nhiên sạch ở đầu ra



Hình 2.6: Cấu trúc XOR Corrector [1]

2.5 CHUẨN KIỂM THỬ NIST SP800-22

NIST SP800-22 (National Institute of Standards and Technology Special Publication 800-22) [6] là bộ kiểm thử thống kê chuẩn quốc tế do Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ phát triển, dùng để đánh giá chất lượng ngẫu nhiên của bitstream đầu ra từ các bộ tạo số ngẫu nhiên ứng dụng trong mật mã học.

2.5.1 Các phép thử trong NIST SP800-22

Bộ kiểm thử bao gồm 15 phép thử thống kê độc lập, mỗi phép thử đánh giá một đặc tính ngẫu nhiên khác nhau:

1. Frequency (Monobit) Test: Xác định mức độ cân bằng giữa số lượng bit '0' và '1' trong chuỗi bit.
2. Block Frequency Test: Phân tích tỉ lệ xuất hiện của bit '1' trong các khối dữ liệu con có độ dài xác định.

3. Runs Test: Kiểm tra tổng số các chuỗi bit giống nhau liên tiếp (runs) để đánh giá tốc độ thay đổi trạng thái của chuỗi.
4. Longest Run of Ones Test: Tìm kiếm và phân tích độ dài của chuỗi bit '1' dài nhất trong từng khối dữ liệu.
5. Binary Matrix Rank Test: Đánh giá tính độc lập tuyến tính giữa các phân đoạn của chuỗi thông qua hạng của ma trận nhị phân.
6. Discrete Fourier Transform (Spectral) Test: Sử dụng biến đổi Fourier để phát hiện các thành phần có tính chu kỳ hoặc lặp lại trong chuỗi bit.
7. Non-overlapping Template Matching Test: Đếm tần suất xuất hiện của các mẫu bit mẫu cụ thể mà không cho phép chúng chồng lấp lên nhau.
8. Overlapping Template Matching Test: Tương tự như trên nhưng cho phép các mẫu bit mẫu được chồng lấp.
9. Maurer's Universal Statistical Test: Đánh giá độ nén của chuỗi bit; một chuỗi ngẫu nhiên thực sự sẽ không thể bị nén một cách đáng kể.
10. Linear Complexity Test: Xác định độ phức tạp của chuỗi dựa trên chiều dài của thanh ghi dịch phản hồi tuyến tính (LFSR) ngắn nhất có khả năng tạo ra chuỗi đó.
11. Serial Test: Kiểm tra sự phân bố đồng đều của tất cả các tổ hợp m-bit có thể xuất hiện trong toàn bộ chuỗi.
12. Approximate Entropy Test: So sánh tần suất xuất hiện của các mẫu bit có độ dài kế tiếp nhau (ví dụ m và m+1) để đánh giá tính dự báo.
13. Cumulative Sums Test: Thực hiện kiểm tra bước đi ngẫu nhiên thông qua tổng tích lũy của các bit (theo cả chiều thuận và chiều nghịch).
14. Random Excursions Test: Phân tích số lần các trạng thái cụ thể xuất hiện trong một chu trình bước đi ngẫu nhiên.
15. Random Excursions Variant Test: Một biến thể của phép thử trên nhằm kiểm tra sâu hơn sự phân bố của các trạng thái trong bước đi ngẫu nhiên.

2.5.2 Tiêu chí đánh giá

Kết quả của mỗi phép thử được định lượng thông qua giá trị xác suất P-value nằm trong khoảng . Về mặt lý thuyết, P-value là xác suất để một bộ sinh số

ngẫu nhiên lý tưởng tạo ra một chuỗi dữ liệu có tính ngẫu nhiên thấp hơn hoặc bằng với chuỗi đang được kiểm thử. Các quy tắc đánh giá chuẩn:

Điều kiện đạt (PASS): Chuỗi bit được công nhận là ngẫu nhiên nếu $P_value \geq 0.001$.

Điều kiện không đạt (FAIL): Nếu $P_value < 0.001$, chuỗi bit bị coi là có cấu trúc xác định và không phù hợp.

Ngưỡng 0,001 tương ứng với mức ý nghĩa thống kê $\alpha = 0,1\%$, nghĩa là xác suất bác bỏ sai giả thuyết ngẫu nhiên nhỏ hơn 0,1%. Khuyến nghị của NIST là kiểm thử trên tập dữ liệu đủ lớn (từ hàng triệu đến hàng trăm triệu bit) để đảm bảo kết quả thống kê có ý nghĩa và đáng tin cậy.

2.6 TỔNG QUAN NỀN TẢNG PHẦN CỨNG VÀ GIAO TIẾP

2.6.1 Kiến trúc Zynq-7000 và bo mạch PYNQ-Z1

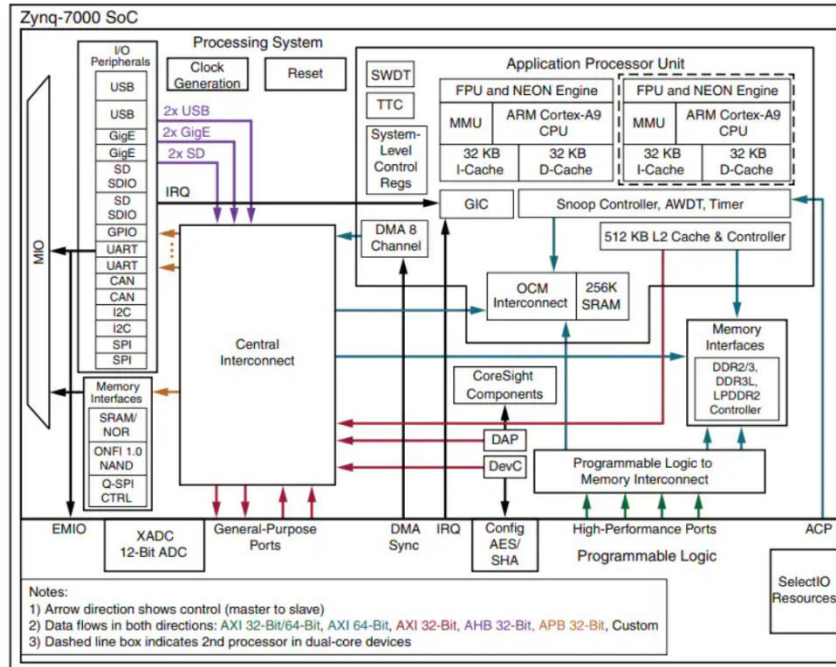
2.6.1.1 Tổng quan

Xilinx Zynq-7000 là họ vi mạch tích hợp hệ thống SoC kết hợp hai thành phần có bản chất khác nhau trên cùng một die silicon: phần logic khả trình PL dựa trên kiến trúc FPGA Artix-7 và hệ thống xử lý PS tích hợp bộ xử lý ARM Cortex-A9 lõi kép. Sự tích hợp này tạo ra nền tảng linh hoạt cho các hệ thống nhúng phức tạp, trong đó PL đảm nhiệm các tác vụ xử lý phần cứng tùy chỉnh với độ trễ thấp và PS đảm nhiệm các tác vụ phần mềm đòi hỏi tính linh hoạt cao. Bo mạch PYNQ-Z1 sử dụng chip XC7Z020 thuộc họ Zynq-7000 làm nền tảng phần cứng.

2.6.1.2 Cấu trúc

Phần PL của XC7Z020 được xây dựng trên công nghệ SRAM-based LUT 6 đầu vào tương thích Artix-7, cung cấp tài nguyên logic số có thể lập trình hoàn toàn thông qua ngôn ngữ mô tả phần cứng. Phần PS tích hợp bộ xử lý ARM Cortex-A9 lõi kép, bộ điều khiển bộ nhớ DDR3 và các ngoại vi kết nối tiêu chuẩn. Hai thành phần PL và PS được kết nối với nhau thông qua bus AXI với ba

loại cổng kết nối phục vụ các nhu cầu khác nhau: cổng AXI GP cho truyền dữ liệu điều khiển, cổng AXI HP cho truyền dữ liệu băng thông cao trực tiếp đến DDR RAM và cổng AXI ACP cho truy cập bộ nhớ đệm của PS.



Hình 2.7: Kiến trúc tổng quan Zynq-7000 [7]

2.6.2 Giao tiếp PL và PS qua AXI DMA

2.6.2.1 Giao thức AXI4

Trong hệ sinh thái của dòng SoC Zynq-7000, giao thức AXI4 đóng vai trò là chuẩn bus nội bộ thuộc kiến trúc ARM AMBA, cho phép liên kết và đồng bộ hóa hoạt động giữa các khối IP core khác nhau,. Để đáp ứng các yêu cầu đa dạng về truyền dẫn, AXI4 được phân tách thành ba biến thể chính: AXI4 Full: Tối ưu cho việc truyền tải khối lượng dữ liệu lớn có định vị địa chỉ (memory-mapped). AXI4-Lite: Phiên bản đơn giản hóa, chuyên dùng để cấu hình và truy xuất các thanh ghi điều khiển. AXI4-Stream: Được thiết kế riêng cho việc truyền dẫn dữ liệu theo luồng một chiều, không yêu cầu quản lý địa chỉ, giúp tối ưu băng thông cho các ứng dụng xử lý tín hiệu thời gian thực.

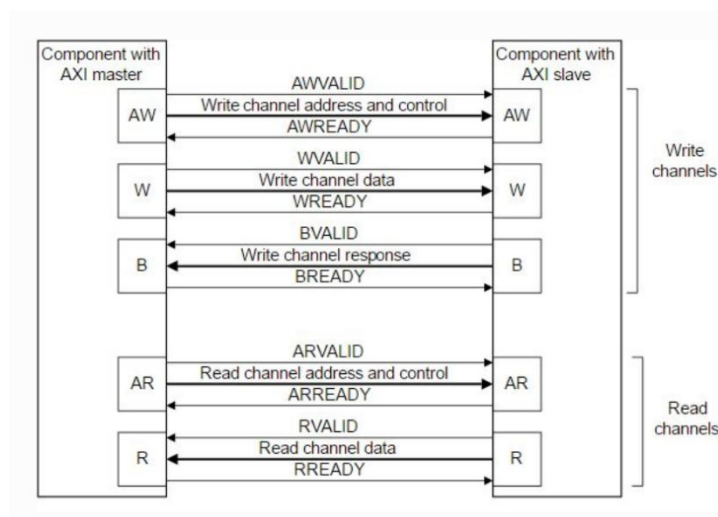
Hoạt động của chuẩn AXI4-Stream dựa trên quy trình bắt tay chặt chẽ giữa bên phát và bên nhận thông qua các tín hiệu điều khiển cốt lõi:

TVALID: Tín hiệu do bên phát quản lý. Khi TVALID ở mức cao, hệ thống xác nhận dữ liệu trên bus TDATA đã hợp lệ và sẵn sàng để chuyển giao.

TREADY: Tín hiệu do bên nhận quản lý. Trạng thái mức cao của TREADY chỉ thị bên nhận đã sẵn sàng để tiếp nhận dữ liệu mới.

TDATA: Bus mang giá trị dữ liệu thực tế, với độ rộng linh hoạt (thường là 32 hoặc 64 bit) tùy thuộc vào cấu hình của hệ thống thu thập.

Điểm mấu chốt trong giao thức này là việc chuyển giao dữ liệu chỉ thực sự diễn ra tại thời điểm cạnh xung nhịp mà cả TVALID và TREADY đồng thời ở mức tích cực. Cơ chế này đảm bảo tính toàn vẹn tuyệt đối cho luồng dữ liệu, ngăn chặn hiện tượng mất mát hoặc sai lệch thông tin khi một trong hai phía chưa đạt trạng thái sẵn sàng.



Hình 2.8: Kiến trúc tổng quan của AXI4 protocol [8]

2.6.2.2 AXI DMA

DMA là cơ chế cho phép thiết bị ngoại vi truyền dữ liệu trực tiếp đến và từ bộ nhớ chính mà không cần CPU tham gia xử lý từng byte dữ liệu. AXI DMA trong Zynq-7000 hoạt động như một AXI Master, sử dụng cổng AXI HP để truy cập trực tiếp vào DDR RAM thông qua bộ điều khiển bộ nhớ trong PS. AXI DMA hỗ trợ hai kênh truyền độc lập: kênh S2MM nhận dữ liệu từ AXI4-Stream

và ghi vào DDR RAM; kênh MM2S đọc dữ liệu từ DDR RAM và xuất qua AXI4-Stream.

2.6.2.3 AXI-Stream FIFO

AXI-Stream FIFO là bộ đệm đàn hồi được chèn giữa MURO-TRNG core và AXI DMA để giải quyết sự không đồng bộ về tốc độ và pha clock giữa hai domain. FIFO sử dụng Block RAM để lưu trữ dữ liệu tạm thời, đảm bảo không mất dữ liệu khi AXI DMA chưa sẵn sàng nhận.

2.6.3 Giao thức UDP và Gigabit Ethernet

UDP là giao thức tầng vận chuyển trong mô hình TCP/IP, hoạt động theo cơ chế không kết nối— tức là dữ liệu được gửi đi mà không cần thiết lập phiên truyền trước, không có cơ chế kiểm soát luồng và không có cơ chế truyền lại khi mất gói. So với TCP, UDP loại bỏ hoàn toàn quá trình bắt tay ba chiều và các cơ chế đảm bảo tin cậy, nhờ đó giảm đáng kể chi phí xử lý. Header UDP chỉ bao gồm 8 byte với bốn trường cố định là cổng nguồn, cổng đích, độ dài và checksum, nhỏ hơn nhiều so với header TCP tối thiểu 20 byte. Đặc tính overhead thấp và độ trễ nhỏ của UDP phù hợp với yêu cầu truyền luồng dữ liệu ngẫu nhiên liên tục với tốc độ cao, trong đó việc mất một tỉ lệ nhỏ gói tin không ảnh hưởng đến giá trị thống kê của toàn bộ tập dữ liệu kiểm thử.

Gigabit Ethernet (chuẩn IEEE 802.3ab, 1000BASE-T) cung cấp băng thông vật lý 1 Gbps qua cáp xoắn đôi không chắn (UTP) Cat5e hoặc Cat6, với độ trễ truyền dẫn thấp ở mức microsecond. Sự kết hợp giữa UDP và Gigabit Ethernet tạo thành kênh truyền dữ liệu có băng thông thực tế đủ lớn để thu thập bitstream từ PYNQ-Z1 về PC mà không trở thành nút thắt cổ chai trong toàn bộ pipeline.

CHƯƠNG 3 TÍNH TOÁN VÀ THIẾT KẾ HỆ THỐNG

3.1 YÊU CẦU HỆ THỐNG

Hệ thống được thiết kế nhằm triển khai bộ sinh số ngẫu nhiên thực TRNG trên nền tảng FPGA PYNQ-Z1 và đánh giá chất lượng ngẫu nhiên của chuỗi bit đầu ra theo tiêu chuẩn NIST. Để đáp ứng mục tiêu nghiên cứu, hệ thống cần thỏa mãn các yêu cầu sau:

1. Hệ thống phải tạo được chuỗi bit ngẫu nhiên dựa trên nguồn entropy vật lý thay vì các thuật toán giả ngẫu nhiên.
2. Kiến trúc TRNG phải được xây dựng dựa trên mô hình MURO-TRNG, trong đó nguồn entropy được tạo ra từ nhiều bộ dao động hoạt động song song.
3. Hệ thống phải có cơ chế lấy mẫu và thu hoạch entropy nhằm chuyển đổi các biến thiên thời gian ngẫu nhiên thành chuỗi bit nhị phân.
4. Chuỗi bit đầu ra phải được xử lý hậu kỳ nhằm giảm thiểu hiện tượng thiên lệch thống kê bias và nâng cao chất lượng ngẫu nhiên.
5. Thiết kế phải được hiện thực hoàn toàn trên nền tảng FPGA bằng ngôn ngữ mô tả phần cứng Verilog và có khả năng tích hợp với hệ thống Zynq của kit PYNQ-Z1.
6. Hệ thống phải hỗ trợ cơ chế truyền và lưu trữ dữ liệu để phục vụ quá trình thu thập số lượng lớn chuỗi bit ngẫu nhiên.
7. Dữ liệu sinh ra phải được truyền luồng (stream) với tốc độ cao qua cổng Gigabit Ethernet về máy tính PC và lưu thành tệp .txt (hoặc .bin) phục vụ quá trình phân tích và kiểm định.
8. Chuỗi bit đầu ra phải được đánh giá bằng bộ kiểm định thống kê NIST SP 800-22 nhằm xác nhận tính ngẫu nhiên của hệ thống.

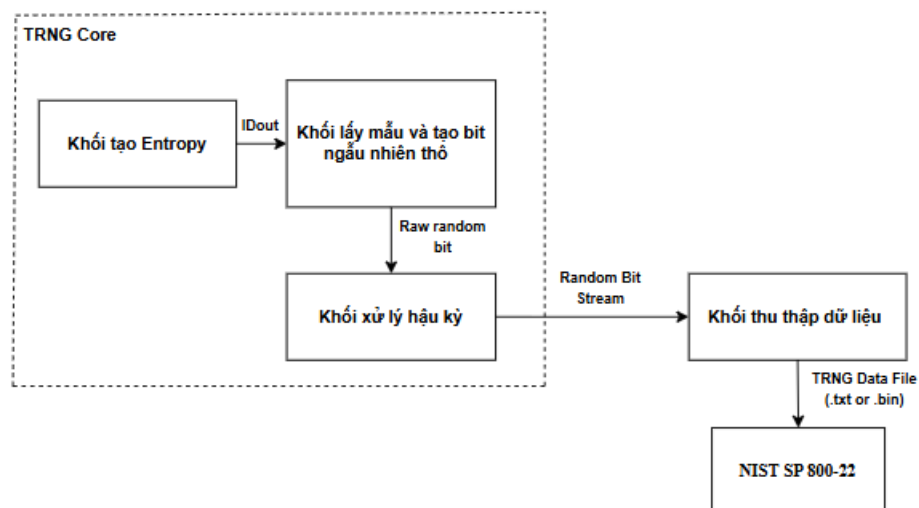
Từ các yêu cầu trên, hệ thống được xây dựng gồm hai thành phần chính: lõi sinh số ngẫu nhiên (TRNG Core) và khối thu thập dữ liệu. Lõi TRNG chịu trách nhiệm tạo và xử lý nguồn entropy để sinh chuỗi bit ngẫu nhiên, trong khi

khối thu thập dữ liệu đảm nhiệm việc lưu trữ và chuyển dữ liệu sang môi trường phần mềm phục vụ kiểm định thống kê. Kiến trúc tổng thể của hệ thống được trình bày trong mục tiếp theo.

3.2 THIẾT KẾ HỆ THỐNG

3.2.1 Sơ đồ khối hệ thống

Hệ thống MURO-TRNG được thiết kế nhằm tạo ra chuỗi bit ngẫu nhiên thực dựa trên hiện tượng timing Jitter phát sinh từ các bộ dao động ADPLL hoạt động trên nền tảng FPGA PYNQ-Z1. Kiến trúc hệ thống bao gồm các khối chức năng chính từ khâu sinh entropy, lấy mẫu, hậu xử lý dữ liệu cho đến thu thập và lưu trữ dữ liệu phục vụ quá trình đánh giá thống kê. Sơ đồ khối tổng thể của hệ thống được trình bày ở Hình 3.1.



Hình 3.1: Sơ đồ khối hệ thống

Hệ thống được chia thành bốn khối chức năng chính:

Khối tạo Entropy: Đây là nguồn sinh ngẫu nhiên của hệ thống. Khối này bao gồm nhiều bộ dao động dựa trên kiến trúc ADPLL và DCO hoạt động ở các mức tần số khác nhau. Do ảnh hưởng của nhiễu nhiệt, nhiễu nguồn và biến thiên phần cứng bên trong FPGA, các tín hiệu dao động xuất hiện hiện tượng Timing

Jitter. Các dao động mang jitter này đóng vai trò là nguồn entropy vật lý để tạo ra tính ngẫu nhiên thực cho hệ thống.

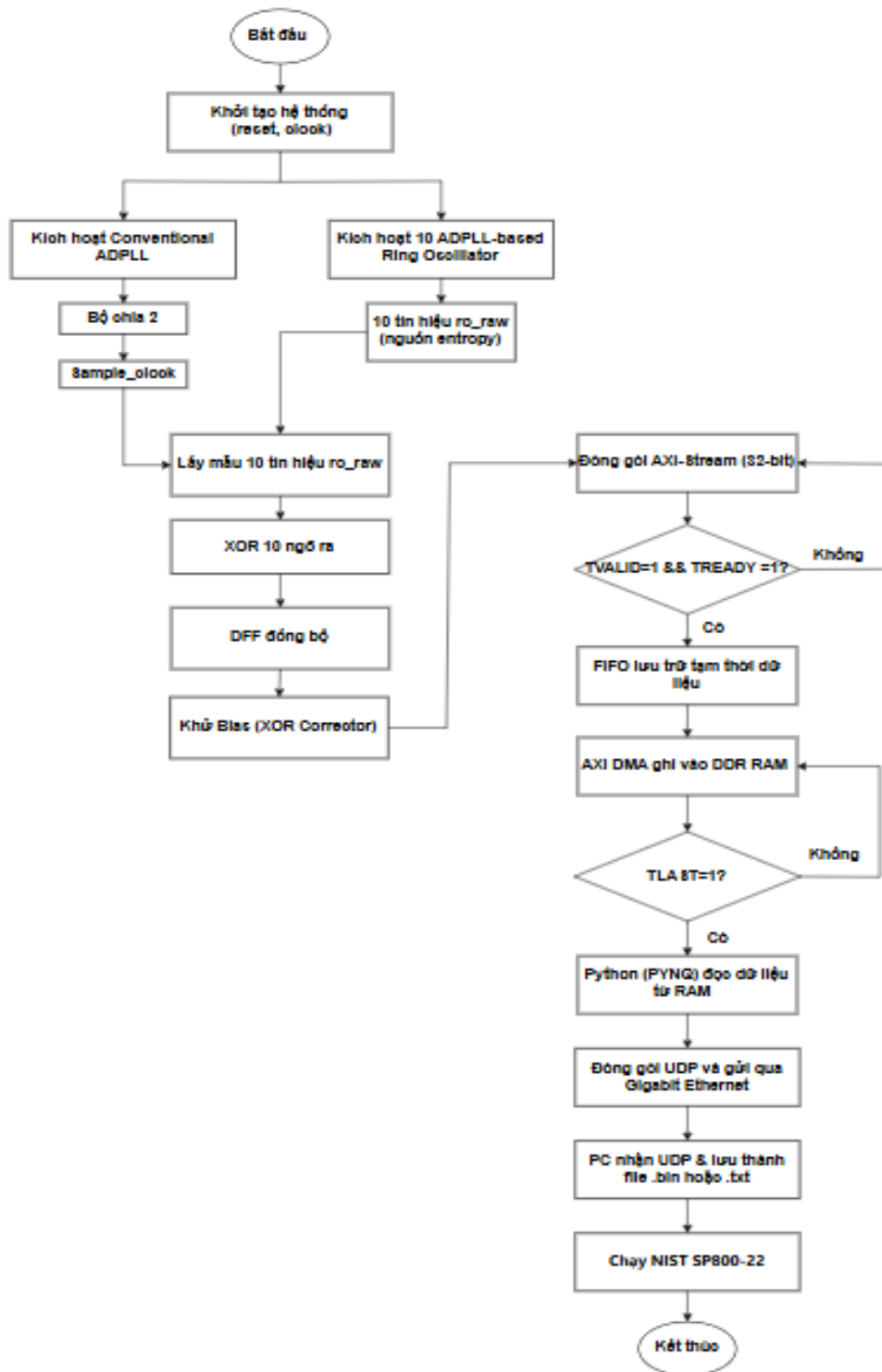
Khối lấy mẫu và tạo bit ngẫu nhiên thô: Khối lấy mẫu sử dụng 10 bộ DFF để lấy mẫu đồng thời các tín hiệu dao động được tạo ra từ 10 bộ ADPLL trong khối tạo entropy dưới tác động của xung clock lấy mẫu. Các bit thu được sau quá trình lấy mẫu được đưa qua một cổng XOR nhằm kết hợp entropy từ nhiều nguồn dao động khác nhau. Kết quả sau XOR tiếp tục được chốt bởi một DFF đồng bộ để tạo thành chuỗi bit ngẫu nhiên thô (Raw Random Bit). Chất lượng của chuỗi bit này phụ thuộc trực tiếp vào mức độ jitter và tính độc lập giữa các nguồn dao động trong hệ thống.

Khối Post-Processing: Chuỗi Raw Random Bit sau khi lấy mẫu vẫn có thể tồn tại hiện tượng thiên lệch xác suất giữa logic ‘0’ và logic ‘1’. Vì vậy hệ thống sử dụng bộ hiệu chỉnh XOR Corrector nhằm giảm bias và nâng cao chất lượng thống kê của dữ liệu ngẫu nhiên. Đầu ra của khối này là chuỗi Random Bit Stream được sử dụng cho các bước đánh giá tiếp theo.

Khối thu thập dữ liệu: đảm nhiệm việc truyền chuỗi bit ngẫu nhiên từ lõi TRNG trên FPGA (Programmable Logic) đến máy tính PC. Dữ liệu được đóng gói theo giao thức AXI4-Stream, lưu tạm qua bộ nhớ FIFO, sau đó được tự động truyền trực tiếp vào bộ nhớ DDR RAM thông qua IP AXI DMA. Chương trình Python chạy trên nền tảng hệ điều hành PYNQ (Processing System) sẽ nạp tệp cấu hình phần cứng (bitstream), đọc dữ liệu từ RAM, đóng gói thành các gói tin UDP và gửi qua cổng Gigabit Ethernet. Tại PC, một chương trình Python khác sẽ nhận luồng dữ liệu UDP này và lưu lại dưới dạng tệp .txt (hoặc .bin) để đưa vào bộ công cụ NIST SP 800-22 đánh giá tính ngẫu nhiên.

3.2.2 Lưu đồ hoạt động của hệ thống

Hình 3.2 thể hiện lưu đồ hoạt động của hệ thống TRNG



Hình 3.2: Lưu đồ hoạt động của hệ thống TRNG

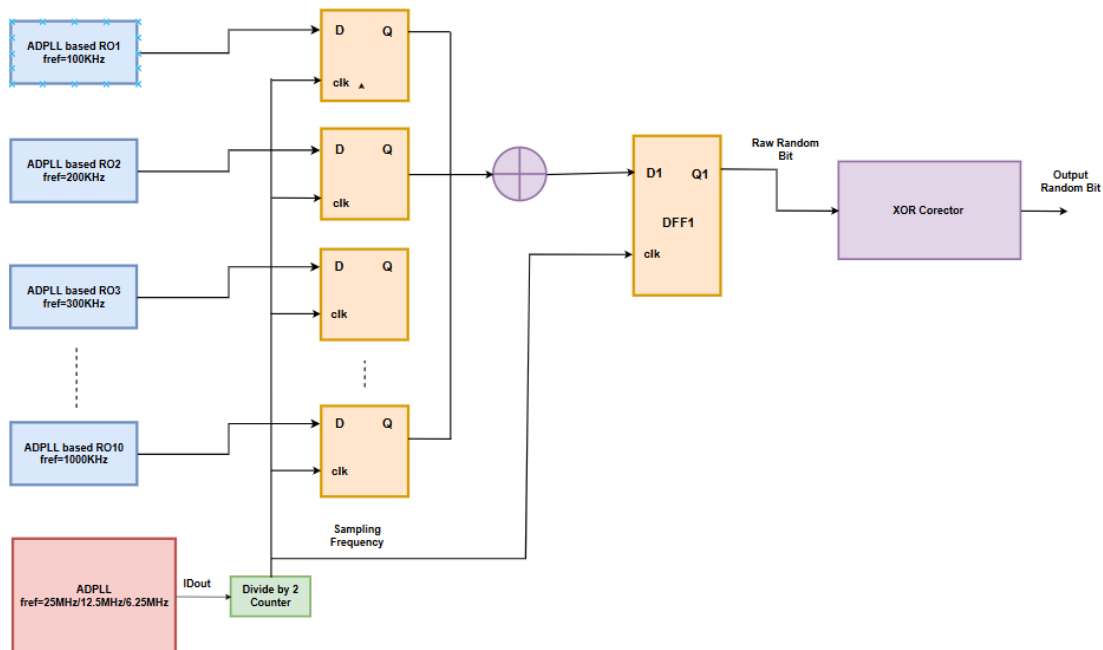
Sau khi hệ thống được khởi tạo, các bộ ADPLL bắt đầu dao động và tạo ra nguồn entropy thông qua hiện tượng Timing Jitter. Các tín hiệu dao động được lấy mẫu bởi các D Flip-Flop đi qua một cổng XOR nhằm kết hợp entropy từ nhiều nguồn dao động khác nhau. Sau đó, một DFF đồng bộ để tạo thành chuỗi

bit ngẫu nhiên thô (Raw Random Bit). Chuỗi bit này tiếp tục được xử lý hậu kỳ bằng mạng XOR nhằm giảm thiên lệch bias trước khi được đóng gói theo giao thức AXI4-Stream. Dữ liệu sau đó được truyền tự động vào thanh ghi RAM DDR thông qua khối AXI DMA. Quá trình thu thập được điều khiển bởi một kịch bản Python trên board PYNQ, bắt đầu từ việc nạp tệp bitstream (.bit và .hwh) xuống FPGA. Khi nhận được tín hiệu ngắt báo hiệu DMA đã truyền xong khối dữ liệu, dữ liệu sẽ được gom thành các gói UDP và truyền liên tục về máy tính (PC) thông qua mạng Gigabit Ethernet. Cuối cùng, một chương trình Python tại máy tính sẽ lắng nghe, nhận đủ dung lượng cần thiết, lưu thành tệp dữ liệu nhị phân (.txt hoặc .bin) và sử dụng làm đầu vào cho bộ công cụ NIST SP 800-22 để đánh giá.

3.3 THIẾT KẾ TỪNG KHỐI

3.3.1 Kiến trúc TRNG

Hình 3.3 trình bày kiến trúc tổng thể của lõi MURO-TRNG được triển khai trong hệ thống. Kiến trúc này được xây dựng dựa trên ba khối chức năng chính gồm: khối tạo entropy (Entropy Generation), khối lấy mẫu (Sampling) và khối xử lý hậu kỳ (Post-Processing).



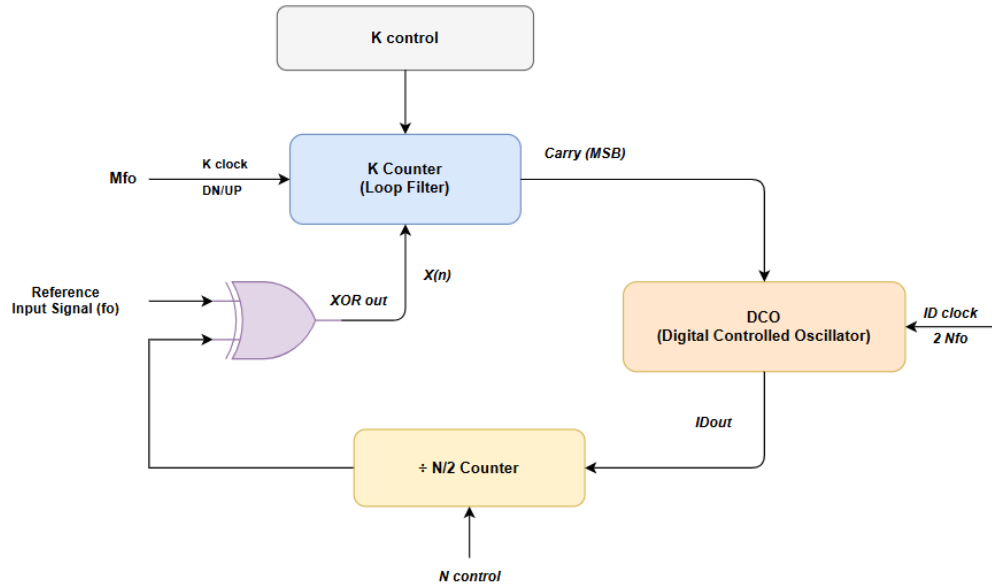
Hình 3.3: Kiến trúc tổng thể của lõi MURO-TRNG

Khối tạo entropy có nhiệm vụ sinh ra các tín hiệu dao động chứa thành phần nhiễu ngẫu nhiên. Các tín hiệu này được đưa đến khối lấy mẫu để chuyển đổi thành chuỗi bit nhị phân thô (Raw Random Bit). Sau đó, dữ liệu tiếp tục được xử lý bởi khối hậu kỳ nhằm cải thiện chất lượng thống kê và giảm hiện tượng thiên lệch bit trước khi tạo ra chuỗi bit ngẫu nhiên đầu ra.

3.3.2 Khối tạo Entropy

3.3.2.1 Tổng quan kiến trúc

Khối tạo entropy đóng vai trò cốt lõi trong kiến trúc MURO-TRNG, quyết định trực tiếp đến chất lượng ngẫu nhiên và các đặc tính an toàn bảo mật của toàn hệ thống. Trong thiết kế này, hệ thống không sử dụng các bộ dao động vòng (Ring Oscillator - RO) truyền thống đơn lẻ vốn dễ bị ảnh hưởng bởi tính tương quan pha tuyến tính. Thay vào đó, kiến trúc vòng khóa pha toàn kỹ thuật số (All Digital Phase Locked Loop - ADPLL) được áp dụng để định hình nguồn entropy.



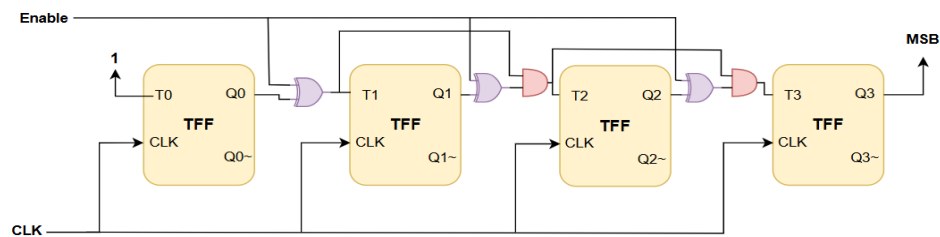
Hình 3.4: Khối tạo entropy sử dụng trong thiết kế

3.3.2.2 Kiến trúc chi tiết và thông số thiết kế

Mỗi khối ADPLL RO được thiết kế từ 4 module thành phần, kết nối theo đồ thị luồng dữ liệu khép kín: Bộ tách pha, Bộ đếm K (đóng vai trò Bộ lọc vòng), Bộ dao động điều khiển số (DCO) và Bộ đếm chia N/2.

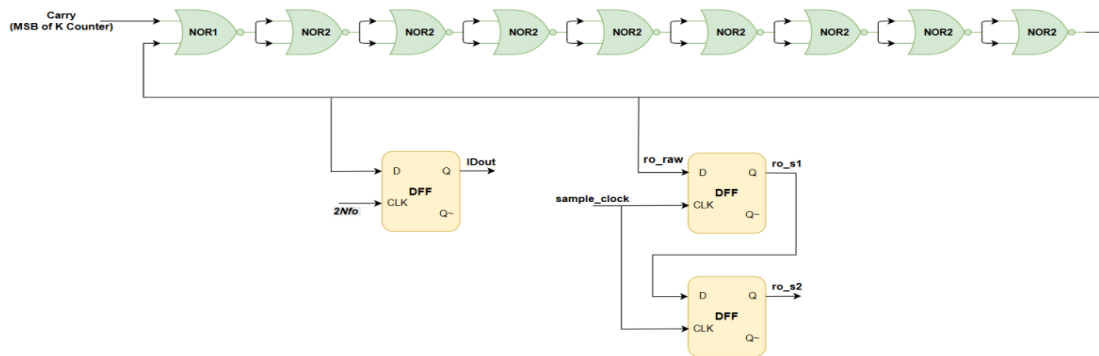
Bộ tách pha (Phase Detector): Triển khai bằng một cổng logic XOR đơn lẻ nhằm so sánh sự chênh lệch pha giữa tín hiệu tham chiếu đầu vào (f_o) và tín hiệu phản hồi từ bộ DCO. Khi hai tín hiệu lệch pha, ngõ ra cổng XOR ở mức logic thấp sẽ kích hoạt bộ đếm K hoạt động.

Bộ lọc vòng dựa trên bộ đếm K: Hoạt động như một bộ lọc vòng kỹ thuật số sử dụng xung nhịp hệ thống bằng M_{fo} để tích lũy sai số pha từ cổng XOR. Khi bộ đếm tích lũy đạt ngưỡng giá trị K cấu hình sẵn, bit có trọng số lớn nhất (MSB/Carry) sẽ dịch chuyển trạng thái để điều khiển khối DCO.



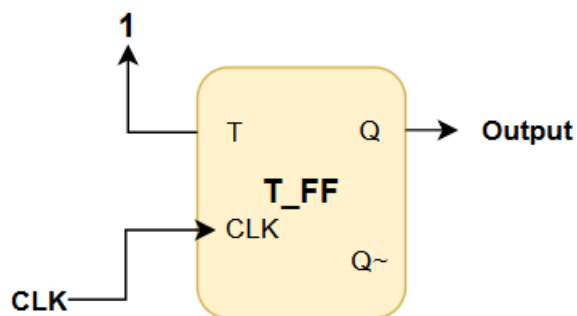
Hình 3.5: Bộ đếm k_counter 2 chế độ up và down

Bộ dao động điều khiển số (DCO): Mạch gồm ba phần như Hình 3.6. Chuỗi 9 cổng NOR tạo vòng lặp thuận tổ hợp qua tín hiệu ro_raw, dao động tự do với tần số phụ thuộc vào trễ vật lý của FPGA. DFF đầu tiên hoạt động tại clock 2Nfo lấy mẫu bất đồng bộ ro_raw tạo IDout, đây là nguồn entropy do ro_raw và 2Nfo hoàn toàn không đồng bộ. Hai DFF còn lại (ro_s1, ro_s2) đều hoạt động tại sample_clock, tạo thành bộ đồng bộ hai tầng (two-stage synchronizer) để xử lý CDC từ domain 2Nfo sang domain sample_clock, loại bỏ nguy cơ metastability trước khi đưa tín hiệu vào khối lấy mẫu.



Hình 3.6: Cấu trúc DCO trong ADPLL

Bộ đếm chia N/2 (Divide by N/2 Counter): Cấu thành từ các T-Flip Flop (T-FF), nhận trực tiếp tần số tốc độ cao Idout từ DCO để chia nhỏ theo tỷ lệ N/2, sau đó hồi tiếp về bộ tách pha nhằm duy trì vòng lặp ADPLL luôn ở trạng thái "gần như khóa".



Hình 3.7: T-Flip Flop trong Bộ đếm chia N/2

Cấu hình thông số thiết kế: Để tạo ra sự đa dạng về entropy, hệ thống sử dụng 10 bộ dao động (RO1 đến RO10) hoạt động với 10 tần số tham chiếu (fo)

khác nhau, trải đều từ 100 KHz đến 1000 KHz (bước nhảy 100 KHz),. Toàn bộ 10 module này đều được đồng bộ với cùng một bộ tham số phần cứng:

Bảng 3.1: Tổng hợp tần số tham chiếu 10 bộ dao động và thông số liên quan

3.3.2.3 Nguyên lý hoạt động

STT	Ký hiệu Module	Tần số tham chiếu gốc (fo)	Tần số xung nhịp K- counter (M x fo)	Tần số xung nhịp DCO (2N x fo)
1	RO1	100 kHz	800 kHz	800 kHz
2	RO2	200 kHz	1600 kHz	1600 kHz
3	RO3	300 kHz	2400 kHz	2400 kHz
4	RO4	400 kHz	3200 kHz	3200 kHz
5	RO5	500 kHz	4000 kHz	4000 kHz
6	RO6	600 kHz	4800 kHz	4800 kHz
7	RO7	700 kHz	5600 kHz	5600 kHz
8	RO8	800 kHz	6400 kHz	6400 kHz
9	RO9	900 kHz	7200 kHz	7200 kHz
10	RO10	1000 kHz	8000 kHz	8000 kHz

Nguyên lý tạo entropy của khối dựa trên cơ chế bám đuôi pha liên tục trong một vòng lặp kín. Ban đầu, bộ tách pha XOR thực hiện so sánh tín hiệu tham chiếu đầu vào (fo) với tín hiệu phản hồi đã qua bộ chia N/2 từ DCO. Khi xuất hiện sự sai lệch pha khiến đầu ra cổng XOR ở mức logic thấp, tín hiệu này kích hoạt bộ lọc vòng K-counter đếm lên. Bộ đếm được cấu hình với K=4 (đếm 4 bit), mỗi khi đếm đủ 16 chu kỳ xung nhịp, tín hiệu Carry (MSB) thay đổi trạng thái và kích hoạt chuỗi 9 cổng NOR bên trong khối DCO. Dưới tác động của tín hiệu Carry và cơ chế phản hồi qua tín hiệu ro_raw (vòng lặp thuần tổ hợp NOR9

→ NOR1), khối DCO tạo ra tín hiệu dao động tần số cao tại đầu ra IDout. Tín hiệu IDout tiếp tục được đưa qua bộ chia $N/2$ để hồi tiếp về cổng XOR, khép kín vòng lặp.

Điểm mấu chốt tạo ra sự ngẫu nhiên là ADPLL trong cấu trúc này không được thiết kế để khoá pha hoàn hảo. Thay vào đó, hệ thống liên tục duy trì trạng thái gần khoá, vòng lặp liên tục thực hiện bám đuổi và điều chỉnh bù trừ pha. Chính sự bất ổn định vi mô trong quá trình này tạo ra jitter pha tại đầu ra IDout. Các tín hiệu chứa jitter từ 10 module RO độc lập trở thành nguồn entropy đầu vào cho khối lấy mẫu ở giai đoạn tiếp theo.

3.3.3 Khối lấy mẫu và tạo bit ngẫu nhiên thô

3.3.3.1 Tổng quan kiến trúc

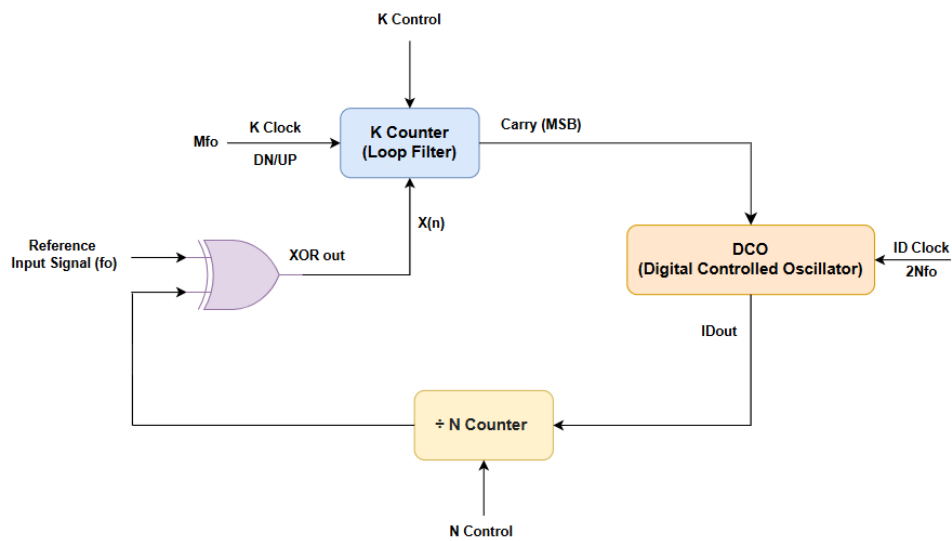
Khối lấy mẫu có nhiệm vụ chuyển đổi các tín hiệu chứa jitter từ 10 bộ sinh entropy dựa trên kiến trúc ADPLL thành chuỗi bit nhị phân. Để thực hiện quá trình này, hệ thống sử dụng một tín hiệu xung nhịp lấy mẫu (Sampling Clock) được tạo ra từ khối ADPLL truyền thống. Tín hiệu Sampling Clock được phân phối đồng thời tới toàn bộ các bộ D-Flip Flop (DFF) trong khối lấy mẫu nhằm đảm bảo quá trình thu thập dữ liệu diễn ra đồng bộ.

Trong kiến trúc đề xuất, đầu ra IDout của 10 bộ ADPLL được đưa vào 10 bộ DFF lấy mẫu hoạt động song song. Do các tín hiệu IDout chứa thành phần jitter pha phát sinh từ các nguồn nhiễu vật lý bên trong FPGA, kết quả lấy mẫu tại các thời điểm khác nhau sẽ thay đổi ngẫu nhiên theo thời gian. Mười bit dữ liệu thu được sau quá trình lấy mẫu tiếp tục được đưa qua mạng logic XOR để thực hiện quá trình trộn entropy từ nhiều nguồn khác nhau. Kết quả đầu ra của XOR được đồng bộ thông qua một DFF cuối cùng nhằm tạo ra chuỗi bit ngẫu nhiên thô (Raw Random Bit).

Khối lấy mẫu đóng vai trò cầu nối giữa tầng sinh entropy và tầng xử lý hậu kỳ của hệ thống. Chất lượng của chuỗi Raw Random Bit thu được phụ thuộc trực tiếp vào mức độ jitter tồn tại trong các tín hiệu IDout cũng như hiệu quả của quá trình lấy mẫu và trộn entropy.

3.3.3.2 Kiến trúc chi tiết và thông số thiết kế

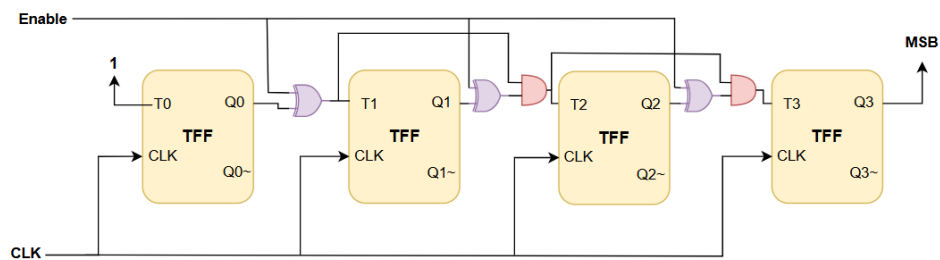
Khối tạo tần số lấy mẫu được triển khai theo cấu trúc của một bộ ADPLL truyền thống (Conventional ADPLL). Phân hệ phần cứng này bao gồm 4 khối chức năng cơ bản kết hợp vòng kín: Bộ tách pha, bộ đếm K (đóng vai trò bộ lọc vòng), bộ dao động điều khiển số truyền thống (Conventional DCO) và bộ đếm chia N.



Hình 3.8: Khối tạo tần số lấy mẫu theo kiến trúc ADPLL truyền thống

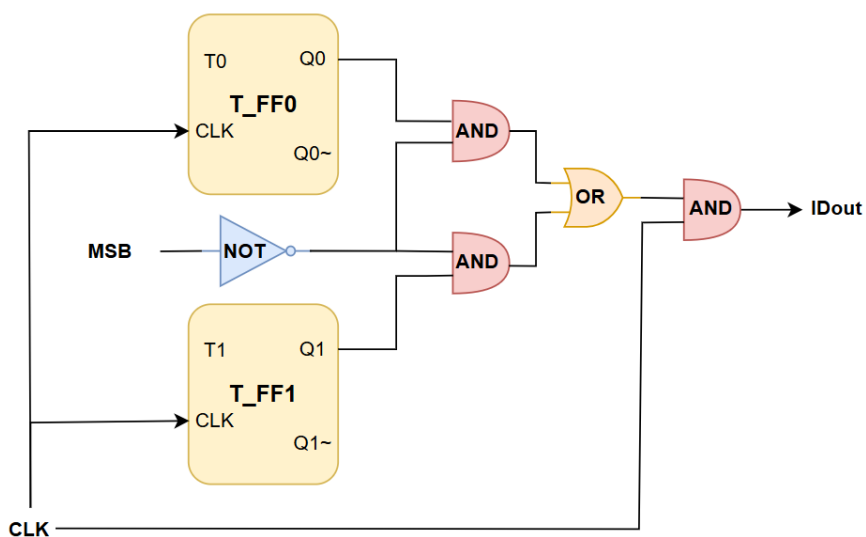
Bộ tách pha (Phase Detector): Triển khai bằng một cổng logic XOR đơn lẻ nhằm so sánh sự chênh lệch pha giữa tín hiệu tham chiếu đầu vào (f_0) và tín hiệu phản hồi từ bộ DCO. Khi hai tín hiệu lệch pha, ngõ ra cổng XOR ở mức logic thấp sẽ kích hoạt bộ đếm K hoạt động.

Bộ lọc vòng dựa trên bộ đếm K: Hoạt động như một bộ lọc vòng kỹ thuật số sử dụng xung nhịp hệ thống bằng Mf_0 để tích lũy sai số pha từ cổng XOR. Khi bộ đếm tích lũy đạt ngưỡng giá trị K cấu hình sẵn, bit có trọng số lớn nhất (MSB/Carry) sẽ dịch chuyển trạng thái để điều khiển khối DCO.



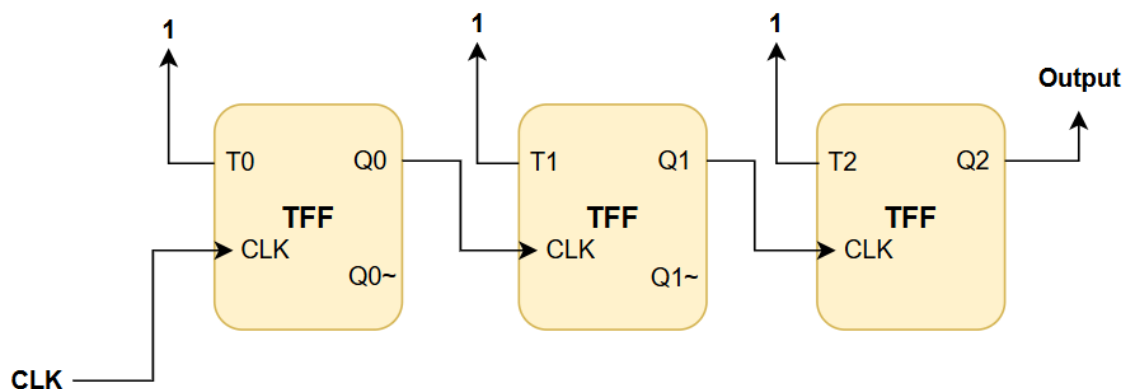
Hình 3.9: Cấu trúc K-counter 2 chế độ up và down [3]

Khối DCO truyền thống (Conventional DCO): Mạch được cấu thành từ các bộ đếm bằng T-Flip Flop (T-FF) kết hợp cùng lưới logic tổ hợp AND, OR và NOT để thực hiện quét và dịch pha.



Hình 3.10: Cấu tạo DCO truyền thống

Bộ đếm chia N: Nằm trên đường phản hồi của vòng lặp Conventional ADPLL. Khối này được cấu thành từ các T-Flip Flop (T-FF) mắc nối tiếp, nhận trực tiếp tín hiệu ID_out từ DCO để thực hiện chia tần số trước khi hồi tiếp về cổng XOR của bộ tách pha.

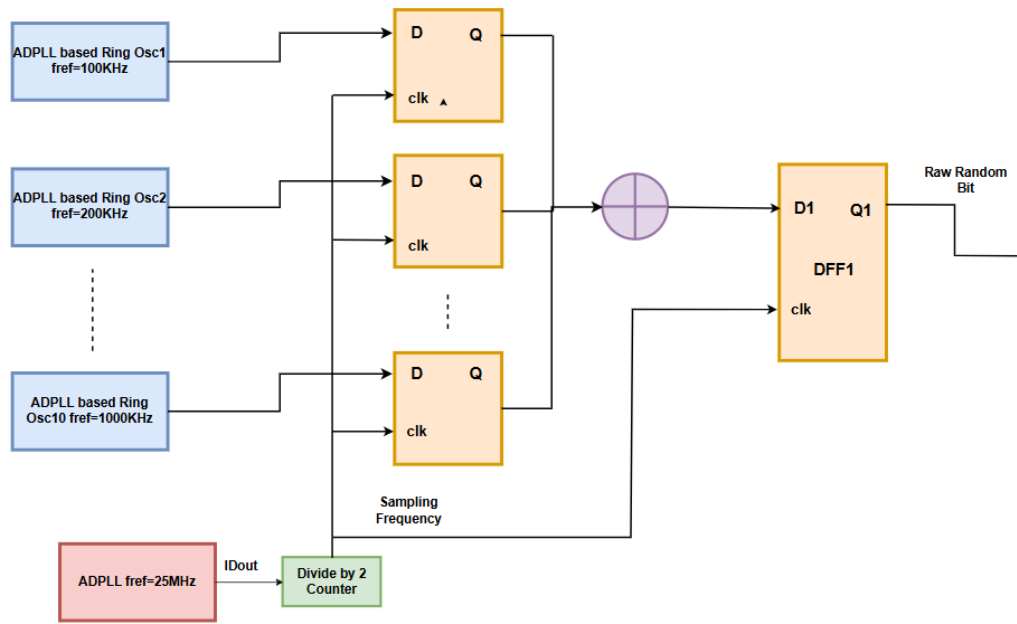


Hình 3.11: Cấu tạo của bộ chia N

Cấu hình thông số thiết kế: Hệ thống được khảo sát tại 3 mức tần số tham chiếu đầu vào khác nhau bao gồm: 25 MHz, 12.5 MHz và 6.25 MHz nhằm đánh giá toàn diện sự ảnh hưởng của tốc độ lấy mẫu đến chất lượng entropy thô.

Bảng 3.2: Tần số tham chiếu và thông số liên quan

Cấu hình	Tần số tham chiếu (f_0) MHz	Hệ số K / M / N	Tần số xung nhịp K-counter ($M \times f_0$) MHz	Tần số xung nhịp DCO ($2N \times f_0$) MHz	Tần số ngõ ra DCO (IDout) MHz	Tần số lấy mẫu cuối cùng (IDout / 2) MHz
1	25.00	4 / 16 / 8	400	400	200	100
2	12.50	4 / 16 / 8	200	200	100	50
3	6.25	4 / 16 / 8	100	100	50	25



Hình 3.12: Kiến trúc khối lấy mẫu và tạo Raw Random Bit

Sau khi xung nhịp lấy mẫu được tạo ra, hệ thống sử dụng 10 bộ D-Flip Flop (DFF) để lấy mẫu đồng thời tín hiệu IDout từ 10 khối tạo entropy. Mỗi DFF nhận một tín hiệu dao động độc lập tại chân D và sử dụng chung tín hiệu Sampling Clock tại chân Clock.

Ngõ ra của 10 DFF được đưa vào một cổng XOR nhằm kết hợp thông tin từ nhiều nguồn entropy khác nhau. Việc thực hiện phép XOR trên nhiều tín hiệu giúp tăng mức độ khuếch tán của nhiễu jitter và giảm ảnh hưởng của các nguồn entropy có độ thiên lệch cao.

Kết quả từ XOR tiếp tục được đưa qua một DFF đồng bộ cuối cùng để ổn định dữ liệu và tạo ra chuỗi bit ngẫu nhiên thô (Raw Random Bit). Chuỗi bit này là đầu vào trực tiếp của khối xử lý hậu kỳ (Post-Processing) được trình bày ở mục tiếp theo.

3.3.3.3 Nguyên lý hoạt động

Khối tạo tần số lấy mẫu hoạt động theo cơ chế phản hồi vòng kín của Conventional ADPLL. Bộ tách pha XOR so sánh tín hiệu tham chiếu với tín hiệu phản hồi từ bộ chia N, sai lệch pha được tích lũy qua K-counter. Khi giá trị

đếm đạt ngưỡng, tín hiệu Carry tác động lên Conventional DCO để điều chỉnh tần số dao động, duy trì IDout ổn định ở tần số mong muốn. IDout tiếp tục đi qua bộ chia 2 (T-FF) tạo Sampling Clock có duty cycle xấp xỉ 50%.

Sampling Clock được phân phối đồng thời tới 10 DFF để lấy mẫu 10 tín hiệu IDout từ 10 ADPLL ring oscillator. Do các tín hiệu này chứa jitter pha tích lũy, kết quả lấy mẫu tạo ra các bit ngẫu nhiên. Ngõ ra 10 DFF được XOR lại thành một bit tổng hợp, sau đó qua một DFF đồng bộ để ổn định tín hiệu, tạo thành chuỗi Raw Random Bit đưa vào khối Post-Processing.

3.3.4 Khối xử lý hậu kỳ (Post-Processing)

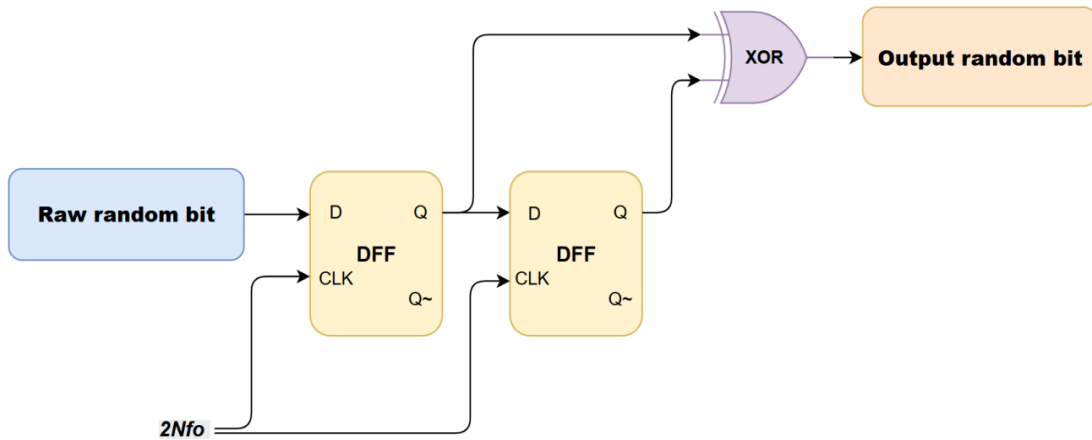
3.3.4.1 Tổng quan kiến trúc

Trong hệ thống TRNG thực tế trên FPGA, chuỗi bit ngẫu nhiên thô (raw random bits) thu được từ khối tạo entropy thường không đạt được độ cân bằng hoàn hảo giữa mức logic '0' và '1'. Sự mất cân bằng này (gọi là hiện tượng thiên lệch - bias) chủ yếu bắt nguồn từ các yếu tố vật lý không đồng nhất của phần cứng, sự bất đối xứng trong định tuyến (routing) của mạch FPGA, hoặc do nhiễu hệ thống mang tính tất định. Để đảm bảo chuỗi bit đầu ra vượt qua các bài kiểm tra thống kê khắt khe của NIST SP 800-22 và có thể ứng dụng trong mật mã học, một khối xử lý hậu kỳ (Post-Processing) là thành phần bắt buộc phải có nhằm loại bỏ bias và cải thiện thuộc tính thống kê của dữ liệu.

3.3.4.2 Kiến trúc chi tiết

Để thực hiện chức năng khử thiên lệch, thiết kế một bộ sửa lỗi XOR Corrector với thiết kế phần cứng vô cùng tối giản nhằm tối ưu hóa diện tích. Mạch Post-Processing này được xây dựng dựa trên một thanh ghi dịch 2-bit (2-bit shift register). Mạch bao gồm hai bộ D-Flip Flop (DFF) được mắc nối tiếp nhau: đầu vào (chân D) của DFF thứ nhất nhận trực tiếp chuỗi bit ngẫu nhiên thô, đầu ra (chân Q) của DFF thứ nhất được nối trực tiếp vào đầu vào của DFF thứ hai. Ngõ ra của cả hai DFF này sau đó được kết nối với hai ngõ vào của một cổng logic XOR để tạo ra bit ngẫu nhiên cuối cùng và mạch này hoạt động đồng bộ

dựa trên xung nhịp điều khiển là tín hiệu Sampling Frequency được tạo ra từ khối ADPLL truyền thống.



Hình 3.13: Khối tạo xử lý hậu kỳ sử dụng trong thiết kế

3.3.4.3 Nguyên lý hoạt động

Nguyên lý hoạt động của XOR Corrector dựa trên phép toán triệt tiêu thiên lệch tĩnh giữa các bit dữ liệu thô liên tiếp. Tại mỗi sườn lên (rising edge) của xung nhịp lấy mẫu, hệ thống sẽ chốt trạng thái và dịch chuyển các bit dữ liệu. Cấu trúc thanh ghi dịch 2-bit cho phép cổng XOR ở ngõ ra thực hiện phép toán logic trên hai bit ngẫu nhiên liên tiếp nhau trong chuỗi thời gian. Về mặt toán học xác suất, nếu chuỗi bit thô ban đầu có xu hướng thiên lệch (ví dụ xác suất xuất hiện bit '1' lớn hơn bit '0'), việc thực hiện phép XOR hai bit độc lập liên tiếp sẽ giúp cân bằng lại phân bố xác suất, ép tỷ lệ xuất hiện của '0' và '1' tiến sát về mức lý tưởng 50% giúp chuỗi bit đạt độ ngẫu nhiên cao trước khi truyền ra ngoài vì để thực hiện kiểm tra chất lượng bit.

3.3.5 Khối thu thập dữ liệu

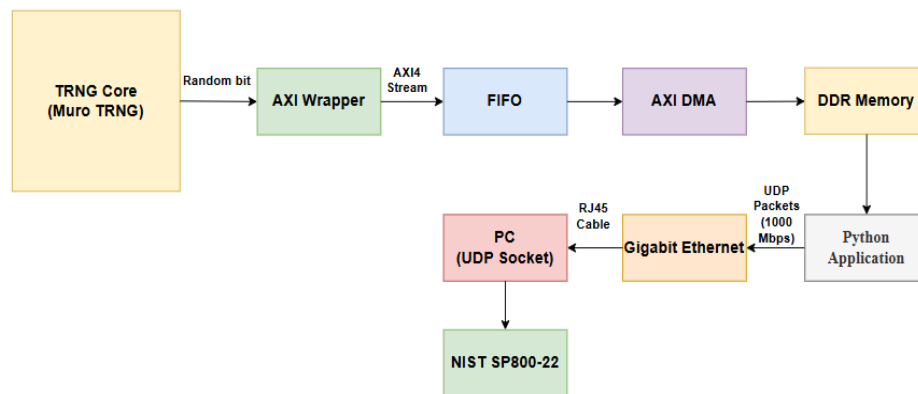
3.3.5.1 Tổng quan kiến trúc

Sau khi trải qua quá trình lấy mẫu và xử lý hậu kỳ, chuỗi bit ngẫu nhiên được chuyển tới khối thu thập dữ liệu nhằm phục vụ việc lưu trữ và đánh giá chất lượng ngẫu nhiên trên máy tính. Do hệ thống MURO-TRNG có tốc độ sinh dữ liệu rất cao, việc truyền dữ liệu theo các chuẩn giao tiếp cơ bản (như USB-UART) sẽ gây thất cổ chai, mất mát dữ liệu và làm giảm hiệu năng.

Để giải quyết bài toán này, hệ thống sử dụng giải pháp luồng dữ liệu (Data Flow) tự động và tối ưu từ phần cứng (PL) lên phần mềm (PS). Kiến trúc truyền tải sử dụng giao thức AXI4-Stream kết hợp bộ đệm FIFO, bộ điều khiển DMA phần cứng và mạng Gigabit Ethernet. Dữ liệu ngẫu nhiên thô được truyền và lưu trữ với tốc độ cao, độ tin cậy tuyệt đối, đảm bảo tính thời gian thực (real-time) và không xuất hiện tình trạng rơi rớt bit.

3.3.5.2 Kiến trúc chi tiết

Khối thu thập dữ liệu được xây dựng từ các thành phần chính như hình 3.14:



Hình 3.14: Kiến trúc khối thu thập dữ liệu

AXI4-Stream Interface & FIFO Generator (Tầng PL): Chuỗi bit ngẫu nhiên từ lõi TRNG được gom đủ thành các từ dữ liệu 32-bit và cấp cờ hợp lệ để đẩy vào FIFO Generator. FIFO đóng vai trò bộ đệm trung gian để tích lũy dữ liệu, tránh hiện tượng tràn bộ đệm khi tốc độ sinh dữ liệu và tốc độ ghi không đồng bộ.

AXI DMA (Tầng PL): Hoạt động như một cỗ máy bơm dữ liệu phần cứng chuyên dụng. Khối DMA chủ động hút dữ liệu từ FIFO bằng cơ chế AXI-Stream và ghi thẳng vào vùng nhớ RAM DDR3 của hệ thống thông qua Direct Memory Access mà không cần sự can thiệp của CPU.

DDR Memory: Vùng nhớ RAM DDR3 (512 MB) là nơi tập kết khối dữ liệu trước khi được phần mềm bốc dỡ và chuyển tiếp ra ngoại vi.

PYNQ Python Application (Tầng PS): Chương trình Python hoạt động trên hệ điều hành PYNQ Linux của chip ARM. Khởi đầu, phần mềm sử dụng thư viện pynq để nạp tệp cấu hình phần cứng .bit xuống lõi FPGA. Ứng dụng này sử dụng cơ chế khóa chốt phần cứng, buộc CPU vào trạng thái chờ cho đến khi DMA báo cờ hoàn tất (TLAST). Khi dữ liệu đã sẵn sàng, Python đọc dữ liệu từ RAM, đóng gói thành các gói UDP và phát liên tục qua cổng Gigabit Ethernet về máy tính.

PC Python Application: Tại máy tính, một chương trình Python (nhận data) sẽ mở socket lắng nghe trên cổng mạng UDP. Khi luồng dữ liệu đổ về, chương trình sẽ tiến hành ghi đè liên tục thành tệp nhị phân (.txt hoặc .bin) cho đến khi đạt mục tiêu dung lượng đề ra.

3.3.5.3 Nguyên lý hoạt động

Quá trình truyền dữ liệu bắt đầu từ việc chương trình Python trên board PYNQ-Z1 sử dụng lệnh Overlay để nạp bitstream cấu hình hệ thống TRNG lên FPGA và cấp phát vùng nhớ đệm trên RAM DDR.

Ở tầng dưới, khi lõi TRNG sinh các bit ngẫu nhiên, chúng được đẩy liên tục vào AXI Data FIFO. Khi FIFO đã có dữ liệu, bộ điều khiển AXI DMA sẽ kích hoạt và thực hiện truyền trực tiếp khối dữ liệu đó vào RAM DDR3. Trong suốt tiến trình này, CPU hoàn toàn không tham gia xử lý nên băng thông và hiệu năng hệ thống đạt mức tối đa.

Mã Python trên lõi ARM ở trạng thái chờ (wait()) sẽ lập tức được đánh thức khi nhận tín hiệu ngắt phần cứng từ DMA báo hiệu dữ liệu đã ghi xong. Chương trình sẽ lấy khối dữ liệu nhị phân này, chia nhỏ và liên tục gửi tới IP của máy tính thông qua chuẩn giao tiếp mạng Gigabit Ethernet với tốc độ cao. Cuối cùng, phần mềm Python trên PC đóng vai trò bộ thu thập, nhận các gói UDP thông qua Socket, nối chúng lại và ghi vào tệp .txt hoặc .bin. Tệp tin này sẵn sàng để chạy trong công cụ phân tích thống kê NIST SP 800-22 nhằm xác thực chất lượng nguồn entropy.

CHƯƠNG 4 KẾT QUẢ THỰC HIỆN

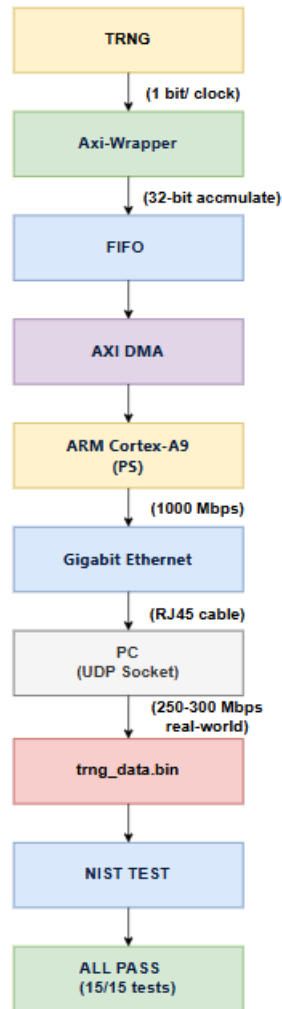
Chương này trình bày toàn bộ kết quả thực hiện của đề tài "Thiết kế và đánh giá bộ sinh số ngẫu nhiên MURO-TRNG trên FPGA PYNQ-Z1 (Xilinx XC7Z020 Zynq 7000). Nội dung bao gồm: thiết kế và mô phỏng từng module; triển khai phần cứng; kết quả tổng hợp tài nguyên, timing, công suất với ba tần số tham chiếu; phân tích chất lượng ngẫu nhiên theo chuẩn NIST SP800-22.

4.1 KẾT QUẢ THIẾT KẾ HỆ THỐNG

Chúng tôi đã triển khai kiến trúc MURO-TRNG trên board PYNQ-Z1 (Xilinx XC7Z020, họ Zynq-7000) sử dụng Xilinx Vivado 2025.2, với toàn bộ RTL được viết bằng Verilog. Hệ thống bao gồm MURO-TRNG core chạy trên PL và pipeline thu thập dữ liệu kết nối sang PS để đưa bitstream về PC kiểm thử.

4.1.1 Tổng quan kiến trúc của hệ thống và pipeline thu thập dữ liệu

Chúng tôi đã xây dựng một pipeline hoàn chỉnh để thu thập bitstream ngẫu nhiên từ MURO-TRNG và chuyển về PC phục vụ kiểm thử NIST SP800-22, được minh họa trong Hình 4.1 dưới đây.



Hình 4.1: Pipeline thu thập dữ liệu

Từ Hình 4.1, mô tả pipeline hoạt động theo các bước sau:

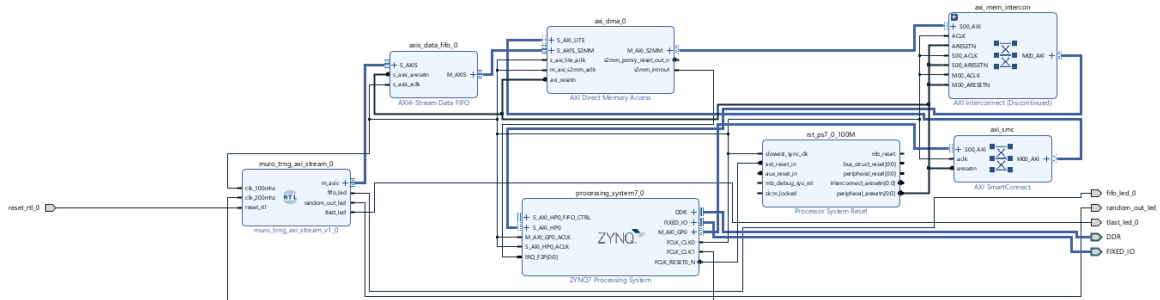
- (1) TRNG Core: MURO-TRNG chạy trên PL, lấy 1 bit ngẫu nhiên mỗi clock cycle thông qua giao diện AXI-Stream.
- (2) AXI Wrapper: thiết kế AXI Wrapper để gom 32 bit liên tiếp (32-bit accumulate) thành một word 32-bit rồi đẩy vào FIFO.
- (3) FIFO: Sử dụng bộ đệm FIFO để giữ dữ liệu cho AXI DMA đọc theo batch, tránh mất dữ liệu khi PS bận.
- (4) AXI DMA: Tích hợp AXI DMA engine để chuyển dữ liệu từ FIFO vào vùng nhớ DDR của ARM Cortex-A9 mà không cần CPU xử lý từng byte.

(5) ARM Cortex-A9 (PS): Viết phần mềm Python trên PS để đọc dữ liệu từ DMA buffer, đóng gói thành UDP packet và gửi qua Gigabit Ethernet.

(6) Gigabit Ethernet → PC (UDP Socket): Dữ liệu được truyền qua cáp RJ45 đạt khoảng 250–300 Mbps thực tế. Phía PC, viết chương trình nhận qua UDP socket và ghi vào file trng_data.bin.

(7) NIST SP800-22 Test: Nạp file trng_data.bin vào bộ kiểm thử NIST SP800-22 và tiến hành test.

Để hiện thực hoá pipeline trên, Chúng tôi sử dụng Vivado IP Integrator (Block Design) để kết nối các IP core với nhau ở cấp độ hệ thống. Hình 4.2 thể hiện toàn bộ Block Design của hệ thống trên PYNQ-Z1.



Hình 4.2: Vivado Block Design – toàn bộ hệ thống MURO-TRNG trên PYNQ-Z1

Block Design bao gồm các thành phần chính sau:

(1) ZYNQ7 Processing System (PS7): Khối ARM Cortex-A9, được Chúng tôi cấu hình kích hoạt AXI HP0 port để AXI DMA truy cập DDR3, kích hoạt Gigabit Ethernet. PS7 cung cấp clock FCLK_CLK0 (100 MHz) làm clock chủ cho toàn bộ PL.

(2) MURO-TRNG Core (Custom IP): Khối RTL tự thiết kế, nhận clock 100 MHz từ PS7 và xuất bitstream ngẫu nhiên qua giao diện AXI4-Stream (TDATA, TVALID, TREADY).

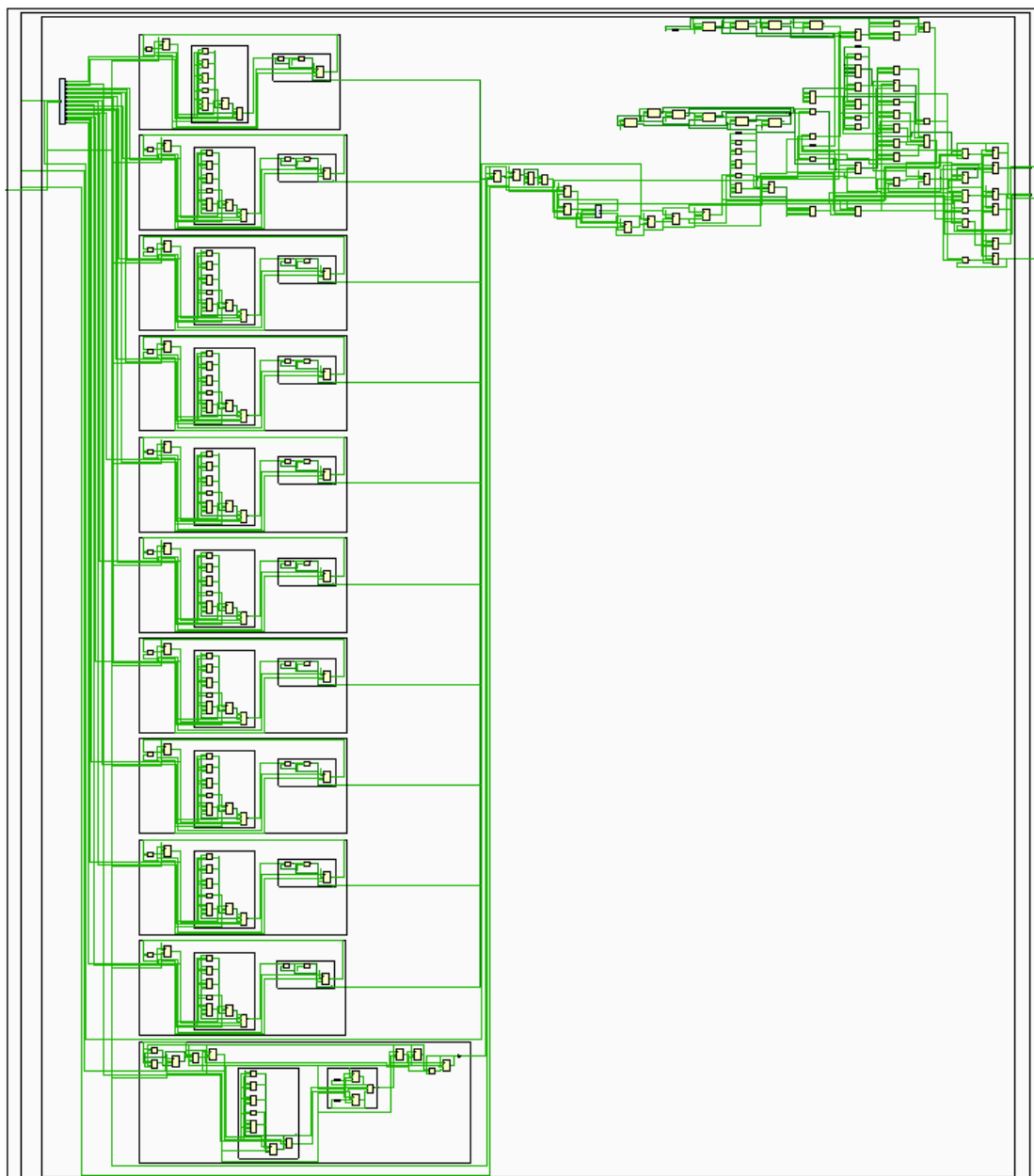
(3) AXI4-Stream FIFO: Bộ đệm FIFO kết nối AXI-Stream output của MURO-TRNG với AXI DMA.

(4) AXI Direct Memory Access (DMA): Nhận dữ liệu từ FIFO qua S2MM channel và ghi trực tiếp vào DDR3 thông qua AXI HP0 port của PS7, không tiêu tốn CPU cycle.

(5) AXI Interconnect: kết nối các AXI master/slave giữa PS7, DMA và FIFO, xử lý địa chỉ và arbitration tự động.

Toàn bộ kết nối clock, reset và AXI bus được Vivado IP Integrator tự động kiểm tra tính hợp lệ (Validate Design), đảm bảo không có xung đột giao thức trước khi Chúng tôi tiến hành synthesis và implementation.

Hình 4.3 Thể hiện schematic tổng quan của khối MURO-TRNG core, bao gồm 10 ADPLL-based ring oscillator, 1 conventional ADPLL sinh sampling clock, 11 DFF lấy mẫu, 1 XOR gate tạo raw random bit và khối post-processing.

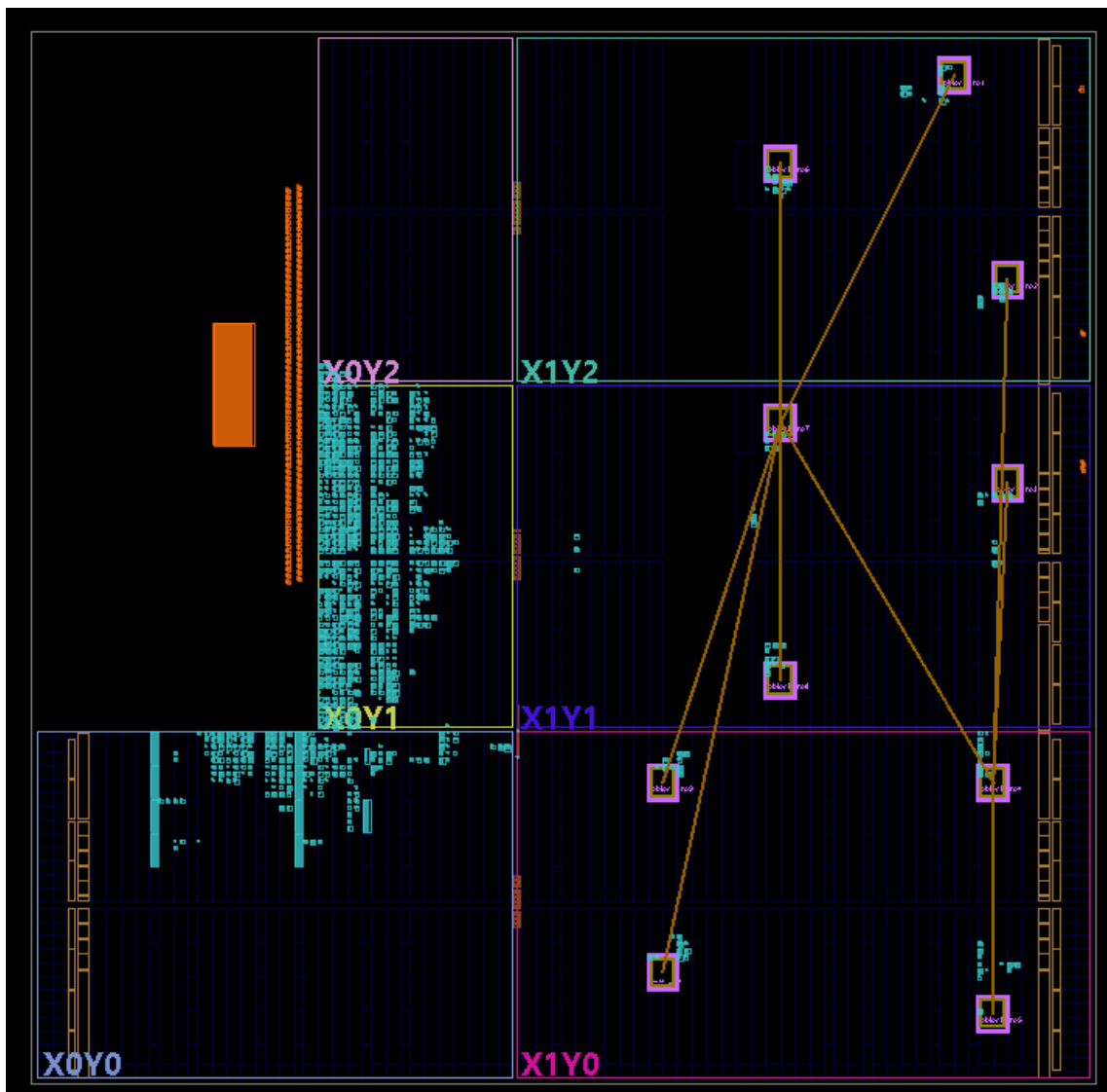


Hình 4.3: Schematic tổng quan của khối MURO-TRNG core

4.1.2 Ràng buộc vị trí và bảo toàn nguồn entropy

Một trong những thách thức lớn nhất khi triển khai kiến trúc MURO-TRNG trên FPGA là sự can thiệp của trình tối ưu hoá trong Vivado. Mặc định, công cụ tổng hợp sẽ có xu hướng tối ưu, rút gọn hoặc gộp chung các khối logic có cấu trúc tương đồng nhau (như các chuỗi NOR chain trong Ring Oscillator). Nếu điều này xảy ra, sự khác biệt vi phân về trễ truyền dẫn giữa các vòng RO – vốn là yếu tố cốt lõi để tạo ra jitter entropy – sẽ bị triệt tiêu hoàn toàn.

Để khắc phục triệt để vấn đề này, nhóm đã áp dụng cơ chế bảo vệ hai lớp. Đầu tiên, tại mức RTL thuộc tính (* DONT_TOUCH = "TRUE" *) nhằm buộc trình tổng hợp phải giữ nguyên cấu trúc gốc của các cổng logic. Tiếp đó, tại mức triển khai vật lý (Implementation), nhóm sử dụng các ràng buộc vị trí trong file XDC để ép mỗi khối ADPLL RO vào một vùng PBLOCK riêng biệt. Sự kết hợp này ngăn chặn tuyệt đối việc Vivado tối ưu hoá gộp chéo, đảm bảo tính độc lập và toàn vẹn của nguồn sinh entropy. Kết quả phân bổ tài nguyên (placement) thực tế trên sơ đồ chip được thể hiện chi tiết trong Hình 4.4.

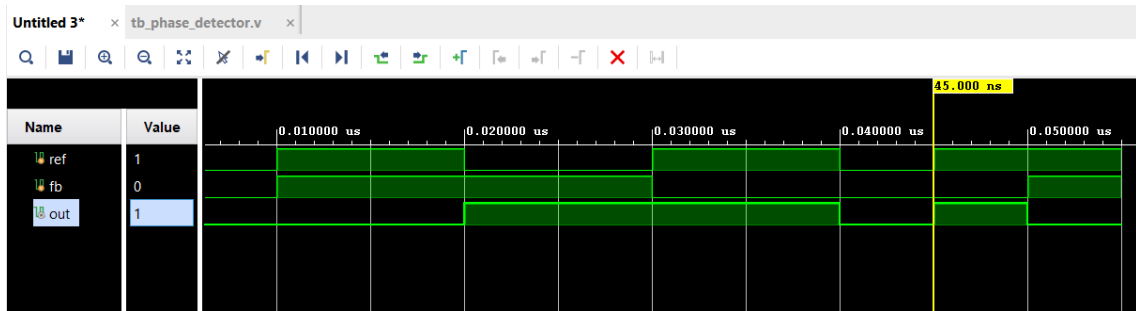


Hình 4.4: Vị trí trên Vivado – các RO được đặt tách biệt bằng PBLOCK

4.2 KẾT QUẢ MÔ PHỎNG TỪNG MODULE

4.2.1 Phase_detector

Chúng tôi tiến hành mô phỏng module phase_detector, vốn thực hiện so sánh pha giữa tín hiệu đầu ra của bộ chia N/2 counter với tín hiệu tham chiếu fref. Khi pha đầu ra trễ hơn tham chiếu, ngõ ra XOR out xuống mức thấp và kích hoạt K counter đếm lên. Kết quả waveform thu được như Hình 4.5.



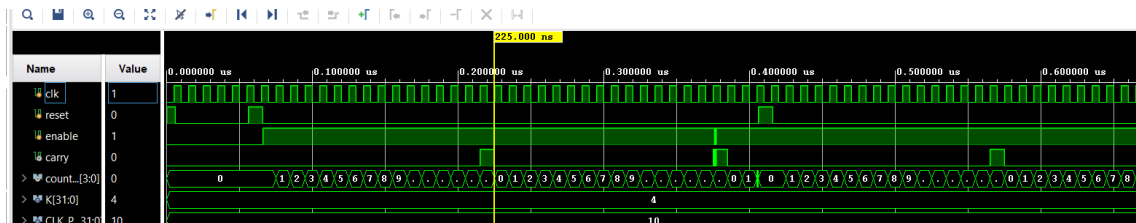
Hình 4.5: Behavioral simulation waveform – module phase_detector

Quan sát waveform:

Tại thời điểm từ 0.01us đến 0.02us, tín hiệu ref và fb đồng pha nên out duy trì ở mức 0. Khoảng thời gian t từ 0.02us đến 0.04us, xuất hiện sai lệch giữa ref và fb, do đó out chuyển lên mức 1 để báo lỗi pha. Qua đó, xác nhận phase detector hoạt động đúng chức năng phát hiện sai lệch pha giữa tín hiệu tham chiếu và tín hiệu hồi tiếp.

4.2.2 K_counter

Tiếp theo, chúng tôi tiến hành mô phỏng module k_counter với clock Mfo. MSB (Carry) của K counter được dùng làm tín hiệu kích DCO.



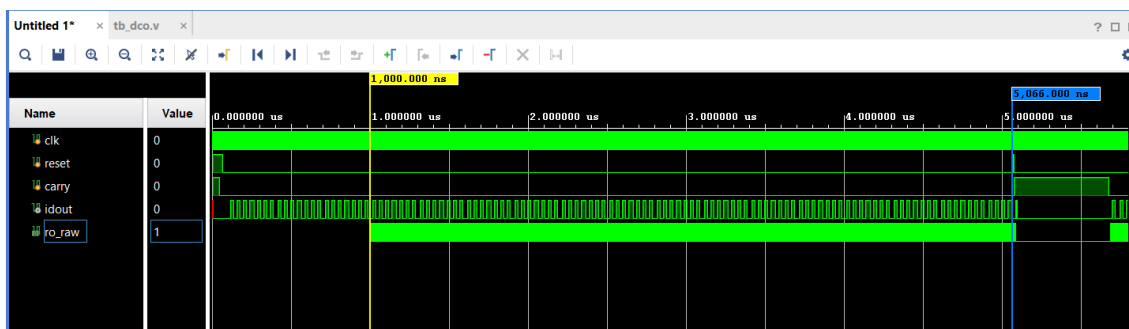
Hình 4.6: Behavioral simulation waveform – module k_counter

Quan sát waveform cho thấy:

Tín hiệu enable: thời điểm t từ 0ns đến 60 ns, enable = 0 nên counter giữ nguyên ở mức 0; sau đó enable lên mức 1, counter bắt đầu đếm. Giá trị count_observe[3:0] tăng tuần tự theo từng cạnh lên của clock theo chuỗi: $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow 15 \rightarrow 0$, cho thấy bộ đếm hoạt động đúng theo cơ chế của module K-Counter (với $K = 4$). Khi counter đạt giá trị cực đại 1111 (15) tại khoảng thời gian $t \approx 215.5\text{ns}$, tín hiệu carry chuyển lên mức 1. Sau chu kỳ clock kế tiếp, counter quay về 0000 và tín hiệu carry trở lại mức 0, xác nhận cơ chế overflow hoạt động chính xác. Trong suốt quá trình mô phỏng, tín hiệu carry chỉ xuất hiện đúng tại thời điểm counter đạt giá trị cực đại, không xuất hiện sai lệch giữa các chu kỳ clock. Kết quả mô phỏng xác nhận module k_counter hoạt động đúng chức năng thiết kế: counter đếm chính xác theo clock Mfo, tín hiệu Carry được tạo đúng thời điểm và sử dụng làm tín hiệu kích ổn định cho khối DCO.

4.2.3 DCO

Chúng tôi tiến hành mô phỏng module DCO theo kiến trúc đã được tối ưu. Khối này bao gồm chuỗi 9 cổng NOR tạo vòng dao động sinh tín hiệu ro_raw, kết hợp với một DFF hoạt động tại xung nhịp clk ($2N_{fo}$) để tạo tín hiệu phản hồi idout. Tần số và trạng thái dao động của mạch chịu sự điều khiển trực tiếp từ tín hiệu carry của K-counter. Dưới đây là kết quả waveform của module DCO:



Hình 4.7: Behavioral simulation waveform – module dco

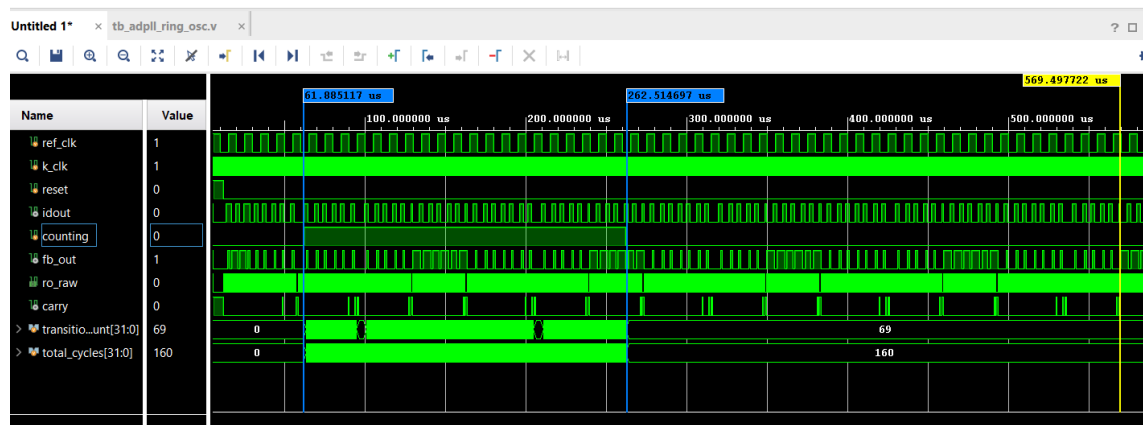
Phân tích kết quả mô phỏng, sau khi kết thúc xung reset ban đầu, mạch bước vào trạng thái dao động tự do (từ 0,1 μs đến 5,0 μs) do tín hiệu điều khiển carry duy trì ở mức 0. Trong giai đoạn này, vòng lặp 9 cổng NOR chạy tự do khiến tín hiệu ro_raw dao động với tần số cực cao, đồng thời DFF liên tục lấy

mẫu tạo ra ngõ ra idout dao động đồng bộ theo nhịp clock. Tại thời điểm đúng $5,0\ \mu\text{s}$, tín hiệu carry lên mức 1, vòng dao động lập tức bị ngắt khiến cả ro_raw và idout đều ngừng chuyển mức và chốt giữ trạng thái hiện tại. Ngay khi carry trở về mức 0, mạch lập tức giải phóng và khôi phục trạng thái dao động.

Kết luận, kết quả này khẳng định module DCO hoạt động hoàn toàn chính xác: mạch vừa sinh được nguồn entropy dồi dào qua ro_raw khi chưa khóa pha (carry = 0), vừa đáp ứng tốt chức năng khóa mạch khi cần thiết (carry = 1).

4.2.4 ADPLL_RO

Kế tiếp, chúng tôi tiến hành mô phỏng module adpll_ring_osc, là một vòng ADPLL hoàn chỉnh dùng làm nguồn sinh entropy. Module tích hợp đầy đủ bốn thành phần kết nối vòng kín gồm phase_detector, k_counter, dco và bộ đếm chia $N/2$. Kết quả mô phỏng waveform thu được như Hình 4.8.



Hình 4.8: Behavioral simulation waveform – module adpll_ring_osc

Phân tích waveform module adpll_ro (fref = 100 kHz, k_clk = Mfref = 800 kHz):

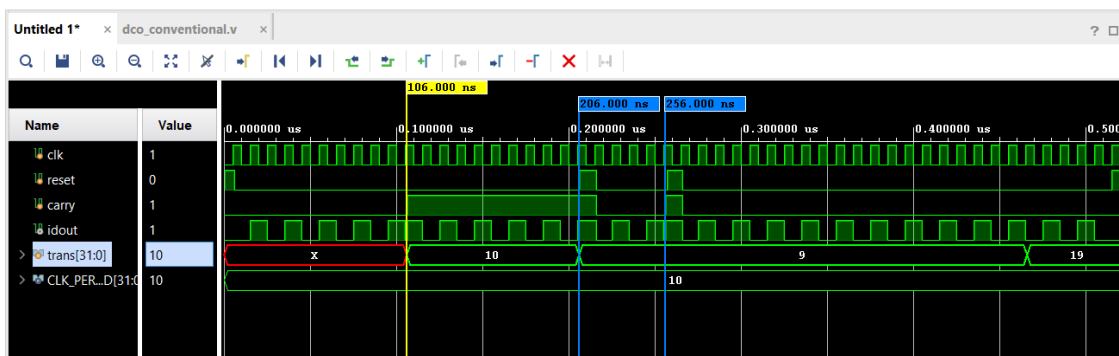
Sau khi reset = 0, k_counter bắt đầu chạy và đẩy tín hiệu carry nhô từng nhịp để kích DCO; chuỗi 9 cổng NOR dao động, thể hiện qua ro_raw, DFF lấy mẫu ro_raw theo k_clk cho ra idout dao động liên tục (ngõ ra đã đồng bộ). Tín hiệu fb_out (T-FF chia tần $\div N/2$) toggle đều và quay về phase_detector, khép kín vòng khóa pha.

Tín hiệu counting lên mức 1 (khoảng 60us đến 260us) sau khoảng chờ ổn định của testbench; từ đó bộ đếm transition mới bắt đầu tăng đều trong cửa sổ quan sát. Kết quả định lượng của testcase: total_cycles = 160, transitions = 69, mật độ chuyển mức density = 0,4313. Giá trị mật độ gần 0,5 (lý tưởng) cho thấy idout chuyển mức gần như mỗi nửa chu kỳ lấy mẫu, DCO ring dao động bình thường, không bị đứng.

Kết luận: module adpll_ring_osc được xác nhận hoạt động đúng nguyên lý vòng khóa pha với DCO là ring oscillator. Waveform thể hiện trọn vẹn ba mắt xích: nguồn dao động (ro_raw), lấy mẫu đồng bộ (idout), và hồi tiếp chia tần (fb_out) quay về phase_detector khép kín vòng; nhịp kích DCO do carry điều khiển. idout dao động ổn định (density $\approx 0,43$) xác nhận nguồn entropy hoạt động đúng về mặt cấu trúc.

4.2.5 DCO_conventional

Module dco_conventional được mô phỏng nhằm kiểm chứng hoạt động của bộ dao động điều khiển số sử dụng trong khối Conventional ADPLL phục vụ quá trình tạo xung lấy mẫu. Kết quả waveform thu được như Hình 4.9.



Hình 4.9: Behavioral simulation waveform – module dco_conventional

Phân tích waveform (clk chu kỳ 10 ns; tín hiệu trans là số transition do task count_transitions đo được): Thời từ 0ns đến 6ns (reset = 1), idout = 0, các T-FF bị xóa, trans = X vì chưa gọi đo.

Thời điểm từ 6ns-105ns: khi carry = 0, idout bám theo q1 (output T-FF1), dao động chia đôi clk. Đo được 10 transition/10 cycle, idout = clk/2.

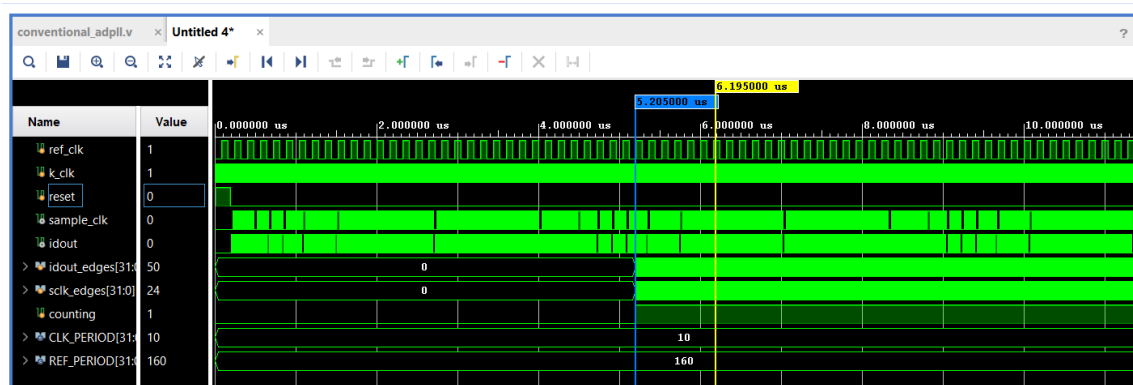
Tại $t = 106 \text{ ns}$ ($\text{carry} = 1$): idout chuyển sang bít q2 (output T-FF2 ngược pha so với q1). Đo được 9 transition / 10 cycle (lệch 1 vì carry thay đổi lệch ở biên tránh bị trùng).

Tại $t = 206 \text{ ns}$: khi có tín hiệu reset được bật thì carry về mức 0 và kéo theo idout đổi pha về q1, xác nhận carry là tín hiệu chọn nguồn/điều khiển pha của DCO.

Quan sát chung: idout là chuỗi xung tuần hoàn đều, chỉ đổi pha tại thời điểm carry chuyển. Kết luận: module dco_conventional hoạt động đúng thiết kế, sinh xung IDout chia đôi clock và đổi pha theo Carry.

4.2.6 ADPLL_conventional

Chúng tôi tiến hành mô phỏng module conventional_adpll, vốn đảm nhiệm chức năng tạo tần số lấy mẫu đồng bộ cho toàn bộ hệ thống MURO-TRNG. Hệ thống được khảo sát tại ba tần số tham chiếu $f_0 = 6,25 \text{ MHz}$, $12,5 \text{ MHz}$ và 25 MHz . Kết quả waveform thu được như Hình 4.10.



Hình 4.10: Behavioral simulation waveform – module conventional_adpll ($f_0 = 6.25 \text{ MHz}$)

Phân tích waveform module conventional_adpll (lần chạy $f_0 = 6.25 \text{ MHz}$; $\text{CLK_PERIOD} = 10$, $\text{REF_PERIOD} = 160$):

Giai đoạn reset: khi $\text{reset} = 1$, idout và sample_clk được giữ ở mức 0; idout_edges = sclk_edges = 0. Khởi khởi tạo đúng trạng thái ban đầu.

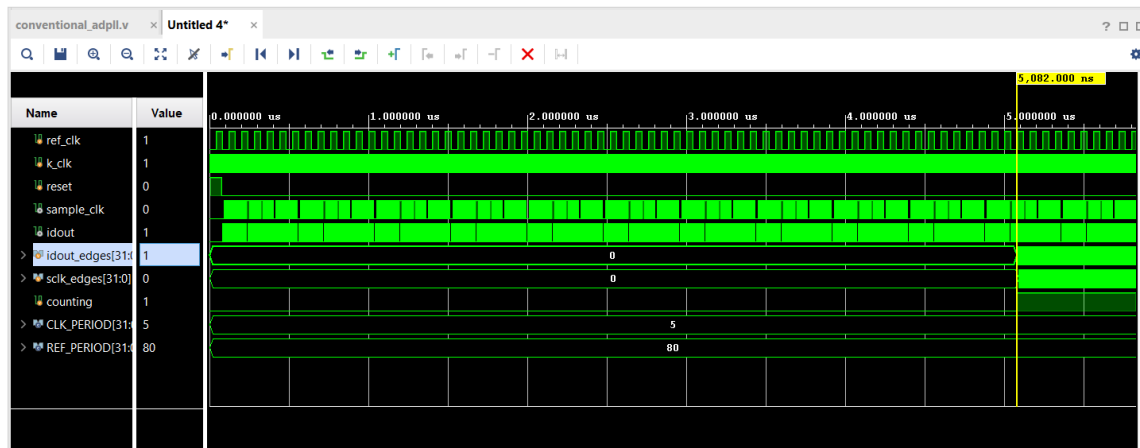
Giai đoạn ổn định/warm-up ($t \approx 0.1\text{--}5\ \mu\text{s}$): ngay sau khi nhả reset, idout và sample_clk đã dao động liên tục. Tuy nhiên hai bộ đếm idout_edges và sclk_edges vẫn bằng 0 trong suốt đoạn này vì tín hiệu counting còn ở mức 0. Đây là khoảng warm-up để chờ vòng ADPLL vào trạng thái dao động ổn định, tránh đếm nhầm các transition nhiều lúc khởi động.

Giai đoạn đo ($t \approx 5.2\ \mu\text{s}$ trở đi): counting lên mức 1, các bộ đếm bắt đầu tăng. idout dao động ở khoảng 50 MHz, tức chu kỳ 20 ns. Trong 1 ms, về lý thuyết idout có khoảng 100.000 cạnh ($1.000.000\ \text{ns} \div 20\ \text{ns} \times 2\ \text{cạnh/chu kỳ}$). Tuy nhiên testbench đếm bằng cách lấy mẫu idout theo cạnh lên của k_clk (mỗi 10 ns một lần), nên mỗi lần lấy mẫu chỉ bắt được trung bình một nửa số lần chuyển mức. Vì vậy bộ đếm cho ra khoảng 50.000 (log thực tế: 50.462 edges/ms) tương đương idout $\approx 50\ \text{MHz} = 8f_0$. sample_clk có tần số bằng idout/2 nên số đếm cũng bằng khoảng một nửa: $\sim 24.000\ \text{edges/ms}$, tương đương $25\ \text{MHz} = 4f_0$.

Kết luận: module conventional_adpll tạo sampling clock đúng thiết kế với $f_0 = 6.25\ \text{MHz}$ ($Mf_0 = 16f_0 = 100\ \text{MHz}$ cấp vào k_clk), idout dao động ổn định ở $Nf_0 = 8f_0 = 50\ \text{MHz}$ và sample_clk = idout/2 = 25 MHz, xác nhận đúng quan hệ chia hai giữa idout và sample_clk. Khởi sinh tần số lấy mẫu đồng bộ cho hệ MURO-TRNG hoạt động chính xác.

Để khảo sát đầy đủ ba tần số tham chiếu theo Bảng 3.2, Chúng tôi mô phỏng thêm module conventional_adpll tại hai điểm vận hành $f_0 = 12,5\ \text{MHz}$ và $f_0 = 25\ \text{MHz}$, với chu kỳ clock đếm Mf_0 (k_clk) tương ứng 200 MHz và 400 MHz.

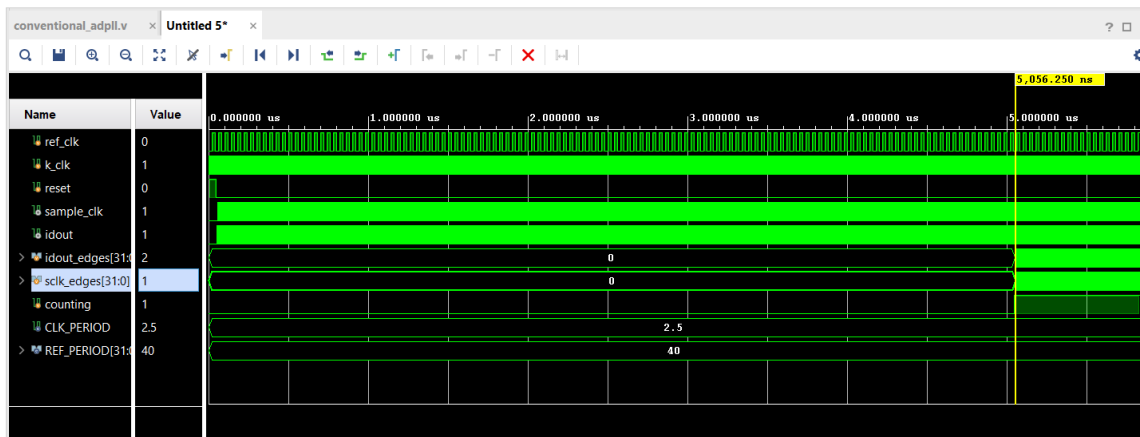
Hình 4.11 mô phỏng module conventional_adpll với các tần số $f_0 = 12.5\text{MHz}$ (CLK_PERIOD = 5, REF_PERIOD = 80)



Hình 4.11: Behavioral simulation waveform – module conventional_adpll ($f_o = 12.5 \text{ MHz}$)

Tại $f_o = 12,5 \text{ MHz}$ ($Mf_o = 200 \text{ MHz}$): idout dao động ở $8f_o = 100 \text{ MHz}$ và $\text{sample_clk} = \text{idout}/2 = 4f_o = 50 \text{ MHz}$; tỉ lệ $\text{idout}/\text{sample_clk} \approx 2,0$. Toàn bộ testcase PASS, kết quả scale đúng gấp đôi so với mức $6,25 \text{ MHz}$.

Hình 4.12 mô phỏng module conventional_adpll với các tần số $f_o = 25 \text{ MHz}$ ($\text{CLK_PERIOD} = 2.5$, $\text{REF_PERIOD} = 400$)



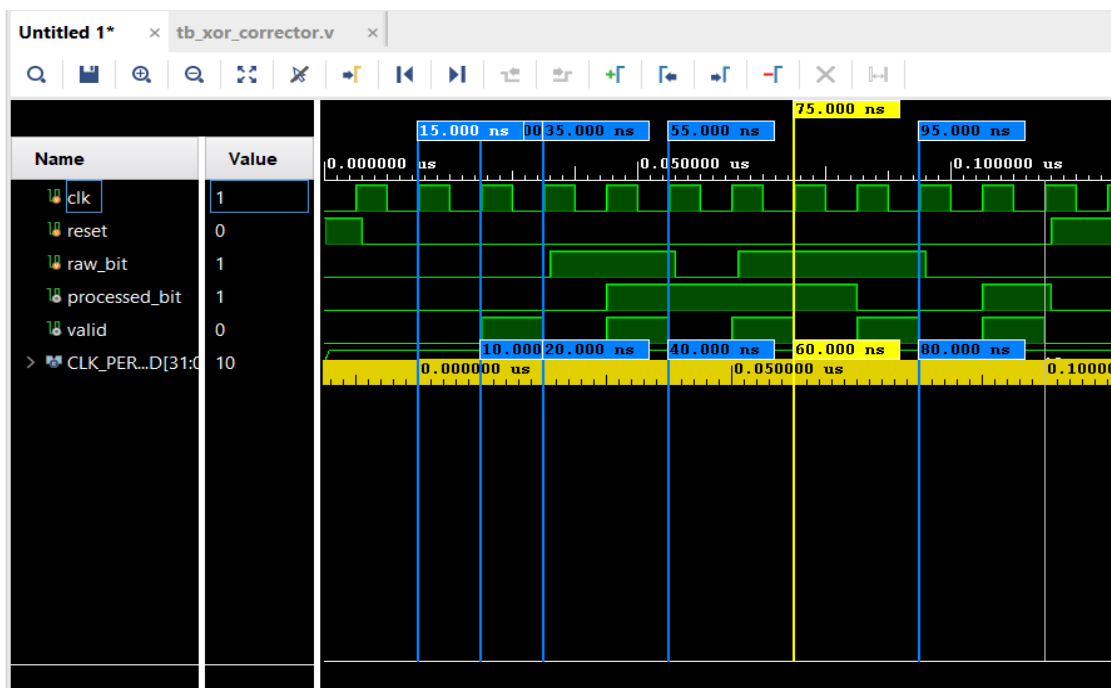
Hình 4.12: Behavioral simulation waveform – module conventional_adpll ($f_o = 25 \text{ MHz}$)

Tại $f_o = 25 \text{ MHz}$ ($Mf_o = 400 \text{ MHz}$): idout dao động ở $8f_o = 200 \text{ MHz}$ và $\text{sample_clk} = 4f_o = 100 \text{ MHz}$; tỉ lệ $\text{idout}/\text{sample_clk} \approx 2,0$. Toàn bộ testcase PASS.

Kết quả ở cả ba điểm vận hành cho thấy quan hệ tần số $idout = Mfo/2$ và $sample_clk = Mfo/4 = idout/2$ luôn được giữ đúng, độc lập với mức f_0 ; điều này xác nhận khối `conventional_adpll` sinh sampling clock chính xác và có thể cấu hình linh hoạt cho ba tần số lấy mẫu của hệ MURO-TRNG (25 / 50 / 100 MHz tương ứng $f_0 = 6,25 / 12,5 / 25$ MHz).

4.2.7 Xor_corrector

Chúng tôi tiến hành mô phỏng module `xor_corrector`. Module được xây dựng từ hai D Flip-Flop mắc nối tiếp tạo thành thanh ghi dịch 2-bit, kết hợp một cổng XOR ở ngõ ra. Tại mỗi sườn lên của sampling clock, cổng XOR thực hiện phép toán logic trên hai bit liền kề trong chuỗi thời gian, qua đó cân bằng lại phân bố xác suất xuất hiện của bit '0' và '1'. Kết quả waveform thu được như Hình 4.13.



Hình 4.13: Behavioral simulation waveform – module `xor_corrector`

Phân tích waveform theo từng mốc thời gian (clk chu kỳ 10 ns):

Thời điểm ban đầu 0-6ns ($reset = 1$): $processed_bit = 0$, $valid = 0$: khởi bị giữ trạng thái ban đầu, không xuất bit rác. Đến khoảng từ 15ns đến 25ns: $reset$

xuống 0, thanh ghi bắt đầu nạp cặp bit thô thứ nhất (0,0), valid lên mức 1 báo hiệu có đủ một cặp bit thô, processed_bit xuất ra = $b_0 \oplus b_1 = 0$.

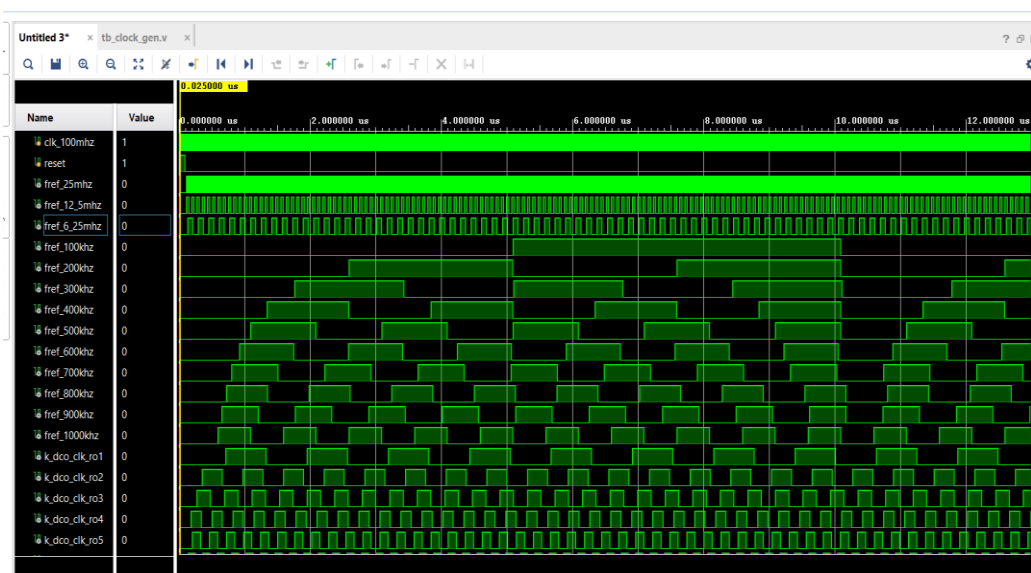
Kế tiếp, tại thời điểm $t \approx 35\text{ns}$: valid lên 1, processed_bit xuất ra = 1 của cặp bit thô thứ hai (0,1).

Tương tự, lần lượt tại từng thời điểm $t \approx 55\text{ns}/75\text{ ns}/95\text{ ns}$: mẫu lặp lại đều đặn, valid nhô lên đúng 1 nhịp mỗi khi có đủ một cặp bit thô, processed_bit cập nhật bằng XOR của cặp bit liền kề.

Quan sát chung: valid là chuỗi xung tuần hoàn, độ rộng đúng 1 chu kỳ; processed_bit chỉ đổi giá trị tại sườn valid và luôn đồng bộ với clk, không có glitch. Kết luận: module xor_corrector thực hiện chính xác phép XOR trên hai bit thô liên tiếp, valid đồng bộ đúng nhịp, khối hậu xử lý khử bias hoạt động đúng thiết kế.

4.2.8 Clock_gen

Mô phỏng module clock_gen, module thực hiện chia tần để tạo ra các tần số tham chiếu cho 10 ADPLL ring oscillator (từ 100 KHz đến 1000 KHz) và ba mức tần số tham chiếu của conventional ADPLL (25 MHz, 12,5 MHz và 6,25 MHz). Kết quả waveform thu được như Hình 4.14.



Hình 4.14: Behavioral simulation waveform – module clock_gen

Ba tín hiệu `fref_25mhz`, `fref_12_5mhz`, `fref_6_25mhz` là ba tần số tham chiếu cấp cho conventional ADPLL. Quan sát waveform cho thấy ba tần số phản ánh đúng quan hệ chia tần 1:2:4.

Các tín hiệu `fref_100khz` đến `fref_1000khz` là mười tần số tham chiếu cấp lần lượt cho RO1 đến RO10. Chu kỳ của từng tín hiệu giảm dần đều từ `fref_100khz` (chu kỳ lớn nhất, xung rộng nhất) đến `fref_1000khz` (chu kỳ nhỏ nhất), xác nhận bộ chia tần hoạt động đúng theo thông số thiết kế.

Các tín hiệu `k_dco_clk_ro1` đến `k_dco_clk_ro10` là các tần số xung nhịp cấp cho K counter và DCO của từng ring oscillator, được tính bằng Mfo với $M = 8$. Chu kỳ của các tín hiệu này nhỏ hơn 8 lần so với `fref` tương ứng, phản ánh đúng hệ số nhân $M = 8$ theo thiết kế.

Kết quả mô phỏng xác nhận module `clock_gen` hoạt động đúng chức năng thiết kế.

4.2.9 MURO-TRNG

Chúng tôi tiến hành mô phỏng module `muro_trng_axi_stream`, tích hợp toàn bộ các module thành phần bao gồm khối tạo xung nhịp (`clock_gen`), 10 khối tạo entropy (`adpll_ring_osc`), khối tạo xung lấy mẫu (`conventional_adpll`), khối hậu xử lý (`xor_corrector`) và khối đóng gói dữ liệu AXI-Stream. Mục đích của phép thử này nhằm kiểm chứng sự hoạt động đồng bộ và luồng dữ liệu (`data path`) của toàn hệ thống từ khâu sinh entropy đến khâu xuất dữ liệu đầu ra.



Hình 4.15: Behavioral simulation waveform – module MURO-TRNG top-level
(tb_muro_trng_top)

Quan sát waveform (Hình 4.15), phân tích theo từng lớp chức năng module MURO-TRNG:

Clock và reset: Các tín hiệu clock hệ thống (clk_100mhz, clk_400mhz) cấp nguồn ổn định. Khối conventional_adpll sinh ra xung sample_clk đều đặn ngay sau khi hệ thống thoát khỏi trạng thái reset.

Conventional ADPLL: với tần số tham chiếu $f_0 = 25 \text{ MHz}$, $k_{\text{clk}} = Mf_0 = 16f_0 = 400 \text{ MHz}$, dco_out dao động ở $Nf_0 = 8f_0 = 200 \text{ MHz}$, carry nhô từng nhịp kích DCO, fb_out chia tần hồi tiếp, và $\text{sample_clk} = \text{dco_out}/2 = 100 \text{ MHz}$ dao động đều. Xung lấy mẫu sample_clk đạt giá trị đúng với thiết kế, cho thấy conventional ADPLL đã hoạt động đúng với yêu cầu.

Nguồn entropy: Hệ thống sử dụng 10 khối dao động, đại diện trong waveform là ro1 chạy ở tần số tham chiếu $f_{ref} = 100 \text{ kHz}$. Tín hiệu ro_raw dao động tự do thành một dải tín hiệu dày đặc, chuyển mức liên tục, chập chòn và không đồng bộ với bất kỳ xung nhịp nào, đáp ứng đúng yêu cầu tạo ra nguồn entropy thực sự. Các tín hiệu nội bộ (ref_clk, k_clk, carry, idout, fb_out): Đây là các tín hiệu điều khiển của mạch ADPLL (vòng khóa pha). Chúng giúp duy trì sự dao động ổn định của khối RO, biến đổi chậm theo chu kỳ của tần số tham chiếu 100 kHz.

Lấy mẫu và giải Metastable (Metastability): Vì ro_raw dao động tốc độ cao và hoàn toàn bất đồng bộ, việc dùng sample_clk để lấy mẫu có rủi ro rất lớn khiến DFF rơi vào trạng thái lơ lửng (metastable). Để giải quyết, mỗi tín hiệu ro_raw bị ép đi qua bộ đồng bộ hai tầng (Two-stage Synchronizer: ro_s1 chốt trạng thái, sau đúng 1 chu kỳ sample_clk chuyển tiếp sang ro_s2 để ổn định tuyệt đối). Sau đó, 10 bit từ 10 RO được đưa qua cổng XOR để trộn nhiễu thành raw_bit. Tín hiệu raw_bit trên waveform xuất hiện với các mức logic 0 và 1 đan xen rõ ràng, không chứa vạch đỏ bất định (X) hay các xung rác (glitch). Điều này xác nhận cơ chế giải metastable hoạt động hoàn hảo, jitter từ 10 nguồn RO độc lập đã được số hóa và khuếch tán thành công thành chuỗi bit thô an toàn, sẵn sàng cho khâu hậu xử lý.

xor_corrector: Khối lấy raw_bit theo sample_clk: bit_phase lật xen kẽ để gom cặp hai bit liên tiếp, first_bit giữ bit thứ nhất, processed_bit = first_bit $\oplus$ bit thứ hai, và valid nhô đúng một nhịp cho mỗi cặp. Cơ chế XOR corrector hoạt động đúng như đã kiểm chứng riêng.

Gom 32-bit và đóng gói AXI-Stream: Trong miền clk_100mhz, mỗi trng_valid dịch một bit vào shift_reg, bit_count đếm 00→1F. Khi đủ 32 bit, m_axis_tdata nhận word mới và m_axis_tvalid lên 1; bắt tay TVALID/TREADY với phía nhận diễn ra đúng, khi tready=1 và tvalid=1 thì word được lấy và tvalid hạ xuống. packet_cnt đếm tới 255 để phát m_axis_tlast đóng gói. Quan sát m_axis_tdata: ban đầu là 0, qua một vùng X ngắn (warm-up khi vài bit rìng chưa

xác định), sau đó xuất các word giá trị xác định (ví dụ 1742356682), xác nhận đường gom và đóng gói dữ liệu 32-bit hoạt động đúng.

Kết luận: Behavioral simulation của top module xác nhận toàn bộ chuỗi khối: clock_gen → 10 ring oscillator → lấy mẫu → XOR → xor_corrector → gom 32-bit → AXI-Stream, liên thông và hoạt động đúng end-to-end, sinh được word dữ liệu 32-bit hợp lệ với bắt tay TVALID/TREADY và đóng gói TLAST chính xác. Mọi khối khởi tạo đúng khi reset và đồng bộ đúng giữa các miền clock. Điều này khẳng định việc tích hợp các module con là chính xác.

4.3 KẾT QUẢ TỔNG HỢP

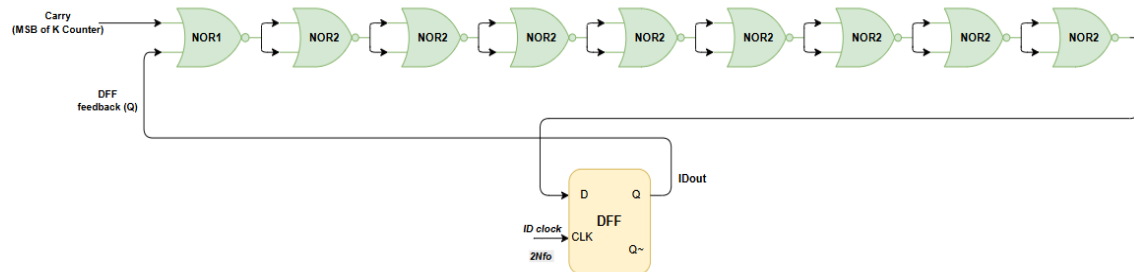
Trong quá trình triển khai và kiểm thử, chúng tôi nhận thấy kết quả NIST SP800-22 không đạt yêu cầu và tỉ lệ bit 0/1 bị lệch đáng kể. Chúng tôi tiến hành phân tích sâu và phát hiện nguyên nhân xuất phát từ lỗi kiến trúc trong thiết kế DCO của [1]. Phần này trình bày quá trình phân tích, giải pháp cải tiến và kết quả đạt được sau khi chúng tôi áp dụng cải tiến.

4.3.1 Phát hiện lỗi kiến trúc trong DCO

Chúng tôi quan sát thấy kết quả NIST fail ở nhiều test và tỉ lệ bit 1 trong raw bitstream cao hơn bình thường (~51%). Để truy tìm nguyên nhân, chúng tôi tiến hành phân tích lại kiến trúc DCO (Digital Controlled Oscillator) – khối sinh jitter cốt lõi của MURO-TRNG.

Theo thiết kế trong [1], DCO gồm 9 cổng NOR nối thành chuỗi và 1 DFF. Tuy nhiên, chúng tôi nhận thấy trong thiết kế này, DFF được đặt nằm trực tiếp trong vòng phản hồi (feedback loop) của chuỗi NOR, với clock là $2N_{fo}$. Điều này dẫn đến một lỗi kiến trúc nghiêm trọng: vòng dao động bị khoá bởi clock $2N_{fo}$ thay vì chạy tự do. Hệ quả là tần số dao động bị cố định theo clock $2N_{fo}$, mỗi cạnh lên của clock, DFF chốt lại trạng thái hiện tại và đưa ngược về đầu vào, khiến vòng lặp chỉ có thể chuyển trạng thái đúng theo nhịp clock. Hệ quả là: Không tích lũy được jitter pha tự nhiên, tần số dao động bị cố định theo $2N_{fo}$ thay vì phụ thuộc vào trễ lan truyền vật lý của các cổng NOR, ngõ ra idout trở

thành tín hiệu tuần hoàn, có thể dự đoán được – đây chính là nguyên nhân khiến các bài test trong NIST thất bại hoàn toàn.



Hình 4.16: Kiến trúc DCO theo thiết kế gốc [1] – DFF nằm trong feedback loop, clock $2Nf_o$

4.3.2 Giải pháp cải tiến

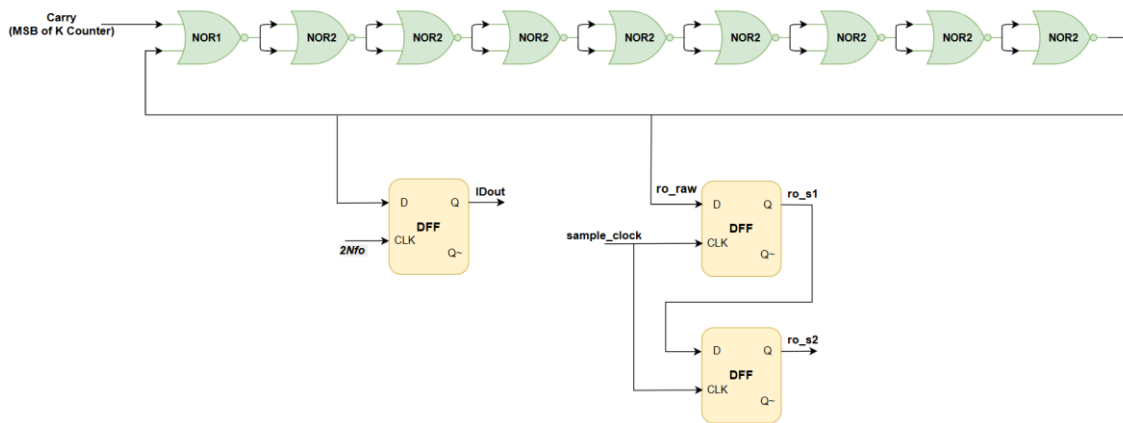
Để khắc phục lỗi kiến trúc trên, chúng tôi đề xuất tách vòng phản hồi của chuỗi NOR thành một dây tín hiệu độc lập có tên `ro_raw`. Cụ thể, ngõ ra của cổng NOR thứ 9 được kết nối trực tiếp trở về ngõ vào của cổng NOR thứ 1 thành một vòng lặp thuần tổ hợp (purely combinational loop), cho phép mạch dao động tự do với tần số chỉ phụ thuộc vào trễ lan truyền thực tế của các cổng NOR trên FPGA.

Lúc này, DFF của bộ DCO được đưa ra ngoài vòng lặp và chỉ đóng vai trò lấy mẫu đồng bộ tại mỗi sườn lên của xung nhịp $2Nf_o$. DFF chốt lại giá trị của `ro_raw` để tạo ra tín hiệu `idout`. Tín hiệu `idout` này hoàn toàn không tham gia vào quá trình dao động sinh nhiễu, mà chỉ đóng vai trò làm tín hiệu phản hồi ổn định để duy trì hoạt động của vòng khóa pha ADPLL.

Đồng thời, tín hiệu tổ hợp tại node cuối chuỗi NOR (`ro_raw`) được trích xuất ra ngoài làm nguồn entropy độc lập thực sự cho mạch TRNG. Tuy nhiên, việc sử dụng một xung nhịp chậm (`sample_clk`) để đi lấy mẫu một tín hiệu dao động tự do siêu tốc và bất đồng bộ như `ro_raw` sẽ chắc chắn dẫn đến rủi ro bất định (metastability). Nếu trạng thái điện áp lơ lửng này được đưa thẳng vào cổng

XOR phía sau, kết quả logic sẽ không xác định và sẽ gây lan truyền lỗi làm sập toàn bộ hệ thống số phía sau.

Để khắc phục, chúng tôi áp dụng kỹ thuật two-stage synchronizer tiêu chuẩn trong thiết kế số: tín hiệu `ro_raw` của từng RO không đưa thẳng vào XOR mà đi qua hai tầng DFF nối tiếp (`ro_s1` đến `ro_s2`), cả hai cùng sử dụng `sample_clk`. Tầng thứ nhất (`ro_s1`) lấy mẫu `ro_raw` tầng này có thể metastable. Tầng thứ hai (`ro_s2` cho `ro_s1` thêm đúng một chu kỳ `sample_clk` để ổn định hoàn toàn về mức 0 hoặc 1 trước khi tín hiệu được đưa vào cổng XOR tổng hợp bit entropy. Hình 4.17 minh họa kiến trúc DCO sau khi chúng tôi áp dụng cải tiến.



Hình 4.17: Kiến trúc DCO sau cải tiến

4.3.3 Kết quả sau khi áp dụng cải tiến

Sau khi chúng tôi áp dụng cải tiến kiến trúc DCO, hệ thống được tổng hợp lại, nạp bitstream xuống PYNQ-Z1 và thu thập lại dữ liệu. Kết quả kiểm thử cho thấy sự cải thiện rõ rệt.

Bảng 4.1: Tỷ lệ bit 0/1 trước và sau cải tiến kiến trúc DCO

Tần số f_0	Tỷ lệ bit 1 (trước cải tiến) %	Tỷ lệ bit 1 (sau cải tiến) %
6,25 MHz	50.7639	49.9997
12,5 MHz	50.8012	50.0035
25 MHz	50.8246	50.0033

Chúng tôi tiến hành kiểm thử NIST SP800-22 đầy đủ 15 test trên 200.000.000 bit đầu ra cho cả ba tần số tham chiếu. Kết quả được trình bày trong Bảng 4.2 đến 4.4.

Bảng 4.2: Kết quả NIST SP800-22 – MURO-TRNG tại $f_0 = 6,25$ MHz

NIST Test	P-value	Kết quả
Frequency (Monobit) Test	0.311542	Pass
Block Frequency Test	0.484646	Pass
Runs Test	0.001156	Pass
Longest Run of Ones Test	0.392456	Pass
Binary Matrix Rank Test	0.311542	Pass
Discrete Fourier Transform (DFT) Test	0.186566	Pass
Non-overlapping Template Matching Test	0.585209	Pass
Overlapping Template Matching Test	0.021262	Pass

NIST Test	P-value	Kết quả
Maurer's Universal Statistical Test	0.010606	Pass
Linear Complexity Test	0.242986	Pass
Serial Test	0.875539	Pass
Approximate Entropy Test	0.689019	Pass
Cumulative Sums Test (Forward)	0.105618	Pass
Cumulative Sums Test (Backward)	0.392456	Pass
Random Excursions Test	0.689019	Pass

Bảng 4.3: Kết quả NIST SP800-22 – MURO-TRNG tại $f_0 = 12,5$ MHz

NIST Test	P-value	Kết quả
Frequency (Monobit) Test	0.484646	Pass
Block Frequency Test	0.141256	Pass
Runs Test	0.585209	Pass
Longest Run of Ones Test	0.484646	Pass
Binary Matrix Rank Test	0.105618	Pass
Discrete Fourier Transform (DFT) Test	0.021262	Pass
Non-overlapping Template Matching	0.788728	Pass

Test		
Overlapping Template Matching Test	0.585209	Pass
Maurer's Universal Statistical Test	0.015065	Pass
Linear Complexity Test	0.392456	Pass
Serial Test	0.585209	Pass
Approximate Entropy Test	0.003577	Pass
Cumulative Sums Test (Forward)	0.029796	Pass
Cumulative Sums Test (Backward)	0.689019	Pass
Random Excursions Test	0.689019	Pass

Bảng 4.4: Kết quả NIST SP800-22 – MURO-TRNG tại $f_0 = 25$ MHz

NIST Test	P-value	Kết quả
Frequency (Monobit) Test	0.484646	Pass
Block Frequency Test	0.484646	Pass
Runs Test	0.186566	Pass
Longest Run of Ones Test	0.186566	Pass
Binary Matrix Rank Test	0.392456	Pass
Discrete Fourier Transform (DFT) Test	0.105618	Pass

Non-overlapping Template Matching Test	0.000004	Fail
Overlapping Template Matching Test	0.392456	Pass
Maurer's Universal Statistical Test	0.010606	Pass
Linear Complexity Test	0.057146	Pass
Serial Test	0.078086	Pass
Approximate Entropy Test	0.242986	Pass
Cumulative Sums Test (Forward)	0.392456	Pass
Cumulative Sums Test (Backward)	0.392456	Pass
Random Excursions Test	0.534146	Pass

Từ kết quả trên, chúng tôi có nhận xét như sau:

Tại $f_0 = 6,25$ MHz và $f_0 = 12,5$ MHz, toàn bộ 15/15 test đều đạt (P-value $\geq 0,001$), xác nhận bitstream đạt tiêu chuẩn ngẫu nhiên mật mã học.

Tại $f_0 = 25$ MHz, Non-overlapping Template Matching Test fail với P-value = 0,000004, đạt 14/15 test. Chúng tôi nhận định nguyên nhân có thể do tương quan giữa sampling clock và tần số dao động của các ring oscillator tại $f_0 = 25$ MHz, khiến một số template ngắn xuất hiện với tần suất bất thường.

4.4 ĐÁNH GIÁ HỆ THỐNG

Chúng tôi tiến hành đánh giá toàn diện hệ thống MURO-TRNG sau cải tiến theo ba khía cạnh: theo nhiệt độ và điện áp, theo tần số tham chiếu, so sánh với kiến trúc gốc và các công trình đã công bố, và đánh giá tổng tài nguyên phần cứng sử dụng.

4.4.1 Theo nhiệt độ và điện áp

Bảng 4.5: Công suất hệ thống theo nhiệt độ và điện áp VDD

Nhiệt độ môi trường °C	Nhiệt độ bên trong °C	VDD=0.95V	VDD=1.0 0V	VDD=1.05V
-20	4.1	1.65 W	1.653 W	1.657 W
0	19.2	1.661 W	1.665 W	1.669 W
25	44.6	1.699 W	1.705 W	1.711 W
50	70.8	1.791 W	1.801 W	1.813 W
60	81.4	1.859 W	1.874 W	1.891 W

Để đánh giá độ ổn định của hệ thống theo điều kiện môi trường, chúng tôi tiến hành khảo sát công suất tiêu thụ theo nhiệt độ môi trường và điện áp tại tần số tham chiếu $f_0 = 12,5$ MHz, kết quả được trình bày trong Bảng 4.5. Dải nhiệt độ môi trường được khảo sát từ -20°C đến 60°C , tương ứng nhiệt độ chip từ $4,1^{\circ}\text{C}$ đến $81,4^{\circ}\text{C}$, toàn bộ nằm trong giới hạn an toàn $0^{\circ}\text{C} - 85^{\circ}\text{C}$ của chip XC7Z020 Commercial. Kết quả cho thấy công suất biến thiên nhẹ theo nhiệt độ và điện áp, hệ thống hoạt động ổn định trong toàn dải khảo sát.

4.4.2 Theo tần số tham chiếu

Chúng tôi đánh giá hệ thống theo ba tần số tham chiếu của conventional ADPLL: $f_0 = 6,25$ MHz, $12,5$ MHz và 25 MHz. Các chỉ tiêu đánh giá bao gồm timing, công suất và throughput, được tổng hợp trong Bảng 4.6.

Bảng 4.6: Kết quả timing, throughput, công suất theo tần số tham chiếu

Thông số	fo = 6,25 MHz	fo = 12,5 MHz	fo = 25 MHz
Setup slack (ns)	2.141	2.348	0.811
Hold slack (ns)	0.02	0.024	0.055
Timing	Met	Met	Met
Throughput (Mbps)	264	220	240
Dynamic power (W)	1.552	1.564	1.681
Static power (W)	0.141	0.141	0.144
Total on-chip power (W)	1.693	1.705	1.825
Energy/bit (nJ/bit)	6.41	7.75	7.6

Từ Bảng 4.6, chúng tôi nhận thấy:

Về timing, cả ba tần số đều đạt yêu cầu về timing. Setup slack tốt nhất thuộc về $f_o = 12,5$ MHz (2,348 ns), tiếp theo là $f_o = 6,25$ MHz (2,141 ns). Tần số $f_o = 25$ MHz có setup slack thấp nhất (0,811 ns), cho thấy timing margin bị thu hẹp hơn khi tần số sampling clock tăng.

Về throughput, $f_o = 6,25$ MHz đạt giá trị cao nhất với 264 Mbps, cao hơn $f_o = 25$ MHz (240 Mbps) và $f_o = 12,5$ MHz (220 Mbps). Điều này có thể giải thích bởi sự khác biệt về tần số DCO output của 10 ring oscillator tại mỗi tần số tham chiếu, ảnh hưởng đến tốc độ lấy mẫu thực tế.

Về công suất tiêu thụ, $f_0 = 6,25$ MHz tiêu thụ ít nhất với tổng công suất 1,693 W (dynamic 1,552 W, static 0,141 W), trong khi $f_0 = 25$ MHz tiêu thụ nhiều nhất với 1,825 W (dynamic 1,681 W, static 0,144 W). Sự tăng công suất dynamic theo tần số là hợp lý do hoạt động switching của các flip-flop trong conventional ADPLL tăng theo tần số.

Về energy/bit, $f_0 = 6,25$ MHz cho kết quả tốt nhất với 6,41 nJ/bit, thấp hơn $f_0 = 25$ MHz (7,6 nJ/bit) và $f_0 = 12,5$ MHz (7,75 nJ/bit). Kết quả này phản ánh đồng thời throughput cao và công suất thấp của $f_0 = 6,25$ MHz.

Nhìn chung, $f_0 = 6,25$ MHz thể hiện trade-off tốt nhất trên FPGA PYNQ-Z1 với throughput cao nhất (264 Mbps), công suất thấp nhất (1,693 W) và energy/bit thấp nhất (6,41 nJ/bit).

4.4.3 Đánh giá hệ thống đang thực hiện so với các hệ thống đã được công bố

Bảng 4.7 trình bày sự so sánh chi tiết giữa hệ thống mà chúng tôi đang thực hiện và các hệ thống TRNG trên FPGA đã được công bố, bao gồm kiến trúc gốc [1] và công trình gần đây [2]. Mục đích của bảng này là đánh giá các thông số kỹ thuật chủ chốt như nền tảng phần cứng, tài nguyên sử dụng, throughput, power và kết quả kiểm thử NIST SP800-22, từ đó nhận định hiệu năng và ưu điểm của hệ thống đề xuất so với các công trình trước đó.

Bảng 4.7: So sánh hệ thống MURO-TRNG với kiến trúc gốc và các công trình liên quan

Tham chiếu	Entropy Source	Device	Hardware Resource	Throughput	Power	Energy/bit
Nhóm cải tiến ($f_0=6,25$ M	ADPLL Jitter	PYNQ Z1 Xilinx	16 LUT 6 FFs	264 Mbps	1.693 W	6.41 (nJ/bit)

Hz)		xc7z020clg400				
Nhóm cải tiến (fo=12,5 MH)	ADPLL Jitter	PYNQ Z1 Xilinx xc7z020clg400	16 LUT 6 FFs	220 Mbps	1.705 W	7.75 (nJ/bit)
Nhóm cải tiến (fo=25M Hz)	ADPLL Jitter	PYNQ Z1 Xilinx xc7z020clg400	16 LUT 6 FFs	240 Mbps	1.825 W	7.6 (nJ/bit)
[1] (fo=6,25 MHz)	ADPLL Jitter	Zynq 7000 Xilinx 7z010c1g225	4 LUTs 4 FFs	206.82 Mbps	1.138 W	5.5 (nJ/bit)
[1] (fo=12,5 MHz)	ADPLL Jitter	Zynq 7000 Xilinx 7z010c1g225	3 LUTs 3 FFs	209.68 Mbps	1.126 W	5.37 (nJ/bit)

[1] (fo=25M Hz)	ADPLL Jitter	Zynq 7000 Xilinx 7z010c lg225	2 LUTs 2 FFs	207.85Mbps	1.116 W	5.3 (nJ/bit)
[2]	Metastabi lity	Xilinx Zynq- 7000 XC7Z0 20	38 LUT 121 DFFs	300 Mbps	119 mW	0.4 (nJ/bit)

Về nền tảng phần cứng, chúng tôi triển khai trên PYNQ-Z1 (XC7Z020) thuộc cùng họ Zynq-7000 với [1], cho phép so sánh trực tiếp kết quả tài nguyên và hiệu năng. Công trình [2] cũng sử dụng cùng XC7Z020 với chúng tôi.

Về công suất và energy/bit, chúng tôi ghi nhận tổng công suất hệ thống trong khoảng 1,693W – 1,825W, cao hơn so với [1] (1,116W – 1,138W). Tuy nhiên, sự chênh lệch này không phản ánh hiệu năng của MURO-TRNG core mà xuất phát từ sự khác biệt về phạm vi đo lường: hệ thống của chúng tôi bao gồm toàn bộ pipeline thu thập dữ liệu gồm PS7, AXI DMA và Gigabit Ethernet, trong khi [1] không công bố rõ cấu hình thu thập dữ liệu tương đương. Riêng MURO-TRNG core chỉ tiêu thụ khoảng 0,007W theo Hierarchical Power Report của Vivado, cho thấy bản thân kiến trúc ADPLL ring oscillator vẫn duy trì ưu thế tiêu thụ tài nguyên và năng lượng cực thấp như [1] đã báo cáo.

Về tài nguyên phần cứng, mỗi khối ADPLL-based ring oscillator trong kiến trúc chúng tôi tiêu tốn 16 LUT và 6 FF, nhỉnh hơn so với [1] (2–4 LUT, 2–4 FF mỗi khối), nhưng vẫn thấp hơn đáng kể so với [2] (38 LUT, 121 FF tổng). Điều này cho thấy kiến trúc ADPLL-based ring oscillator có ưu thế vượt trội về tài nguyên so với [2].

Về throughput, hệ thống chúng tôi đạt throughput trong khoảng 220–264 Mbps, cao hơn [1] (206–209 Mbps) và thấp hơn [2] (300 Mbps).

Về kết quả NIST SP800-22, đây là điểm khác biệt quan trọng nhất: Chúng tôi đạt Pass 15/15 ở 2 tần số 6.25 MHz và 12.5 MHz nhưng ở 25 Mhz chỉ pass 14/15 test sau khi sửa lỗi kiến trúc DCO, trong khi [1] chỉ pass 14/15 test và fail Binary Matrix Rank ở cả ba tần số tham chiếu (P-value $\approx 5,979e-05$) do lỗi kiến trúc DCO được chúng tôi phát hiện và phân tích tại mục 4.3. Công trình [2] pass NIST theo chuẩn SP800-22.

Nhìn chung, hệ thống MURO-TRNG sau cải tiến của chúng tôi thể hiện sự cân bằng tốt giữa tài nguyên thấp, throughput cao và chất lượng ngẫu nhiên đạt chuẩn NIST SP800-22 đầy đủ 15 test (ở tại 6,25 MHz và 12,5 MHz) – cải thiện hơn với [1] nhưng cũng trade-off nhẹ về mặt tài nguyên.

4.4.4 Đánh giá tổng tài nguyên phần cứng

Chúng tôi tổng hợp tài nguyên phần cứng theo từng thành phần của hệ thống, từ MURO-TRNG core đến pipeline thu thập dữ liệu, được trình bày trong Bảng 4.8 dưới đây.

Bảng 4.8: Tài nguyên phần cứng từng thành phần – MURO-TRNG trên PYNQ-Z1 (XC7Z020)

Thành phần	LUT	FF	BRAM
ADPLL Ring Oscillator ($\times 10$)	173	189	0
conventional_adpll	160	60	0
xor_corrector	13	12	0
clock_gen	3	4	0
Logic mức top	45	145	0
Tổng MURO-TRNG	368	410	0

Pipeline thu thập dữ liệu	1735	2389	9.5
Tổng toàn bộ hệ thống	2093	2836	9.5

Bảng 4.8 trình bày kết quả tài nguyên phần cứng của toàn bộ hệ thống sau bước synthesis trên FPGA XC7Z020. Chúng tôi thực hiện tổng hợp ở chế độ giữ nguyên phân cấp và áp thuộc tính DONT_TOUCH lên các khối ring oscillator nhằm bảo toàn chuỗi NOR, tránh công cụ tổng hợp tối ưu hoá làm triệt tiêu nguồn entropy.

Xét riêng lõi MURO-TRNG, mười khối ADPLL-based ring oscillator chiếm tỉ trọng lớn nhất với 173 LUT và 189 FF, mỗi khối tiêu tốn xấp xỉ 16 LUT và 6 FF, thể hiện tính module hoá tốt của thiết kế. Khối conventional_adpll tiêu tốn 160 LUT và 60 FF, phản ánh độ phức tạp của một ADPLL đầy đủ với K counter, DCO và feedback divider. Logic mức top chiếm 45 LUT và 145 FF, trong đó FF cao do các thanh ghi pipeline của FSM và packer 32-bit. Các khối xor_corrector và clock_gen chiếm tài nguyên không đáng kể. Tổng lõi MURO-TRNG tiêu tốn 368 LUT, 410 FF và không sử dụng BRAM, tương đương 0,69% LUT và 0,39% FF của PYNQ Z1 XC7Z020.

Phần còn lại của tài nguyên PL được sử dụng bởi pipeline thu thập dữ liệu bao gồm AXI-Stream FIFO, AXI DMA và AXI Interconnect, trong đó AXI DMA chiếm phần lớn do độ phức tạp của cơ chế truyền dữ liệu trực tiếp giữa PL và DDR RAM, còn AXI-Stream FIFO sử dụng 8,5 BRAM làm bộ đệm dữ liệu. Toàn bộ hệ thống PL sử dụng 2093 LUT, 2836 FF và 9,5 BRAM, tương đương 3,93% LUT, 2,67% FF và 6,79% BRAM của PYNQ Z1 XC7Z020.

Tổng thể, kiến trúc MURO-TRNG đề xuất rất nhẹ về mặt tài nguyên, hoàn toàn phù hợp để tích hợp như một khối IP trong các hệ thống nhúng bảo mật, trong khi phần pipeline thu thập dữ liệu chiếm phần lớn tài nguyên PL còn lại nhưng vẫn đảm bảo toàn bộ hệ thống hoạt động trong giới hạn tài nguyên cho phép của PYNQ Z1 XC7Z020.

4.5 TÓM TẮT CHƯƠNG 4

Trong quá trình phát triển và đánh giá hệ thống MURO-TRNG trên PYNQ-Z1, chúng tôi đã xây dựng một bộ test case toàn diện nhằm đảm bảo tính đúng đắn của từng module, phát hiện và khắc phục lỗi kiến trúc, cũng như xác nhận chất lượng ngẫu nhiên của bitstream đầu ra. Thông qua bảng tổng hợp dưới đây (Bảng 4.9), người đọc có thể nhanh chóng nắm bắt mục tiêu, nội dung và kết quả của từng kiểm thử, từ đó đánh giá đầy đủ hiệu năng và tính chính xác của hệ thống MURO-TRNG sau cải tiến.

Bảng 4.9: Bảng tóm tắt các test case

TT	Test Case	Mô tả	Kết quả
1	Mô phỏng các module trong thiết kế	Tiến hành behavioral simulation lần lượt cho các module trong hệ thống.	Tất cả module hoạt động đúng logic chức năng trên simulation.
2	Thiết kế pipeline thu thập dữ liệu từ phần cứng	Triển khai và kiểm tra pipeline: AXI Wrapper → FIFO → AXI DMA → ARM Cortex-A9 → UDP Ethernet → PC.	Pipeline hoạt động thông suốt, file trng_data.txt được tạo đúng định dạng và đủ dữ liệu để đưa vào kiểm thử NIST SP800-22
3	Triển khai phần cứng và phát hiện vấn đề	Tổng hợp, triển khai và nạp bitstream xuống FPGA; thu thập bitstream thực tế và kiểm tra tỉ lệ bit 0/1, chạy NIST SP800-22 – phát hiện nhiều test NIST fail.	Bitstream thực tế có bias ~51/49 và NIST fail ở nhiều test – xác nhận có vấn đề nghiêm trọng trong kiến trúc không thể phát hiện qua behavioral simulation do simulator

TT	Test Case	Mô tả	Kết quả
			không mô phỏng được jitter vật lý.
4	Phân tích và phát hiện lỗi kiến trúc DCO	Đối chiếu lại kiến trúc DCO trong [1] với lý thuyết ring oscillator – Cải tiến lại kiến trúc DCO.	Xác định được lỗi kiến trúc trong [1]: DFF trong feedback loop làm mất bản chất free-running của ring oscillator, dẫn đến entropy thấp và bias hệ thống.
5	Cải tiến kiến trúc DCO	Tách vòng tổ hợp NOR chain thành tín hiệu ro_raw độc lập triển khai lại lên FPGA và thu thập bitstream mới.	Kiến trúc sau cải tiến hoạt động đúng bản chất free-running oscillator; jitter tích lũy liên tục; tỉ lệ bit 0/1 cân bằng.
6	Kiểm thử NIST SP800-22 đầy đủ 15 test	Chạy bộ kiểm thử NIST SP800-22 trên 200.000.000 bit đầu ra tại ba tần số $f_0 = 6,25$ MHz, 12,5 MHz và 25 MHz sau khi áp dụng cải tiến kiến trúc DCO.	Đạt 15/15 test ở $f_0 = 6,25$ MHz/12,5 MHz; đạt 14/15 test ở $f_0 = 25$ MHz.
7	Đánh giá hiệu năng theo tần số tham chiếu	Đo và so sánh timing, throughput, power và energy/bit tại ba tần số $f_0 = 6,25$ MHz, 12,5 MHz và 25 MHz trên PYNQ-	Xác định được tần số tối ưu cho PYNQ-Z1 dựa trên trade-off giữa throughput, tài nguyên và energy/bit.

TT	Test Case	Mô tả	Kết quả
		Z1	
8	So sánh cải tiến và các công trình đã công bố	Đối chiếu các thông số FPGA, LUT, FF, throughput, energy/bit và kết quả NIST với [1] và [2].	Hệ thống đạt NIST Pass 15/15, cải thiện so với [1]; tài nguyên và throughput cạnh tranh tốt với các công trình cùng loại.
9	Đánh giá tổng tài nguyên phần cứng	Kiểm tra số lượng LUT, FF, của từng thành phần trong toàn bộ hệ thống.	MURO-TRNG core tiêu tốn rất ít tài nguyên tương đương [1]; pipeline thu thập dữ liệu chiếm phần lớn tài nguyên hệ thống.

CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này trình bày phân tổng kết toàn bộ quá trình nghiên cứu và triển khai đề tài "Thiết kế và đánh giá bộ tạo số ngẫu nhiên cho ứng dụng mật mã học trên nền tảng FPGA", trong đó chúng tôi đúc kết những kết quả đạt được, những vấn đề kỹ thuật đã phân tích và giải quyết trong suốt thời gian thực hiện. Bên cạnh đó, chúng tôi cũng đề xuất một số định hướng nghiên cứu và phát triển tiếp theo nhằm nâng cao chất lượng hệ thống và mở rộng phạm vi ứng dụng của kiến trúc MURO-TRNG trong thực tiễn.

5.1 KẾT LUẬN

Triển khai trên nền tảng FPGA PYNQ-Z1 (Xilinx XC7Z020, họ Zynq-7000) với toàn bộ RTL được mô tả bằng Verilog, chúng tôi đã hiện thực hoá thành công kiến trúc MURO-TRNG. Hệ thống hoạt động ổn định ở cả ba tần số tham chiếu, sinh ra bitstream ngẫu nhiên liên tục và đạt yêu cầu kiểm thử NIST SP800-22 sau quá trình nghiên cứu và cải tiến. Để phục vụ quá trình thu thập và kiểm thử, chúng tôi đã xây dựng thêm một pipeline truyền dữ liệu từ phần cứng về PC thông qua AXI-Stream kết hợp DMA và Gigabit Ethernet, đã biến một IP Core đơn thuần thành một hệ thống SoC hoàn chỉnh, cho phép khai thác dữ liệu ngẫu nhiên theo thời gian thực phục vụ các ứng dụng bảo mật thực tiễn.

Trong suốt quá trình thực hiện đề tài, chúng tôi đã nghiên cứu và nắm vững nguyên lý hoạt động của kiến trúc MURO-TRNG, bao gồm cơ chế khoá pha của ADPLL, cấu trúc DCO dựa trên chuỗi cổng NOR, nguyên lý tích lũy jitter pha làm nguồn entropy và cách thức lấy mẫu đồng bộ qua DFF. Ngoài ra, nhóm đã tìm hiểu và so sánh các kiến trúc TRNG đã được công bố trong tài liệu khoa học, từ đó hiểu rõ vị trí cũng như ưu điểm nổi bật của kiến trúc ADPLL trong bức tranh tổng thể về TRNG trên FPGA.

Đặc biệt, trong quá trình triển khai và phân tích kết quả, chúng tôi đã phát hiện và khắc phục một lỗi kiến trúc trong thiết kế DCO của [1] vốn là nguyên nhân khiến hệ thống không tích lũy được jitter entropy thực sự. Sau khi áp dụng giải pháp cải tiến, kết quả kiểm thử NIST SP800-22 được cải thiện rõ rệt và vượt

qua [1] về số lượng test đạt chuẩn, xác nhận tính đúng đắn của hướng phân tích mà chúng tôi đề xuất.

5.2 HƯỚNG PHÁT TRIỂN

Trong bối cảnh công nghệ bán dẫn ngày càng phát triển và nhu cầu bảo mật phần cứng ngày càng trở nên cấp thiết trong các hệ thống điện tử hiện đại, bộ sinh số ngẫu nhiên thực sự (TRNG) đang được nghiên cứu và tích hợp sâu hơn vào nhiều nền tảng phần cứng khác nhau. Định hướng nghiên cứu và phát triển tiếp theo của đề tài tập trung vào các mục tiêu sau:

Chuyển đổi sang quy trình thiết kế ASIC: Thực hiện một hướng phát triển tự nhiên từ kết quả của đề tài là chuyển kiến trúc MURO-TRNG từ FPGA sang quy trình thiết kế ASIC theo công nghệ CMOS. Thực hiện đầy đủ các bước Physical Design từ tổng hợp logic đến Place-and-Route, Physical Verification và timing sign-off, nhằm giảm đáng kể diện tích silicon và tiêu thụ năng lượng so với triển khai trên FPGA. Khi được tích hợp trực tiếp vào phân hệ bảo mật (security subsystem) của các SoC hiện đại, kiến trúc dựa trên ADPLL với ưu thế tiêu thụ tài nguyên cực thấp sẽ trở thành lựa chọn phù hợp cho các thiết bị IoT và hệ thống nhúng bảo mật thế hệ mới.

Đáp ứng mật mã hậu lượng tử (Post-Quantum Cryptography): Sự xuất hiện của các thuật toán mật mã hậu lượng tử vừa được NIST chuẩn hoá chính thức như CRYSTALS-Kyber và CRYSTALS-Dilithium đang đặt ra yêu cầu ngày càng cao về chất lượng nguồn entropy. Các thuật toán này phụ thuộc trực tiếp vào tính ngẫu nhiên thực sự trong quá trình sinh khoá và tham số bảo mật, nơi MURO-TRNG có thể đáp ứng tốt hiệu năng.

Ứng dụng trong các tăng tốc trí tuệ nhân tạo (AI Accelerator): Sự phát triển của trí tuệ nhân tạo đang tạo ra nhu cầu mới đối với TRNG khi các hệ thống AI ngày càng yêu cầu nguồn dữ liệu ngẫu nhiên đáng tin cậy phục vụ cho quá trình huấn luyện mô hình và bảo vệ dữ liệu. Một nguồn entropy phần cứng có hiệu năng cao và độ tin cậy cao hoàn toàn có thể đảm nhận vai trò này trên các nền tảng tính toán biên (edge computing) hoặc chip tăng tốc AI.

TÀI LIỆU THAM KHẢO

- [1] H. B. Meitei and M. Kumar, "Design and Implementation of Multiple Ring Oscillator-Based TRNG Architecture by Using ADPLL," *IEEE Access*, vol. 13, pp. 5865–5879, 2025.
- [2] F. Frustaci, F. Spagnolo, S. Perri, and P. Corsonello, "A high-speed FPGA-based true random number generator using metastability with clock managers," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 2, pp. 756–760, Feb. 2023.
- [3] N. D. Thảo, N. T. Hải, N. T. Duy, and V. D. Dũng, *Giáo trình Kỹ Thuật Số*. TP. HCM, Việt Nam: Nhà xuất bản Đại học Quốc Gia TP Hồ Chí Minh, 2019.
- [4] B. Sunar, W. J. Martin, and D. R. Stinson, "A provably secure true random number generator with built-in tolerance to active attacks," *IEEE Trans. Comput.*, vol. 56, no. 1, pp. 109–119, Jan. 2007.
- [5] P.-L. Chen, C.-C. Chung, and C.-Y. Lee, "An all-digital PLL with cascaded dynamic phase average loop for wide multiplication range applications," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Kobe, Japan, May 2005, pp. 4875–4878.
- [6] National Institute of Standards and Technology, *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*, NIST Special Publication 800-22 Rev. 1a, Gaithersburg, MD, USA, 2010.
- [7] AMD/Xilinx, "Zynq-7000 All Programmable SoC Technical Reference Manual," UG585 (v1.13), AMD Xilinx Inc., San Jose, CA, USA, Oct. 2023. [Online]. Available: <https://docs.amd.com/r/en-US/ug585-zynq-7000-TRM>

- [8] AMD/Xilinx, "AXI DMA v7.1 LogiCORE IP Product Guide," PG021 (v7.1), AMD Xilinx Inc., 2022. [Online]. Available: https://docs.amd.com/r/en-US/pg021_axi_dma
- [9] M. Matsumoto and T. Nishimura, "Mersenne Twister: A 623-dimensionally equidistributed uniform pseudo-random number generator," *ACM Trans. Model. Comput. Simul.*, vol. 8, no. 1, pp. 3–30, Jan. 1998.
- [10] C. S. Petrie and J. A. Connelly, "A noise-based IC random number generator for applications in cryptography," *IEEE Trans. Circuits Syst. I, Fundam. Theory Appl.*, vol. 47, no. 5, pp. 615–621, May 2000.
- [11] M. Herrero-Collantes and J. C. Garcia-Escartin, "Quantum random number generators," *Rev. Mod. Phys.*, vol. 89, no. 1, p. 015004, Mar. 2017.
- [12] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed. Boston, MA: Addison-Wesley, 1997.
- [13] I. Koyuncu, H. Şeker, M. Alçın, and M. Tuna, "A novel Dormand-Prince based hybrid chaotic true random number generator on FPGA," *Balkan J. Electr. Comput. Eng.*, vol. 9, no. 1, pp. 40–47, 2021.
- [14] E. Barker and J. Kelsey, "Recommendation for Random Number Generation Using Deterministic Random Bit Generators," NIST SP 800-90A Rev. 1, NIST, Gaithersburg, MD, USA, Jun. 2015. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-90Ar1>